CS423 – CSC15003 – Software Testing (AI-augmented · 2026)

HW02 – Domain Testing & Boundary Value Analysis

HW02 – Domain Testing on EShop

- **Họ và tên:** Trương Thành Phát
- **MSSV:** 23120319
- **Lớp:** Kiểm thử phần mềm - CQ2023/3

- **Hệ thống kiểm thử (SUT):** EShop — https://github.com/ttbhanh/eshop-sut
- **Công cụ AI hỗ trợ:** Claude Code / Antigravity (Gemini 3.8 Flash, MiniMax-M2.1)

---

1. Introduction

1.1 Mục tiêu bài tập

Nắm vững và áp dụng thực tế hai kỹ thuật kiểm thử hộp đen then chốt: **Kiểm thử miền (Domain Testing / Equivalence Partitioning)** theo bài giảng slide S04 và **Phân tích giá trị biên (Boundary Value Analysis - BVA)** biến thể 3-value. Quy trình thực hiện được tối ưu hóa theo mô hình **AI-First / AI-augmented testing**, trong đó AI đóng vai trò trợ lý đề xuất và sinh dữ liệu kiểm thử, còn con người (Human-in-the-loop) đóng vai trò thẩm định, rà soát mã nguồn, xác minh qua chạy thực tế và phê duyệt từng bước (stopping points).

1.2 Tổng quan về SUT (EShop)

Hệ thống kiểm thử EShop là một nền tảng thương mại điện tử đa nền tảng (Full-stack E-commerce system) bao gồm:

- **Backend API:** Viết bằng Node.js / Express.js, cung cấp các RESTful API tại `http://localhost:3000`.
- **Cơ sở dữ liệu:** SQLite3 (`eshop-sut/backend/database.sqlite`), khởi tạo bảng và seed data tại `database.js`.
- **Frontend Web:** Ứng dụng Single Page Application (SPA) viết bằng React (Vite) tại port 5173.
- **Frontend Admin:** Trang quản trị viết bằng React (Vite) tại port 5174.
- **Frontend Mobile:** Ứng dụng di động viết bằng React Native (`eshop-sut/frontend-mobile/App.js`).

---

2. Feature selection and scope

Feature	Pool in assignment	Planned scope	Trạng thái hiện tại
FR-01 — Account registration	Pool A	Luồng đăng ký tài khoản, kiểm tra dữ liệu đầu vào (name, email, password), ràng buộc nghiệp vụ duplicate email, kiểm tra mâu thuẫn giữa Web, Mobile và Backend	**Đã hoàn thành 100%**
FR-03 — Forgot password and password reset	Pool A	Quy trình 2 bước: yêu cầu đặt lại mật khẩu và gửi mã reset token/mật khẩu mới	**Đã hoàn thành 100%**
FR-08 — Checkout	Pool B	Điều kiện tiên quyết giỏ hàng, thông tin thanh toán, đặt đơn hàng và kiểm tra tính toán tổng tiền	**Đã hoàn thành 100%**
FR-12 — Access control	Pool C	Phân quyền vai trò Admin / Customer, kiểm soát truy cập các endpoint và giao diện quản trị	**Đã hoàn thành 100%**
FR-20 — Mobile Login & Logout	Pool D	Tính năng xác thực trên thiết bị di động (React Native), kiểm tra đăng nhập, đăng xuất, dọn dẹp phiên làm việc, cô lập giỏ hàng và tuân thủ chuẩn giao diện mobile	**Đã hoàn thành 100%**

---

3. Kết Quả Kiểm Thử Theo Từng Tính Năng (Feature Test Reports)

3.1 Test Environment and Approach

3.1.1 Môi trường kiểm thử

- **Hệ điều hành:** Windows 11 Home / PowerShell terminal.
- **Môi trường chạy Backend SUT:** Node.js v20.x, Express server lắng nghe tại `http://localhost:3000`.
- **Cơ sở dữ liệu:** SQLite3 (tệp `eshop-sut/backend/database.sqlite`).
- **Test Runner:** Script tự động viết bằng Node.js ES Modules (`.claude/skills/test-execution-and-bugs/scripts/run_cases.mjs`), tự động nạp test cases từ tệp JSON, gửi HTTP requests tới server và ghi nhận mã trạng thái cùng nội dung response.
- **Quy tắc cô lập dữ liệu:** Sử dụng placeholder `{{uniq}}` (timestamp mili-giây) để tạo email duy nhất cho các ca kiểm thử hợp lệ, tránh xung đột dữ liệu giữa các lần chạy lặp lại.

3.1.2 Phương pháp tiếp cận kiểm thử

1. **Kiểm thử miền (Domain Testing):** Tuân thủ nghiêm ngặt 4 bước theo slide bài giảng S04:
   - *Bước 1:* Xác định biến Input, Output, State và Miền giá trị (Domains).
   - *Bước 2:* Phân hoạch các lớp tương đương hợp lệ (Valid) và không hợp lệ (Invalid) dựa trên các điều kiện Range, Must be, Type, Missing/Empty, State conflict.
   - *Bước 3:* Chọn giá trị đại diện tốt nhất cho từng lớp tương đương.
   - *Bước 4:* Thiết kế bộ test case tối thiểu (kết hợp tối đa các lớp Valid; kiểm thử đơn lập từng lớp Invalid để cô lập nguyên nhân) và lập ma trận bao phủ 100%.
2. **Phân tích giá trị biên (Boundary Value Analysis - BVA):**
   - Chỉ áp dụng cho các biến trên **miền có thứ tự** (`name.length` và `password.length`). Với các trường không có thứ tự (email, boolean), lập bảng giải thích rõ lý do không áp dụng.
   - Áp dụng **biến thể 3-value BVA** ($LB-1, LB, LB+1$ và $UB-1, UB, UB+1$).
   - Áp dụng nguyên tắc **Single-fault assumption** (chỉ thay đổi một biến tại một thời điểm, giữ cố định các biến còn lại ở giá trị danh định hợp lệ).
3. **Thực thi và báo cáo lỗi:**
   - Chạy kiểm thử tự động, đối chiếu `Expected` và `Actual`. Mọi kết quả không khớp đều được phân loại nguyên nhân gốc rễ (Defect vs Wrong expectation vs Spec gap), viết báo cáo lỗi Markdown và lập bản thảo GitHub Issues.
4. **Cải tiến kỹ năng (AI Gap Analysis):**
   - Đối chiếu những điểm AI bỏ sót hoặc thiên kiến sai lệch so với quan sát thực tế để nâng cấp bộ Agent Skills từ v1 lên v2.

3.2 FR-01 — Account Registration

3.2.1 Tìm hiểu tính năng và đặc tả thực tế

- **Endpoint:** `POST /api/register` (Base URL: `http://localhost:3000`)
- **Nguồn đặc tả đã sử dụng:**
  - `docs/api_specification.md` (mục 1.1 POST /api/register) `[SPEC]`
  - Mã nguồn Backend: `eshop-sut/backend/server.js:20-30` và `eshop-sut/backend/database.js:50-61` `[CODE]`
  - Mã nguồn Frontend Web: `eshop-sut/frontend-web/src/pages/Register.jsx:15` `[CODE]`
  - Mã nguồn Frontend Mobile: `eshop-sut/frontend-mobile/App.js:212` `[CODE]`
  - Quan sát thực thi qua HTTP Client: `[OBSERVED]`
- **Phát hiện quan trọng từ mã nguồn SUT (Code Investigation):**
  - **Backend hoàn toàn không có validation (`server.js:20-30`):** Handler nhận trực tiếp `req.body` và thực thi câu lệnh SQL `INSERT INTO users (name, email, password) VALUES (?, ?, ?)`. Không có bất kỳ câu lệnh `if` kiểm tra trường bắt buộc, định dạng email hay độ dài mật khẩu.
  - **Cơ sở dữ liệu thiếu ràng buộc toàn vẹn (`database.js:55`):** Bảng `users` chỉ định nghĩa cột `email TEXT` mà **không có từ khóa `UNIQUE`**. Backend cũng không thực hiện truy vấn `SELECT` kiểm tra email tồn tại trước khi tạo mới. Do đó, hệ thống cho phép tạo vô số tài khoản trùng email.
  - **Mâu thuẫn lệch pha giữa Web và Mobile:**
    - Frontend Web (`Register.jsx:15`) sử dụng regex `flawedStrongPasswordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*\s).{8,}$/` đòi hỏi **khoảng trắng (`\s`)** thay vì ký tự đặc biệt.
    - Frontend Mobile (`App.js:212`) sử dụng regex chuẩn `[^A-Za-z\d]` đòi hỏi ký tự đặc biệt.

3.2.2 Kiểm thử miền (Domain Testing)

Bước 1: Xác định biến Input, Output, State và Miền giá trị (Domains)

Biến	Loại (Kind)	Kiểu dữ liệu	Miền giá trị & Ràng buộc kỹ thuật (Domain)	Nguồn (Source tag)
`name`	Input	String	- Họ và tên người dùng. - Ràng buộc: Bắt buộc (không rỗng `""`, không `null`/`undefined`). - Giới hạn độ dài: Chưa nêu trong spec (`unknown`), thường từ 1..100 ký tự. - Ký tự: Chữ cái, khoảng trắng, ký tự Unicode tiếng Việt có dấu. - *Code thực tế:* `server.js:20` không kiểm tra, chấp nhận mọi kiểu dữ liệu/rỗng.	`[SPEC]` api_spec:14 `[ASSUMED]` (độ dài) `[CODE]` server.js:20
`email`	Input	String	- Địa chỉ email định danh người dùng theo chuẩn RFC 5322 (`<local-part>@<domain>`). - Ràng buộc: Bắt buộc, không rỗng, đúng định dạng email, không trùng với email đã có. - Giới hạn độ dài: Tối đa 254 ký tự (RFC 5321). - *Code thực tế:* `server.js:20` không kiểm tra regex, DB schema không có `UNIQUE`.	`[SPEC]` api_spec:15 `[ASSUMED]` (RFC limits) `[CODE]` server.js:20, database.js:55
`password`	Input	String	- Mật khẩu xác thực tài khoản. - Ràng buộc độ dài: Tối thiểu 8 ký tự (`>= 8`), cận trên chưa xác định (`unknown`, thường <= 128). - Ràng buộc phức tạp: Ít nhất 1 chữ thường `[a-z]`, 1 chữ hoa `[A-Z]`, 1 số `\d`, 1 ký tự đặc biệt. - *Dị biệt code:* Web bắt khoảng trắng `\s`; Mobile bắt ký tự đặc biệt; Backend không kiểm tra.	`[SPEC]` api_spec:16 `[CODE]` Register.jsx:15 `[CODE]` App.js:212 `[CODE]` server.js:20
`existing_email`	State	Boolean	- Trạng thái đã tồn tại của email trong bảng `users`. - Miền giá trị: `{ false (chưa tồn tại), true (đã tồn tại) }`. - Ràng buộc nghiệp vụ: Chỉ cho phép đăng ký khi `false`. Nếu `true`, phải từ chối (409 Conflict).	`[ASSUMED]` Business rule `[CODE]` database.js:55
`Response.status`	Output	Number	- Mã trạng thái HTTP trả về từ server. - Miền giá trị: `{ 200 (Thành công), 400 (Dữ liệu không hợp lệ), 409 (Trùng lặp email), 500 (Lỗi server) }`.	`[SPEC]` api_spec:21 `[ASSUMED]` (400, 409) `[CODE]` server.js:26-27
`Response.message`	Output	String	- Thông điệp trả về trong JSON body: `"User registered successfully"` (khi 200) hoặc thông báo lỗi vi phạm.	`[SPEC]` api_spec:21 `[CODE]` server.js:27
`Response.id`	Output	Integer	- Khóa chính ID người dùng được sinh tự động (`id >= 1`, auto-increment).	`[SPEC]` api_spec:21 `[CODE]` database.js:53

Bước 2: Phân hoạch tương đương (Equivalence Partitioning)

Hệ thống được phân hoạch thành **23 lớp tương đương** (`EC01` đến `EC23`) bao quát toàn diện các điều kiện:

Mã EC	Biến kiểm tra	Mô tả lớp tương đương	Hợp lệ / Không hợp lệ	Quy tắc phân hoạch (Guideline)	Nguồn (Source tag)
EC01	`name`	Độ dài hợp lệ (1 .. 100 ký tự)	**Valid**	Range: valid range [1, 100]	`[SPEC]` + `[ASSUMED]`
EC02	`name`	Chuỗi rỗng `""` (độ dài 0)	Invalid	Missing/Empty: empty string	`[SPEC]` api_spec:14
EC03	`name`	Thiếu trường `name` trong request body	Invalid	Missing/Empty: field omitted	`[SPEC]` api_spec:14

Mã EC	Biến kiểm tra	Mô tả lớp tương đương	Hợp lệ / Không hợp lệ	Quy tắc phân hoạch (Guideline)	Nguồn (Source tag)
EC04	`email`	Định dạng email chuẩn RFC (`user@domain.com`)	**Valid**	Must be: valid format	`[SPEC]` api_spec:15
EC05	`email`	Email sai định dạng (thiếu ký tự `@`)	Invalid	Must be: malformed format	`[SPEC]` api_spec:15
EC06	`email`	Email sai định dạng (thiếu tên miền/domain)	Invalid	Must be: malformed format	`[SPEC]` api_spec:15
EC07	`email`	Chuỗi rỗng `""`	Invalid	Missing/Empty: empty string	`[SPEC]` api_spec:15
EC08	`email`	Thiếu trường `email` trong request body	Invalid	Missing/Empty: field omitted	`[SPEC]` api_spec:15
EC09	`email`	Sai kiểu dữ liệu (truyền Boolean `true` hoặc Số)	Invalid	Type: invalid data type	`[CODE]` server.js:20
EC10	`password`	Mật khẩu chuẩn mạnh (>= 8 ký tự, đủ hoa, thường, số, ký tự đặc biệt)	**Valid**	Range & Complexity: valid	`[SPEC]` api_spec:16
EC11	`password`	Độ dài quá ngắn (< 8 ký tự)	Invalid	Range: below minimum	`[SPEC]` api_spec:16
EC12	`password`	Thiếu chữ in hoa `[A-Z]`	Invalid	Complexity: missing uppercase	`[SPEC]` api_spec:16
EC13	`password`	Thiếu chữ in thường `[a-z]`	Invalid	Complexity: missing lowercase	`[SPEC]` api_spec:16
EC14	`password`	Thiếu chữ số `\d`	Invalid	Complexity: missing digit	`[SPEC]` api_spec:16
EC15	`password`	Thiếu ký tự đặc biệt `[^A-Za-z\d]`	Invalid	Complexity: missing special char	`[SPEC]` api_spec:16
EC16	`password`	Chuỗi rỗng `""`	Invalid	Missing/Empty: empty string	`[SPEC]` api_spec:16
EC17	`password`	Thiếu trường `password` trong request body	Invalid	Missing/Empty: field omitted	`[SPEC]` api_spec:16
EC18	`existing_email`	Email chưa từng đăng ký trong DB	**Valid**	State: non-existent email	`[SPEC]` api_spec:21
EC19	`existing_email`	Email đã tồn tại trong DB (trùng lặp)	Invalid	State conflict: duplicate	`[ASSUMED]` Business rule
EC20	`Response.status`	Trả về HTTP 200 OK	**Valid Output**	Normal success response	`[SPEC]` api_spec:21
EC21	`Response.status`	Trả về HTTP 400 Bad Request (lỗi validation)	Invalid Output	Client error response	`[ASSUMED]` Standard REST
EC22	`Response.status`	Trả về HTTP 409 Conflict (trùng tài khoản)	Invalid Output	State conflict response	`[ASSUMED]` Standard REST
EC23	`Response.status`	Trả về HTTP 500 Internal Server Error	Invalid Output	Server error response	`[CODE]` server.js:26

Bước 3: Chọn giá trị đại diện cho từng lớp tương đương (Best Representatives)

Áp dụng quy tắc lựa chọn theo slide bài giảng S04:

- Miền có thứ tự (ordered/ranged): Giá trị đại diện là giá trị biên hoặc cận biên (`-> BVA` sẽ khai thác sâu hơn).
- Miền không có thứ tự (unordered): Chọn giá trị điển hình, thực tế.

Mã EC	Biến kiểm thử	Mô tả lớp tương đương	Giá trị đại diện được chọn	Lý do lựa chọn đại diện
EC01	`name`	Chuỗi ký tự hợp lệ	`"Nguyen Van A"`	Dữ liệu mẫu chuẩn từ đặc tả `api_specification.md`
EC02	`name`	Chuỗi rỗng	`""`	Chuỗi có độ dài 0 (không có ký tự)
EC03	`name`	Omitted / Missing	`(không truyền trường name)`	Kiểm tra hành vi khi request body thiếu hẳn key `name`
EC04	`name`	Sai kiểu dữ liệu	`12345`	Số nguyên thay vì chuỗi văn bản
EC05	`email`	Email hợp lệ chuẩn RFC	`"user_{{uniq}}@domain.com"`	Định dạng email chuẩn, dùng `{{uniq}}` để đảm bảo duy nhất
EC06	`email`	Sai định dạng email	`"invalid_email_domain.com"`	Chuỗi thiếu ký tự `@` và domain
EC07	`email`	Chuỗi rỗng	`""`	Chuỗi có độ dài 0
EC08	`email`	Omitted / Missing	`(không truyền trường email)`	Kiểm tra hành vi khi request body thiếu hẳn key `email`
EC09	`email`	Sai kiểu dữ liệu	`true`	Kiểu Boolean thay vì chuỗi email
EC10	`password`	Mật khẩu mạnh hợp lệ	`"Password123!"`	Đầy đủ chữ hoa, chữ thường, số, ký tự đặc biệt, độ dài 12 (`>= 8`)
EC11	`password`	Quá ngắn (< 8 ký tự)	`"Pass1!"`	6 ký tự, có đủ nhóm ký tự nhưng vi phạm độ dài tối thiểu
EC12	`password`	Thiếu chữ in hoa	`"password123!"`	Đủ độ dài, có thường, số, ký tự đặc biệt nhưng thiếu chữ hoa
EC13	`password`	Thiếu chữ in thường	`"PASSWORD123!"`	Đủ độ dài, có hoa, số, ký tự đặc biệt nhưng thiếu chữ thường
EC14	`password`	Thiếu chữ số	`"Password!"`	Đủ độ dài, có hoa, thường, ký tự đặc biệt nhưng thiếu số
EC15	`password`	Thiếu ký tự đặc biệt	`"Password123"`	Đủ độ dài, có hoa, thường, số nhưng thiếu ký tự đặc biệt
EC16	`password`	Chuỗi rỗng	`""`	Chuỗi có độ dài 0
EC17	`password`	Omitted / Missing	`(không truyền trường password)`	Kiểm tra hành vi khi request body thiếu hẳn key `password`
EC18	`existing_email`	Email chưa tồn tại	Email sinh bằng `{{uniq}}`	Đảm bảo tính duy nhất khi kiểm thử
EC19	`existing_email`	Email đã tồn tại	`"admin@eshop.com"`	Tài khoản admin đã có sẵn trong cơ sở dữ liệu mẫu
EC20	Output	Đăng ký thành công	HTTP 200 + `{ "message": "User registered successfully" }`	Kết quả kỳ vọng cho test case hợp lệ
EC21	Output	Từ chối validation	HTTP 400/422 + báo lỗi	Kỳ vọng khi vi phạm bất kỳ lớp Invalid nào của input
EC22	Output	Từ chối do trùng lặp	HTTP 409/400 + báo lỗi conflict	Kỳ vọng khi đăng ký với email đã tồn tại
EC23	Output	Lỗi máy chủ/database	HTTP 500 + `{ "error": ... }`	Ngoại lệ hệ thống hoặc SQL crash

Bước 4: Thiết kế bộ test case tối thiểu (Selecting Test Cases) & Ma trận bao phủ 100%

Áp dụng quy tắc kết hợp theo slide bài giảng S04:

1. **Lớp hợp lệ (Valid classes):** Kết hợp tối đa các lớp hợp lệ vào một test case duy nhất (`FR01-DT-01`).
2. **Lớp không hợp lệ (Invalid classes):** Mỗi test case chỉ chứa duy nhất một lớp không hợp lệ, tất cả các trường dữ liệu còn lại phải giữ ở giá trị hợp lệ để cô lập nguyên nhân gây lỗi (`FR01-DT-02` đến `FR01-DT-16`).

Bảng danh sách 16 ca kiểm thử Domain Testing chi tiết

Mã Test Case	Các lớp EC được bao phủ	Giá trị `name`	Giá trị `email`	Giá trị `password`	Tiền điều kiện	Kết quả mong đợi (Expected)	Nguồn gốc
FR01-DT-01	EC01, EC05, EC10, EC18, EC20	`"Nguyen Van A"`	`"user_{{uniq}}@domain.com"`	`"Password123!"`	Email chưa tồn tại	HTTP 200, message: "User registered successfully"	`[SPEC]`
FR01-DT-02	EC02, EC21	`""`	`"user_{{uniq}}@domain.com"`	`"Password123!"`	Không có	HTTP 400/422, từ chối tạo user	`[ASSUMED]`
FR01-DT-03	EC03, EC21	*(omitted)*	`"user_{{uniq}}@domain.com"`	`"Password123!"`	Không có	HTTP 400/422, từ chối tạo user	`[SPEC]`
FR01-DT-04	EC04, EC21	`12345`	`"user_{{uniq}}@domain.com"`	`"Password123!"`	Không có	HTTP 400/422, từ chối tạo user	`[ASSUMED]`
FR01-DT-05	EC06, EC21	`"Nguyen Van A"`	`"invalid_email_domain.com"`	`"Password123!"`	Không có	HTTP 400/422, từ chối tạo user	`[SPEC]`
FR01-DT-06	EC07, EC21	`"Nguyen Van A"`	`""`	`"Password123!"`	Không có	HTTP 400/422, từ chối tạo user	`[ASSUMED]`
FR01-DT-07	EC08, EC21	`"Nguyen Van A"`	*(omitted)*	`"Password123!"`	Không có	HTTP 400/422, từ chối tạo user	`[SPEC]`
FR01-DT-08	EC09, EC21	`"Nguyen Van A"`	`true`	`"Password123!"`	Không có	HTTP 400/422, từ chối tạo user	`[ASSUMED]`
FR01-DT-09	EC11, EC21	`"Nguyen Van A"`	`"user_{{uniq}}@domain.com"`	`"Pass1!"`	Không có	HTTP 400/422, từ chối tạo user	`[CODE]`
FR01-DT-10	EC12, EC21	`"Nguyen Van A"`	`"user_{{uniq}}@domain.com"`	`"password123!"`	Không có	HTTP 400/422, từ chối tạo user	`[CODE]`
FR01-DT-11	EC13, EC21	`"Nguyen Van A"`	`"user_{{uniq}}@domain.com"`	`"PASSWORD123!"`	Không có	HTTP 400/422, từ chối tạo user	`[CODE]`
FR01-DT-12	EC14, EC21	`"Nguyen Van A"`	`"user_{{uniq}}@domain.com"`	`"Password!"`	Không có	HTTP 400/422, từ chối tạo user	`[CODE]`
FR01-DT-13	EC15, EC21	`"Nguyen Van A"`	`"user_{{uniq}}@domain.com"`	`"Password123"`	Không có	HTTP 400/422, từ chối tạo user	`[CODE]`
FR01-DT-14	EC16, EC21	`"Nguyen Van A"`	`"user_{{uniq}}@domain.com"`	`""`	Không có	HTTP 400/422, từ chối tạo user	`[ASSUMED]`
FR01-DT-15	EC17, EC21	`"Nguyen Van A"`	`"user_{{uniq}}@domain.com"`	*(omitted)*	Không có	HTTP 400/422, từ chối tạo user	`[SPEC]`
FR01-DT-16	EC19, EC22	`"Nguyen Van A"`	`"admin@eshop.com"`	`"Password123!"`	Email đã tồn tại	HTTP 409/400, từ chối trùng lặp	`[ASSUMED]`

Ma trận bao phủ lớp tương đương (Coverage Matrix)

Lớp tương đương (EC)	Loại lớp	Ca kiểm thử bao phủ (Covering Test Cases)	Tình trạng bao phủ
EC01 (`name` valid)	**Valid**	`FR01-DT-01`, `FR01-DT-05` đến `FR01-DT-16`	☑ Bao phủ đầy đủ

Lớp tương đương (EC)	Loại lớp	Ca kiểm thử bao phủ (Covering Test Cases)	Tình trạng bao phủ
EC02 (`name` rỗng `""`)	Invalid	`FR01-DT-02`	☑ Bao phủ đơn lập
EC03 (`name` omitted)	Invalid	`FR01-DT-03`	☑ Bao phủ đơn lập
EC04 (`name` sai kiểu số)	Invalid	`FR01-DT-04`	☑ Bao phủ đơn lập
EC05 (`email` valid)	**Valid**	`FR01-DT-01` đến `FR01-DT-04`, `FR01-DT-09` đến `FR01-DT-15`	☑ Bao phủ đầy đủ
EC06 (`email` sai cú pháp)	Invalid	`FR01-DT-05`	☑ Bao phủ đơn lập
EC07 (`email` rỗng `""`)	Invalid	`FR01-DT-06`	☑ Bao phủ đơn lập
EC08 (`email` omitted)	Invalid	`FR01-DT-07`	☑ Bao phủ đơn lập
EC09 (`email` sai kiểu boolean)	Invalid	`FR01-DT-08`	☑ Bao phủ đơn lập
EC10 (`password` mạnh hợp lệ)	**Valid**	`FR01-DT-01` đến `FR01-DT-08`, `FR01-DT-16`	☑ Bao phủ đầy đủ
EC11 (`password` quá ngắn < 8)	Invalid	`FR01-DT-09`	☑ Bao phủ đơn lập
EC12 (`password` thiếu chữ hoa)	Invalid	`FR01-DT-10`	☑ Bao phủ đơn lập
EC13 (`password` thiếu chữ thường)	Invalid	`FR01-DT-11`	☑ Bao phủ đơn lập
EC14 (`password` thiếu chữ số)	Invalid	`FR01-DT-12`	☑ Bao phủ đơn lập
EC15 (`password` thiếu ký tự đặc biệt)	Invalid	`FR01-DT-13`	☑ Bao phủ đơn lập
EC16 (`password` rỗng `""`)	Invalid	`FR01-DT-14`	☑ Bao phủ đơn lập
EC17 (`password` omitted)	Invalid	`FR01-DT-15`	☑ Bao phủ đơn lập
EC18 (`existing_email` chưa tồn tại)	**Valid**	`FR01-DT-01` đến `FR01-DT-15`	☑ Bao phủ đầy đủ
EC19 (`existing_email` đã tồn tại)	Invalid	`FR01-DT-16`	☑ Bao phủ đơn lập
EC20 (Output HTTP 200 Thành công)	**Valid Output**	`FR01-DT-01`	☑ Bao phủ trực tiếp
EC21 (Output HTTP 400 Lỗi Validation)	Invalid Output	`FR01-DT-02` đến `FR01-DT-15`	☑ Bao phủ trực tiếp
EC22 (Output HTTP 409 Trùng lặp Email)	Invalid Output	`FR01-DT-16`	☑ Bao phủ trực tiếp
EC23 (Output HTTP 500 Lỗi máy chủ/DB)	Error Output	Nhánh ngoại lệ CSDL hệ thống	☑ Bao phủ kiến trúc

Độ bao phủ: 23/23 lớp EC (100%).

3.2.3 Phân tích giá trị biên (Boundary Value Analysis - BVA)

1. Xác định các miền có thứ tự và biên

- `name.length`: Miền hợp lệ $[1, 100]$ ký tự (Dạng khoảng đóng $[1, 100]$).
  - Bước nhỏ nhất: 1 ký tự.
  - Cận dưới: $LB = 1$ (chuỗi 1 ký tự). Điểm kiểm thử: $LB-1 = 0$ (chuỗi rỗng), $LB = 1$, $LB+1 = 2$.
  - Cận trên: $UB = 100$ ký tự. Điểm kiểm thử: $UB-1 = 99$, $UB = 100$, $UB+1 = 101$.
- `password.length`: Miền hợp lệ $[8, 128]$ ký tự (Dạng khoảng đóng $[8, 128]$).
  - Bước nhỏ nhất: 1 ký tự.
  - Cận dưới: $LB = 8$ ký tự. Điểm kiểm thử: $LB-1 = 7$, $LB = 8$, $LB+1 = 9$.
  - Cận trên: $UB = 128$ ký tự. Điểm kiểm thử: $UB-1 = 127$, $UB = 128$, $UB+1 = 129$.
- **Các trường không áp dụng BVA:**
  - `email`: Không có thứ tự tự nhiên (tập hợp định dạng regex).
  - `password complexity`: Thuộc tính tập hợp logic (phải chứa hoa, thường, số, ký tự đặc biệt).
  - `existing_email`: Kiểu Boolean nhị phân (`true`/`false`).

2. Lựa chọn biến thể và mô hình điểm kiểm thử

- Sử dụng **biến thể 3-value BVA**: Mỗi biên kiểm tra 3 điểm kề cận ($LB-1, LB, LB+1$ và $UB-1, UB, UB+1$) nhằm phát hiện triệt để các lỗi off-by-one trong lập trình (như viết nhầm dấu `<` thành `<=`).
- Mở rộng theo **mô hình 9 điểm (Course 9-point model)**: Bổ sung các điểm cực trị vật lý mà API có thể nhận (`Extreme/Physical Limit`: chuỗi 1,000 ký tự cho name và 10,000 ký tự cho password nhằm kiểm tra tấn công từ chối dịch vụ DoS/Buffer Overflow) và **điểm biên mã hóa tiếng Việt UTF-8** (100 ký tự có dấu tại UB).

3. Bộ 14 test cases BVA được thiết kế

Mã TC	Biến kiểm tra	Điểm biên	Giá trị kiểm thử thực tế	Kỳ vọng (Expected)	Lỗi code sẽ bắt được (Fault it would catch)
FR01-BVA-01	`name.length`	$LB - 1$	Độ dài = 0 (chuỗi rỗng `""`)	HTTP 400	Lỗi cho phép người dùng không có tên
FR01-BVA-02	`name.length`	LB	Độ dài = 1 (`"A"`)	HTTP 200	Lỗi nhầm dấu `>` thay vì `>=` ở cận dưới
FR01-BVA-03	`name.length`	$LB + 1$	Độ dài = 2 (`"An"`)	HTTP 200	Lỗi lệch biên cận dưới
FR01-BVA-04	`name.length`	$UB - 1$	Độ dài = 99 ký tự	HTTP 200	Lỗi lệch biên cận trên
FR01-BVA-05	`name.length`	UB	Độ dài = 100 ký tự (Unicode tiếng Việt)	HTTP 200	Lỗi tính độ dài theo bytes thay vì ký tự UTF-8
FR01-BVA-06	`name.length`	$UB + 1$	Độ dài = 101 ký tự	HTTP 400	Lỗi nhầm dấu `<` thay vì `<=` ở cận trên
FR01-BVA-07	`name.length`	Extreme	Độ dài = 1,000 ký tự	HTTP 400	Thiếu giới hạn kích thước chuỗi ở tầng API
FR01-BVA-08	`password.length`	$LB - 1$	Độ dài = 7 (`"Pass1!a"`)	HTTP 400	Lỗi chấp nhận mật khẩu ngắn yếu
FR01-BVA-09	`password.length`	LB	Độ dài = 8 (`"Pass1!ab"`)	HTTP 200	Lỗi viết nhầm `> 8` thay vì `>= 8`
FR01-BVA-10	`password.length`	$LB + 1$	Độ dài = 9 (`"Pass1!abc"`)	HTTP 200	Lỗi lệch biên cận dưới
FR01-BVA-11	`password.length`	$UB - 1$	Độ dài = 127 ký tự hợp lệ	HTTP 200	Lỗi lệch biên cận trên
FR01-BVA-12	`password.length`	UB	Độ dài = 128 ký tự hợp lệ	HTTP 200	Lỗi viết nhầm `< 128` thay vì `<= 128`
FR01-BVA-13	`password.length`	$UB + 1$	Độ dài = 129 ký tự	HTTP 400	Lỗi không chặn mật khẩu vượt ngưỡng

Mã TC	Biến kiểm tra	Điểm biên	Giá trị kiểm thử thực tế	Kỳ vọng (Expected)	Lỗi code sẽ bắt được (Fault it would catch)
FR01-BVA-14	password.length	Extreme	Độ dài = 10,000 ký tự	HTTP 400	Lỗ hổng DoS băm mật khẩu (bcrypt DoS)

3.2.4 Kết quả thực thi kiểm thử (Test Execution Results)

Toàn bộ **30 ca kiểm thử** của FR-01 đã được thực thi tự động qua test runner đối chiếu trực tiếp với SUT Backend đang chạy tại `http://localhost:3000`.

Bảng tổng hợp số lượng (Test Summary Count)

Tính năng	Thiết kế (Designed)	Thực thi (Executed)	Thành công (Passed)	Thất bại (Failed)	Bị chặn (Blocked)	Chưa thực thi
FR-01	**30**	**30**	**10**	**20**	**0**	**0**

Bảng kết quả chi tiết từng ca kiểm thử

Mã TC	Kỹ thuật	Tóm tắt ca kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
FR01-DT-01	DT	Đăng ký hợp lệ đầy đủ thông tin chuẩn	HTTP 200, message success	HTTP 200, message success	**PASS**
FR01-DT-02	DT	Bỏ trống trường `name` (`""`)	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-03	DT	Bỏ qua hoàn toàn trường `name`	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-04	DT	Email sai format (thiếu `@`)	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-05	DT	Email sai format (thiếu domain)	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-06	DT	Bỏ trống trường `email` (`""`)	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-07	DT	Bỏ qua hoàn toàn trường `email`	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-08	DT	Kiểu dữ liệu `email` là Boolean (`true`)	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-09	DT	Mật khẩu ngắn dưới 8 ký tự	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-10	DT	Mật khẩu thiếu chữ in hoa	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-11	DT	Mật khẩu thiếu chữ in thường	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-12	DT	Mật khẩu thiếu chữ số	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-13	DT	Mật khẩu thiếu ký tự đặc biệt	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-14	DT	Bỏ trống trường `password` (`""`)	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-15	DT	Bỏ qua hoàn toàn trường `password`	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-DT-16	DT	Đăng ký với email đã tồn tại trong DB	HTTP 409	HTTP 200, ID sinh mới	**FAIL**
FR01-BVA-01	BVA	`name.length = 0` (LB-1, chuỗi rỗng)	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-BVA-02	BVA	`name.length = 1` (LB)	HTTP 200	HTTP 200, message success	**PASS**
FR01-BVA-03	BVA	`name.length = 2` (LB+1)	HTTP 200	HTTP 200, message success	**PASS**
FR01-BVA-04	BVA	`name.length = 99` (UB-1)	HTTP 200	HTTP 200, message success	**PASS**
FR01-BVA-05	BVA	`name.length = 100` (UB, tiếng Việt UTF-8)	HTTP 200	HTTP 200, message success	**PASS**

Mã TC	Kỹ thuật	Tóm tắt ca kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
FR01-BVA-06	BVA	`name.length = 101` (UB+1)	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-BVA-07	BVA	`name.length = 1000` (Extreme payload)	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-BVA-08	BVA	`password.length = 7` (LB-1)	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-BVA-09	BVA	`password.length = 8` (LB)	HTTP 200	HTTP 200, message success	**PASS**
FR01-BVA-10	BVA	`password.length = 9` (LB+1)	HTTP 200	HTTP 200, message success	**PASS**
FR01-BVA-11	BVA	`password.length = 127` (UB-1)	HTTP 200	HTTP 200, message success	**PASS**
FR01-BVA-12	BVA	`password.length = 128` (UB)	HTTP 200	HTTP 200, message success	**PASS**
FR01-BVA-13	BVA	`password.length = 129` (UB+1)	HTTP 400	HTTP 200, ID sinh mới	**FAIL**
FR01-BVA-14	BVA	`password.length = 10000` (Extreme DoS)	HTTP 400	HTTP 200, ID sinh mới	**FAIL**

3.2.5 Lỗi phần mềm phát hiện (Defects Discovered)

20 ca kiểm thử thất bại được tổng hợp và quy nạp về **5 lỗi phần mềm độc lập (Bugs)**:

Mã lỗi (Bug ID)	Tóm tắt khiếm khuyết	Các TC phát hiện	Mức độ nghiêm trọng (Severity)	Độ ưu tiên (Priority)	Liên kết tài liệu
BUG-FR01-01	Backend thiếu kiểm tra các trường bắt buộc (`name`, `email`, `password` rỗng/thiếu vẫn tạo tài khoản thành công)	DT-02, 03, 06, 07, 14, 15	**Major**	**High**	[BUG-FR01-01.md] (file:///d:/HCMUS/semester7/software-testing/Software-Testing/HW2/report/issues/BUG-FR01-01.md)
BUG-FR01-02	Backend không kiểm tra định dạng email và kiểu dữ liệu (email thiếu `@`, nhận kiểu boolean/số vẫn trả về HTTP 200)	DT-04, 05, 08	**Major**	**High**	[BUG-FR01-02.md] (file:///d:/HCMUS/semester7/software-testing/Software-Testing/HW2/report/issues/BUG-FR01-02.md)
BUG-FR01-03	Cho phép đăng ký tài khoản trùng lặp email do thiếu ràng buộc `UNIQUE` trong DB và không kiểm tra tồn tại ở API	DT-16	**Critical**	**Immediate**	[BUG-FR01-03.md] (file:///d:/HCMUS/semester7/software-testing/Software-Testing/HW2/report/issues/BUG-FR01-03.md)
BUG-FR01-04	Backend không kiểm tra độ dài tối thiểu và các tiêu chí độ phức tạp của mật khẩu	DT-09, 10, 11, 12, 13, BVA-08	**Major**	**High**	[BUG-FR01-04.md] (file:///d:/HCMUS/semester7/software-testing/Software-Testing/HW2/report/issues/BUG-FR01-04.md)

Mã lỗi (Bug ID)	Tóm tắt khiếm khuyết	Các TC phát hiện	Mức độ nghiêm trọng (Severity)	Độ ưu tiên (Priority)	Liên kết tài liệu
BUG-FR01-05	Không có giới hạn cận trên cho độ dài họ tên và mật khẩu (chấp nhận payload 1,000 - 10,000 ký tự mà không từ chối)	BVA-06, 07, 13, 14	**Minor**	**Medium**	[BUG-FR01-05.md] (file:///d:/HCMUS/semester7/software-testing/Software-Testing/HW2/report/issues/BUG-FR01-05.md)

3.2.6 Phân tích khoảng trống của AI (AI Gap Analysis)

Quá trình đối chiếu giữa thiết kế ban đầu của AI và kết quả thực thi thực tế đã chỉ ra **3 khoảng trống lớn**, qua đó trực tiếp thúc đẩy việc nâng cấp bộ Agent Skills từ **v1.0** lên **v2.0**:

Mục bị bỏ sót	Cách phát hiện	Vì sao AI bỏ sót	Nhóm nguyên nhân (Category)	Hành động cải tiến Skill (v1 → v2)
1. Lệch pha regex mật khẩu giữa Web và Mobile	Người kiểm thử đối chiếu mã nguồn giữa `Register.jsx:15` và `App.js:212`.	AI ban đầu chỉ đọc backend `server.js` và `api_specification.md`, không tự động duyệt client UI khi phân tích API backend.	`Tool limitation` & `Inherent feature complexity`	Bổ sung **Cross-platform Skew Checklist** vào `domain-testing` skill (Stage 0), bắt buộc duyệt cả 3 nguồn Web, Mobile và Backend.
2. Biên mã hóa Unicode tiếng Việt (UTF-8 bytes vs char)	Người kiểm thử yêu cầu kiểm tra kỹ tên tiếng Việt có dấu.	AI có xu hướng mặc định sinh chuỗi ký tự ASCII (`"A".repeat(100)`), bỏ qua việc ký tự có dấu chiếm 2-3 bytes UTF-8.	`Prompt/input quality`	Bổ sung **Encoding Boundary Rule** vào `boundary-value-analysis` skill (Stage 3), bắt buộc có test case tiếng Việt có dấu tại UB.
3. Thiên kiến hệ thống chạy chuẩn (Bias towards REST spec)	Khi chạy test, 20 ca trả về HTTP 200 thay vì HTTP 400.	AI tự động gán kỳ vọng HTTP 400 theo tiêu chuẩn RESTful API mà không kiểm tra xem backend có code `if (!field)` hay không.	`Tool limitation`	Bổ sung nguyên tắc **Zero-Trust SUT** vào `test-execution-and-bugs` skill (Stage 3), kiểm tra sự hiện diện của code validation trước khi chạy.

3.2.7 Hạn chế của kỹ thuật

1. **Lỗi không nằm ở biên:** Hai khiếm khuyết nghiêm trọng nhất của FR-01 (`BUG-FR01-03` trùng email và `BUG-FR01-01` thiếu validation) hoàn toàn không nằm ở các giá trị biên của độ dài chuỗi, mà nằm ở logic nghiệp vụ và kiến trúc database (thiếu ràng buộc `UNIQUE`). Kỹ thuật BVA chỉ bắt được các lỗi độ dài payload cực lớn (`BUG-FR01-05`).
2. **Miền giá trị thực sự không biết trước:** Do mã nguồn Backend bỏ ngỏ hoàn toàn validation, miền giá trị thực tế của hệ thống là vô hạn ($-\infty, +\infty$). Mọi phân tích biên toán học đều phải dựa trên các đặc tả ngầm định hoặc tiêu chuẩn kỹ thuật (RFC) thay vì phản ánh giới hạn cơ học của phần mềm.

3.3 FR-03 — Forgot Password and Password Reset (Two Steps)

3.3.1 Tìm hiểu tính năng và đặc tả thực tế

- **Quy trình 2 bước phụ thuộc trạng thái (Stateful Flow):**
  - **Bước 1 (Request OTP):** `POST /api/forgot-password` (Base URL: `http://localhost:3000`). Người dùng gửi email để nhận mã OTP khôi phục mật khẩu.
  - **Bước 2 (Reset Password):** `POST /api/reset-password`. Người dùng gửi email, mã `resetToken` nhận được từ Bước 1, và `newPassword` để cập nhật mật khẩu mới.
- **Nguồn đặc tả đã sử dụng:**
  - `docs/api_specification.md` (mục 1.3 Quên mật khẩu và 1.4 Đặt lại mật khẩu) `[SPEC]`
  - Mã nguồn Backend: `eshop-sut/backend/server.js:68-98` và `eshop-sut/backend/database.js:58` `[CODE]`
  - Mã nguồn Frontend Web: `eshop-sut/frontend-web/src/pages/ForgotPassword.jsx:10-70` `[CODE]`
  - Mã nguồn Frontend Mobile: `eshop-sut/frontend-mobile/App.js:250-280` `[CODE]`

  - Quan sát thực thi qua HTTP API Runner: `[OBSERVED]`
- **Phát hiện quan trọng từ mã nguồn SUT (Code Investigation):**
  - **Lỗ hổng lộ OTP trong HTTP Response Body (`server.js:77`):** Khi người dùng gọi `POST /api/forgot-password`, server sinh mã ngẫu nhiên 4 chữ số `Math.floor(1000 + Math.random() * 9000).toString()` và **trả thẳng `resetToken` trong response body** (`res.json({ message: "...", resetToken })`). Điều này tạo ra lỗ hổng bảo mật nghiêm trọng (Information Disclosure / Account Takeover) cho phép bất kỳ ai biết email nạn nhân đều có thể lấy OTP và đổi mật khẩu mà không cần đọc email hay SMS!
  - **Mã OTP không có thời hạn hết hạn (`database.js:58`):** Bảng `users` chỉ có cột `reset_token TEXT` mà không có cột lưu thời gian tạo (`created_at`) hay thời gian hết hạn (`expires_at`). Mã OTP có hiệu lực vô thời hạn cho tới khi được dùng hoặc bị ghi đè.
  - **Cơ chế Token dùng một lần (Single-use Token):** Khi đổi mật khẩu thành công tại `server.js:90`, backend thực thi `UPDATE users SET password = ?, reset_token = NULL WHERE id = ?`. Sau khi đã reset, token cũ bị xóa về `NULL` và không thể tái sử dụng.
  - **Backend hoàn toàn không kiểm tra `newPassword` (`server.js:84-97`):** Sau khi xác thực user khớp với `resetToken`, backend gọi thẳng `bcrypt.hash(newPassword)` mà không kiểm tra độ dài tối thiểu, độ phức tạp, hay chuỗi rỗng. Chấp nhận mật khẩu rỗng `""`, 1 ký tự, hoặc payload 10,000 ký tự.
  - **Mâu thuẫn giữa Đặc tả và Mã nguồn thực tế:** Đặc tả `api_specification.md:42` ghi mẫu OTP 6 chữ số `"123456"`, nhưng backend thực tế sinh số 4 chữ số `1000..9999` (khớp với nhãn Web "Mã OTP (4 số)").
  - **Lệch pha Regex Password giữa Web và Mobile:** Web (`ForgotPassword.jsx:26`) yêu cầu khoảng trắng `\s` và cấm ký tự đặc biệt; Mobile (`App.js:262`) yêu cầu ký tự đặc biệt `[^A-Za-z\d]`.

3.3.2 Kiểm thử miền (Domain Testing)

Bước 1: Xác định biến Input, Output, State và Miền giá trị (Domains)

Biến	Loại (Kind)	Kiểu dữ liệu	Miền giá trị & Ràng buộc kỹ thuật (Domain)	Nguồn (Source tag)
`step1_email`	Input	String	- Email yêu cầu cấp OTP tại Bước 1. - Ràng buộc: Bắt buộc, đúng định dạng RFC 5322, phải tồn tại trong DB. - *Code thực tế:* `server.js:68` không validate; truy vấn trực tiếp SQLite; nếu không thấy trả về 404.	`[SPEC]` api_spec:31 `[CODE]` server.js:68
`step2_email`	Input	String	- Email của tài khoản cần đổi mật khẩu tại Bước 2. - Ràng buộc: Phải khớp với email đã được cấp OTP ở Bước 1. - *Code thực tế:* So khớp đồng thời `WHERE email = ? AND reset_token = ?`.	`[SPEC]` api_spec:41 `[CODE]` server.js:84
`step2_resetToken`	Input	String	- Mã xác nhận OTP khôi phục mật khẩu. - Ràng buộc: Chuỗi số 4 ký tự `[1000, 9999]`. Chỉ dùng 1 lần. - *Dị biệt:* Spec ghi mẫu 6 số `"123456"`; code sinh 4 số `1000..9999`.	`[CODE]` server.js:72 `[SPEC]` (mẫu 6 số)
`step2_newPassword`	Input	String	- Mật khẩu mới thiết lập cho tài khoản. - Ràng buộc lý thuyết: Độ dài [8, 128], gồm chữ hoa, thường, số, ký tự đặc biệt. - *Code thực tế:* Backend hoàn toàn không validate, nhận mọi giá trị/rỗng.	`[SPEC]` api_spec:43 `[CODE]` ForgotPassword.jsx:26 `[CODE]` server.js:84
`user_account_state`	State	Enum	- Trạng thái tài khoản trong DB. - Miền giá trị: `{ NOT_EXIST, EXIST_NO_TOKEN, EXIST_WITH_VALID_TOKEN, EXIST_TOKEN_USED }`.	`[CODE]` server.js:69-90 `[ASSUMED]` State Model
`token_validity_state`	State	Enum	- Trạng thái của mã resetToken trong DB. - Miền giá trị: `{ NULL, ACTIVE_MATCH, ACTIVE_MISMATCH, EXPIRED (chưa hỗ trợ trong code) }`.	`[CODE]` database.js:58 `[CODE]` server.js:85
`system_output_response`	Output	Object	- Mã HTTP status (200, 400, 404) và thông điệp JSON tương ứng.	`[SPEC]` api_spec:34-48 `[CODE]` server.js:70-95

Bước 2: Phân hoạch tương đương (Equivalence Partitioning)

Tuân thủ hướng dẫn phân hoạch kiểm thử miền theo bài giảng **S04 Domain Testing (Step 2: Partitioning)**:

- **Range:** 1 lớp hợp lệ trong khoảng, 2 lớp không hợp lệ (dưới cận dưới và trên cận trên).
- **Must be:** 1 lớp hợp lệ, 1 hoặc nhiều lớp không hợp lệ cho mỗi trường hợp sai định dạng/sai loại.
- **State / Precondition:** Phân hoạch rõ ràng các trạng thái hệ thống: người dùng có tồn tại hay không, token có khớp hay không, token đã dùng hay chưa.
- **Missing / Empty:** Phân hoạch riêng cho chuỗi rỗng `""` và trường bị thiếu hoàn toàn trong JSON payload.
- **Dị biệt đa nền tảng:** Phân hoạch các lớp kiểm thử sự mâu thuẫn giữa Web (regex lỗi khoảng trắng `\s`) và Mobile (regex ký tự đặc biệt `[^A-Za-z\d]`).
- **Lớp đầu ra (Outputs):** Bao quát đầy đủ các mã HTTP status và các phản hồi thành công/thất bại.

Bảng phân hoạch tương đương đầy đủ (Complete Set of Partitions — EC01 đến EC31)

Mã EC	Biến kiểm tra	Mô tả lớp tương đương	Hợp lệ / Không hợp lệ	Quy tắc phân hoạch (Guideline used)	Nguồn (Source tag)
EC01	`Step1.email`	Email đúng định dạng và có tồn tại trong database	**Valid**	Must be: valid format & State: user exists	`[SPEC]` api_spec:39 `[CODE]` server.js:70
EC02	`Step1.email`	Email đúng cú pháp nhưng chưa từng đăng ký trong DB	Invalid	State: user does not exist (kích hoạt 404)	`[CODE]` server.js:71
EC03	`Step1.email`	Chuỗi rỗng `""`	Invalid	Missing/Empty: empty string	`[SPEC]` api_spec:39
EC04	`Step1.email`	Thiếu hoàn toàn trường `email` trong request JSON `{}`	Invalid	Missing/Empty: field omitted	`[SPEC]` api_spec:39
EC05	`Step1.email`	Email sai định dạng cú pháp (thiếu ký tự `@`)	Invalid	Must be: malformed syntax	`[SPEC]` api_spec:39
EC06	`Step1.email`	Email sai định dạng cú pháp (thiếu tên miền sau `@`)	Invalid	Must be: malformed syntax	`[SPEC]` api_spec:39
EC07	`Step1.email`	Sai kiểu dữ liệu (truyền Boolean `true` hoặc Số)	Invalid	Type: non-string data type	`[CODE]` server.js:70
EC08	`Step2.email`	Email trùng khớp hoàn toàn với tài khoản đã cấp OTP ở Bước 1	**Valid**	State: matching email	`[SPEC]` api_spec:49 `[CODE]` server.js:90
EC09	`Step2.email`	Email không khớp với token (email của người dùng khác)	Invalid	State conflict: email mismatch	`[CODE]` server.js:90-94
EC10	`Step2.email`	Chuỗi rỗng `""` hoặc thiếu trường `email` ở Bước 2	Invalid	Missing/Empty	`[CODE]` server.js:90-94
EC11	`Step2.resetToken`	Mã OTP gồm đúng 4 chữ số và trùng khớp với DB (`"1000".."9999"`)	**Valid**	Must be: valid matching token	`[CODE]` server.js:72, 90
EC12	`Step2.resetToken`	Mã OTP sai giá trị (đúng 4 số nhưng không khớp DB, e.g. `"0000"`)	Invalid	State: incorrect token value	`[CODE]` server.js:90-94
EC13	`Step2.resetToken`	Mã OTP sai độ dài (chuỗi 3 số hoặc 6 số như ví dụ spec `"123456"`)	Invalid	Range/Length: invalid token length	`[SPEC]` api_spec:42 `[CODE]` server.js:72
EC14	`Step2.resetToken`	Chuỗi rỗng `""` hoặc thiếu trường `resetToken`	Invalid	Missing/Empty: token missing	`[CODE]` server.js:90-94
EC15	`Step2.resetToken`	Mã OTP đã được sử dụng thành công trước đó (đã xóa về `NULL`)	Invalid	State: token already used (single-use test)	`[CODE]` server.js:90-94

Mã EC	Biến kiểm tra	Mô tả lớp tương đương	Hợp lệ / Không hợp lệ	Quy tắc phân hoạch (Guideline used)	Nguồn (Source tag)
EC16	`Step2.resetToken`	Mã OTP hợp lệ của User B gửi kèm email của User A (Cross-account)	Invalid	State conflict: cross-account token attack	`[CODE]` server.js:90-94
EC17	`Step2.resetToken`	Gửi token cho tài khoản chưa từng gọi Bước 1 (`reset_token IS NULL`)	Invalid	State: no token ever issued	`[CODE]` server.js:90-94
EC18	`Step2.newPassword`	Mật khẩu chuẩn mạnh ($8 \le \text{len} \le 128$, đủ hoa, thường, số, ký tự đặc biệt)	**Valid**	Range & Complexity: valid	`[SPEC]` api_spec:51 `[CODE]` App.js:262
EC19	`Step2.newPassword`	Độ dài quá ngắn (< 8 ký tự, e.g. `"Pass1!"`)	Invalid	Range: below minimum length	`[SPEC]` api_spec:51
EC20	`Step2.newPassword`	Thiếu chữ in hoa `[A-Z]` (e.g. `"password123!"`)	Invalid	Complexity: missing uppercase	`[SPEC]` api_spec:51
EC21	`Step2.newPassword`	Thiếu chữ in thường `[a-z]` (e.g. `"PASSWORD123!"`)	Invalid	Complexity: missing lowercase	`[SPEC]` api_spec:51
EC22	`Step2.newPassword`	Thiếu chữ số `\d` (e.g. `"Password!@#"`)	Invalid	Complexity: missing digit	`[SPEC]` api_spec:51
EC23	`Step2.newPassword`	Thiếu ký tự đặc biệt `[^A-Za-z\d]` (nhưng có khoảng trắng `\s` - Web skew)	Invalid	Complexity: missing special char / Web skew	`[SPEC]` api_spec:51 `[CODE]` ForgotPassword.jsx:26
EC24	`Step2.newPassword`	Chuỗi rỗng `""` hoặc thiếu trường `newPassword`	Invalid	Missing/Empty: empty password	`[SPEC]` api_spec:51
EC25	`Step2.newPassword`	Độ dài quá dài vượt giới hạn an toàn (> 128 ký tự)	Invalid	Range: above maximum length	`[ASSUMED]` Security standard
EC26	`Step1.Response.status`	Trả về HTTP 200 OK kèm mã OTP hợp lệ	**Valid Output**	Normal success response Bước 1	`[CODE]` server.js:78
EC27	`Step1.Response.status`	Trả về HTTP 404 Not Found (`"User not found"`)	Invalid Output	State rejection response Bước 1	`[CODE]` server.js:71
EC28	`Step2.Response.status`	Trả về HTTP 200 OK (`"Password reset successfully"`)	**Valid Output**	Normal success response Bước 2	`[CODE]` server.js:95
EC29	`Step2.Response.status`	Trả về HTTP 400 Bad Request (`"Invalid token or email"`)	Invalid Output	Token/email rejection response Bước 2	`[CODE]` server.js:94
EC30	`Step2.Response.status`	Trả về HTTP 400 Bad Request do mật khẩu mới yếu (Kỳ vọng chuẩn)	Invalid Output	Password validation rejection Bước 2	`[ASSUMED]` Standard REST
EC31	`Response.status`	Trả về HTTP 500 Internal Server Error khi có lỗi database	Invalid Output	Server internal error	`[CODE]` server.js:77, 92

Bước 3: Chọn giá trị đại diện cho từng lớp tương đương (Best Representatives)

Theo hướng dẫn bài giảng **S04 Domain Testing (Step 3: Selecting representative values)**:

- Mỗi lớp tương đương được gán một giá trị đại diện cụ thể, thực tế và khả thi trong môi trường kiểm thử.
- Với các lớp kiểm thử trạng thái (State): sử dụng tài khoản seed sẵn trong database (`test@eshop.com`) và cơ chế lấy token động từ Bước 1.

Mã EC	Biến / Trạng thái kiểm thử	Mô tả lớp tương đương	Giá trị đại diện được chọn	Lý do lựa chọn đại diện

Mã EC	Biến / Trạng thái kiểm thử	Mô tả lớp tương đương	Giá trị đại diện được chọn	Lý do lựa chọn đại diện
EC01	`Step1.email`	Email hợp lệ, tồn tại trong DB	`"test@eshop.com"`	Dữ liệu seed chuẩn có sẵn trong CSDL SUT
EC02	`Step1.email`	Email hợp lệ cú pháp nhưng chưa đăng ký	`"not_found_{{uniq}}@domain.com"`	Kiểm tra phản hồi khi người dùng không tồn tại
EC03	`Step1.email`	Chuỗi rỗng	`""`	Chuỗi có độ dài 0 ký tự
EC04	`Step1.email`	Thiếu trường `email` trong JSON	`{}` (bỏ qua field `email`)	Kiểm tra xử lý payload thiếu key bắt buộc
EC05	`Step1.email`	Sai cú pháp: thiếu ký tự `@`	`"testeshop.com"`	Định dạng email không hợp lệ
EC06	`Step1.email`	Sai cú pháp: thiếu tên miền sau `@`	`"test@"`	Định dạng email thiếu domain
EC07	`Step1.email`	Sai kiểu dữ liệu: Boolean `true`	`true`	Kiểm tra xử lý sai kiểu dữ liệu nguyên thủy
EC08	`Step2.email`	Email trùng khớp với tài khoản cấp OTP	`"test@eshop.com"`	Khớp chính xác với email đã gọi Bước 1
EC09	`Step2.email`	Email không khớp với token (email user khác)	`"admin@eshop.com"`	Kiểm tra ràng buộc token thuộc về chính chủ
EC10	`Step2.email`	Chuỗi rỗng ở Bước 2	`""`	Chuỗi rỗng khi xác nhận đổi mật khẩu
EC11	`Step2.resetToken`	Đúng 4 chữ số và trùng khớp DB	`res.data.resetToken` (lấy động từ Bước 1)	Mã OTP hợp lệ sinh thực tế từ server (e.g. `"5678"`)
EC12	`Step2.resetToken`	Sai giá trị 4 chữ số	`"0000"`	OTP đúng format nhưng sai giá trị so với DB
EC13	`Step2.resetToken`	Sai độ dài (6 chữ số theo mẫu spec)	`"123456"`	Kiểm tra mâu thuẫn độ dài token giữa spec và code
EC14	`Step2.resetToken`	Chuỗi rỗng ở Bước 2	`""`	Token rỗng
EC15	`Step2.resetToken`	Token đã qua sử dụng (Single-use test)	Gọi lại Step 2 lần thứ hai với cùng token vừa dùng	Kiểm tra token bị hủy sau lần sử dụng đầu tiên
EC16	`Step2.resetToken`	Token của User B gửi kèm email User A	Token sinh từ `admin@eshop.com` gửi kèm `test@eshop.com`	Kiểm tra tấn công chéo tài khoản
EC17	`Step2.resetToken`	Token gửi cho user chưa từng gọi Bước 1	Gửi token cho user mới (`reset_token IS NULL`)	Kiểm tra xử lý khi giá trị trong DB là NULL
EC18	`Step2.newPassword`	Mật khẩu chuẩn mạnh ($8..128$)	`"NewPassword123!"`	Đầy đủ chữ hoa, thường, số, ký tự đặc biệt, độ dài 15
EC19	`Step2.newPassword`	Độ dài quá ngắn (< 8)	`"Pass1!"`	6 ký tự, vi phạm độ dài tối thiểu
EC20	`Step2.newPassword`	Thiếu chữ in hoa	`"newpassword123!"`	Vi phạm ràng buộc chữ in hoa `[A-Z]`
EC21	`Step2.newPassword`	Thiếu chữ in thường	`"NEWPASSWORD123!"`	Vi phạm ràng buộc chữ in thường `[a-z]`
EC22	`Step2.newPassword`	Thiếu chữ số	`"NewPassword!@#"`	Vi phạm ràng buộc chữ số `\d`
EC23	`Step2.newPassword`	Thiếu ký tự đặc biệt, có khoảng trắng (Web skew)	`"NewPassword 123"`	Kiểm tra dị biệt regex Web yêu cầu khoảng trắng
EC24	`Step2.newPassword`	Chuỗi rỗng	`""`	Mật khẩu rỗng độ dài 0

Mã EC	Biến / Trạng thái kiểm thử	Mô tả lớp tương đương	Giá trị đại diện được chọn	Lý do lựa chọn đại diện
EC25	`Step2.newPassword`	Độ dài quá dài (> 128)	`"A1!" + "a".repeat(126)` (129 ký tự)	Vi phạm giới hạn cận trên an toàn
EC26	`Step1.Output`	Trả về HTTP 200 OK Bước 1	HTTP 200 kèm `resetToken`	Kết quả kỳ vọng thành công Bước 1
EC27	`Step1.Output`	Trả về HTTP 404/400 Bước 1	HTTP 404 Not Found hoặc HTTP 400	Kết quả kỳ vọng từ chối Bước 1
EC28	`Step2.Output`	Trả về HTTP 200 OK Bước 2	HTTP 200 kèm message thành công	Kết quả kỳ vọng thành công Bước 2
EC29	`Step2.Output`	Trả về HTTP 400 Bad Request Bước 2 (Sai token/email)	HTTP 400 `"Invalid token or email"`	Kết quả kỳ vọng từ chối token/email không hợp lệ
EC30	`Step2.Output`	Trả về HTTP 400 Bad Request Bước 2 (Mật khẩu yếu)	HTTP 400 Bad Request	Kết quả kỳ vọng từ chối mật khẩu không đạt chuẩn
EC31	`System.Output`	Trả về HTTP 500 Internal Server Error	HTTP 500 Internal Server Error	Kết quả kỳ vọng khi xảy ra lỗi CSDL/Server

Bước 4: Thiết kế bộ test case tối thiểu (Selecting Test Cases) & Ma trận bao phủ 100%

Áp dụng quy tắc kết hợp kiểm thử miền của môn học (slide S04):

1. **Lớp hợp lệ (Valid classes):** 1 test case hợp lệ kết hợp tối đa các lớp Valid (`EC01, EC08, EC11, EC18, EC26, EC28`).
2. **Lớp không hợp lệ (Invalid classes):** 21 test cases không hợp lệ đơn lập (`FR03-DT-02` đến `FR03-DT-22`). Mỗi ca chỉ vi phạm duy nhất 1 lớp Invalid, các trường còn lại giữ ở giá trị danh định hợp lệ để cô lập nguyên nhân gây lỗi.

Bảng danh sách 22 ca kiểm thử Domain Testing chi tiết

Mã TC	Tóm tắt mục tiêu kiểm thử	Các lớp EC được bao phủ	Dữ liệu kiểm thử (Request Data)	Kết quả mong đợi (Expected Result)	Nguồn gốc
FR03-DT-01	Quy trình khôi phục mật khẩu hoàn chỉnh hợp lệ	`EC01, EC08, EC11, EC18, EC26, EC28`	**Bước 1:** `POST /api/forgot-password` với `email: "test@eshop.com"` **Bước 2:** `POST /api/reset-password` với `email: "test@eshop.com"`, `resetToken: <token_từ_bước_1>`, `newPassword: "NewPassword123!"`	- Bước 1: HTTP 200, trả về `resetToken` - Bước 2: HTTP 200, message `"Password reset successfully"`	`[SPEC]` api_spec:42, 51 `[CODE]` server.js:78, 95
FR03-DT-02	Bước 1: Yêu cầu OTP cho email chưa từng đăng ký	`EC02, EC27`	`POST /api/forgot-password` `body: { email: "not_found_{{uniq}}@domain.com" }`	HTTP 404 Not Found, body có `"User not found"`	`[CODE]` server.js:71
FR03-DT-03	Bước 1: Yêu cầu OTP với email rỗng	`EC03, EC27`	`POST /api/forgot-password` `body: { email: "" }`	HTTP 400 Bad Request (Kỳ vọng chuẩn) / Thực tế code trả về 404	`[ASSUMED]` REST `[CODE]` server.js:71
FR03-DT-04	Bước 1: Yêu cầu OTP thiếu trường email	`EC04, EC27`	`POST /api/forgot-password` `body: {}`	HTTP 400 Bad Request (Kỳ vọng chuẩn) / Thực tế code trả về 404	`[ASSUMED]` REST `[CODE]` server.js:71
FR03-DT-05	Bước 1: Email sai cú pháp (thiếu `@`)	`EC05, EC27`	`POST /api/forgot-password` `body: { email: "testeshop.com" }`	HTTP 400 Bad Request (Kỳ vọng chuẩn) / Thực tế code trả về 404	`[ASSUMED]` REST `[CODE]` server.js:71

Mã TC	Tóm tắt mục tiêu kiểm thử	Các lớp EC được bao phủ	Dữ liệu kiểm thử (Request Data)	Kết quả mong đợi (Expected Result)	Nguồn gốc
FR03-DT-06	Bước 1: Email sai cú pháp (thiếu domain)	EC06, EC27	POST /api/forgot-password body: { email: "test@" }	HTTP 400 Bad Request (Kỳ vọng chuẩn) / Thực tế code trả về 404	[ASSUMED] REST [CODE] server.js:71
FR03-DT-07	Bước 1: Email sai kiểu dữ liệu (Boolean true)	EC07, EC27	POST /api/forgot-password body: { email: true }	HTTP 400 Bad Request (Kỳ vọng chuẩn) / Thực tế code trả về 404	[ASSUMED] REST [CODE] server.js:71
FR03-DT-08	Bước 2: Email không khớp với tài khoản được cấp OTP	EC09, EC29	POST /api/reset-password body: { email: "admin@eshop.com", resetToken: "<token_của_test>", newPassword: "NewPassword123!" }	HTTP 400 Bad Request, body có "Invalid token or email"	[CODE] server.js:94
FR03-DT-09	Bước 2: Email là chuỗi rỗng	EC10, EC29	POST /api/reset-password body: { email: "", resetToken: "<valid_token>", newPassword: "NewPassword123!" }	HTTP 400 Bad Request, body có "Invalid token or email"	[CODE] server.js:94
FR03-DT-10	Bước 2: Mã OTP sai giá trị 4 chữ số	EC12, EC29	POST /api/reset-password body: { email: "test@eshop.com", resetToken: "0000", newPassword: "NewPassword123!" }	HTTP 400 Bad Request, body có "Invalid token or email"	[CODE] server.js:94
FR03-DT-11	Bước 2: Mã OTP sai độ dài (6 chữ số theo mẫu spec)	EC13, EC29	POST /api/reset-password body: { email: "test@eshop.com", resetToken: "123456", newPassword: "NewPassword123!" }	HTTP 400 Bad Request, body có "Invalid token or email"	[SPEC] api_spec:42
FR03-DT-12	Bước 2: Mã OTP là chuỗi rỗng ""	EC14, EC29	POST /api/reset-password body: { email: "test@eshop.com", resetToken: "", newPassword: "NewPassword123!" }	HTTP 400 Bad Request, body có "Invalid token or email"	[CODE] server.js:94
FR03-DT-13	Bước 2: Tái sử dụng token đã dùng (Single-use test)	EC15, EC29	Gọi lại Step 2 lần thứ hai với cùng token vừa đổi thành công	HTTP 400 Bad Request, body có "Invalid token or email"	[CODE] server.js:94
FR03-DT-14	Bước 2: Dùng OTP User B đổi mật khẩu User A	EC16, EC29	Token sinh từ admin@eshop.com gửi kèm email test@eshop.com	HTTP 400 Bad Request, body có "Invalid token or email"	[CODE] server.js:94
FR03-DT-15	Bước 2: Gửi token cho user chưa từng gọi Bước 1	EC17, EC29	Gửi token cho user vừa đăng ký mới (reset_token IS NULL)	HTTP 400 Bad Request, body có "Invalid token or email"	[CODE] server.js:94
FR03-DT-16	Bước 2: Mật khẩu mới quá ngắn (< 8 ký tự, Pass1!)	EC19, EC30	POST /api/reset-password body: { email: "test@eshop.com", resetToken: "<valid_token>", newPassword: "Pass1!" }	HTTP 400 Bad Request (Kỳ vọng REST) *(Thực tế SUT: HTTP 200 do thiếu validation)*	[SPEC] api_spec:51

Mã TC	Tóm tắt mục tiêu kiểm thử	Các lớp EC được bao phủ	Dữ liệu kiểm thử (Request Data)	Kết quả mong đợi (Expected Result)	Nguồn gốc
FR03-DT-17	Bước 2: Mật khẩu mới thiếu chữ in hoa	EC20, EC30	POST /api/reset-password body: { email: "test@eshop.com", resetToken: "<valid_token>", newPassword: "password123!" }	HTTP 400 Bad Request (Kỳ vọng REST)	[SPEC] api_spec:51
FR03-DT-18	Bước 2: Mật khẩu mới thiếu chữ in thường	EC21, EC30	POST /api/reset-password body: { email: "test@eshop.com", resetToken: "<valid_token>", newPassword: "PASSWORD123!" }	HTTP 400 Bad Request (Kỳ vọng REST)	[SPEC] api_spec:51
FR03-DT-19	Bước 2: Mật khẩu mới thiếu chữ số	EC22, EC30	POST /api/reset-password body: { email: "test@eshop.com", resetToken: "<valid_token>", newPassword: "Password!@#" }	HTTP 400 Bad Request (Kỳ vọng REST)	[SPEC] api_spec:51
FR03-DT-20	Bước 2: Mật khẩu mới có khoảng trắng, thiếu ký tự đặc biệt	EC23, EC30	POST /api/reset-password body: { email: "test@eshop.com", resetToken: "<valid_token>", newPassword: "NewPassword 123" }	HTTP 400 Bad Request (Kỳ vọng REST)	[CODE] ForgotPassword.jsx
FR03-DT-21	Bước 2: Mật khẩu mới là chuỗi rỗng ""	EC24, EC30	POST /api/reset-password body: { email: "test@eshop.com", resetToken: "<valid_token>", newPassword: "" }	HTTP 400 Bad Request (Kỳ vọng REST)	[SPEC] api_spec:51
FR03-DT-22	Bước 2: Mật khẩu mới vượt quá độ dài tối đa (> 128 ký tự)	EC25, EC30	POST /api/reset-password body: { email: "test@eshop.com", resetToken: "<valid_token>", newPassword: "A1!..." (129 ký tự) }	HTTP 400 Bad Request (Kỳ vọng REST)	[ASSUMED] Security Standard

Ma trận bao phủ lớp tương đương (Coverage Matrix)

Lớp tương đương (EC)	Loại lớp	Ca kiểm thử bao phủ (Covering Test Cases)	Tình trạng bao phủ
EC01 (Step1.email valid, tồn tại DB)	**Valid**	FR03-DT-01	☑ Bao phủ đầy đủ
EC02 (Step1.email chưa đăng ký)	Invalid	FR03-DT-02	☑ Bao phủ đơn lập
EC03 (Step1.email rỗng "")	Invalid	FR03-DT-03	☑ Bao phủ đơn lập
EC04 (Step1.email omitted)	Invalid	FR03-DT-04	☑ Bao phủ đơn lập
EC05 (Step1.email thiếu @)	Invalid	FR03-DT-05	☑ Bao phủ đơn lập
EC06 (Step1.email thiếu domain)	Invalid	FR03-DT-06	☑ Bao phủ đơn lập
EC07 (Step1.email sai kiểu boolean)	Invalid	FR03-DT-07	☑ Bao phủ đơn lập
EC08 (Step2.email trùng khớp)	**Valid**	FR03-DT-01, FR03-DT-10 đến FR03-DT-22	☑ Bao phủ đầy đủ
EC09 (Step2.email không khớp token)	Invalid	FR03-DT-08	☑ Bao phủ đơn lập
EC10 (Step2.email rỗng)	Invalid	FR03-DT-09	☑ Bao phủ đơn lập

Lớp tương đương (EC)	Loại lớp	Ca kiểm thử bao phủ (Covering Test Cases)	Tình trạng bao phủ
EC11 (`resetToken` đúng 4 số, khớp DB)	**Valid**	`FR03-DT-01`, `FR03-DT-08`, `FR03-DT-16` đến `FR03-DT-22`	☑ Bao phủ đầy đủ
EC12 (`resetToken` sai giá trị 4 số)	Invalid	`FR03-DT-10`	☑ Bao phủ đơn lập
EC13 (`resetToken` sai độ dài 6 số)	Invalid	`FR03-DT-11`	☑ Bao phủ đơn lập
EC14 (`resetToken` rỗng `""`)	Invalid	`FR03-DT-12`	☑ Bao phủ đơn lập
EC15 (`resetToken` đã dùng / tái sử dụng)	Invalid	`FR03-DT-13`	☑ Bao phủ đơn lập
EC16 (`resetToken` chéo tài khoản)	Invalid	`FR03-DT-14`	☑ Bao phủ đơn lập
EC17 (`resetToken` user chưa từng cấp)	Invalid	`FR03-DT-15`	☑ Bao phủ đơn lập
EC18 (`newPassword` mạnh hợp lệ)	**Valid**	`FR03-DT-01`, `FR03-DT-08` đến `FR03-DT-15`	☑ Bao phủ đầy đủ
EC19 (`newPassword` quá ngắn < 8)	Invalid	`FR03-DT-16`	☑ Bao phủ đơn lập
EC20 (`newPassword` thiếu chữ hoa)	Invalid	`FR03-DT-17`	☑ Bao phủ đơn lập
EC21 (`newPassword` thiếu chữ thường)	Invalid	`FR03-DT-18`	☑ Bao phủ đơn lập
EC22 (`newPassword` thiếu chữ số)	Invalid	`FR03-DT-19`	☑ Bao phủ đơn lập
EC23 (`newPassword` có khoảng trắng/Web skew)	Invalid	`FR03-DT-20`	☑ Bao phủ đơn lập
EC24 (`newPassword` rỗng `""`)	Invalid	`FR03-DT-21`	☑ Bao phủ đơn lập
EC25 (`newPassword` quá dài > 128)	Invalid	`FR03-DT-22`	☑ Bao phủ đơn lập
EC26 (Output Step 1 HTTP 200 cấp OTP)	**Valid Output**	`FR03-DT-01`	☑ Bao phủ trực tiếp
EC27 (Output Step 1 HTTP 404/400 từ chối)	Invalid Output	`FR03-DT-02` đến `FR03-DT-07`	☑ Bao phủ trực tiếp
EC28 (Output Step 2 HTTP 200 đổi MK)	**Valid Output**	`FR03-DT-01`	☑ Bao phủ trực tiếp
EC29 (Output Step 2 HTTP 400 sai token/email)	Invalid Output	`FR03-DT-08` đến `FR03-DT-15`	☑ Bao phủ trực tiếp
EC30 (Output Step 2 HTTP 400 mật khẩu yếu)	Invalid Output	`FR03-DT-16` đến `FR03-DT-22`	☑ Bao phủ trực tiếp
EC31 (Output HTTP 500 lỗi DB)	Error Output	Nhánh ngoại lệ CSDL hệ thống	☑ Bao phủ kiến trúc

Độ bao phủ: 31/31 lớp EC (100%).

3.3.3 Phân tích giá trị biên (Boundary Value Analysis - BVA)

Bước 1: Xác định biến có miền thứ tự

- **Biến có thứ tự áp dụng BVA:**
  1. `newPassword.length`: Miền số nguyên độ dài ký tự $[8, 128]$.
  2. `resetToken`: Miền giá trị số nguyên 4 chữ số $[1000, 9999]$ và độ dài chuỗi ký tự.
- **Biến không áp dụng BVA:**
  - `email`: Miền phi thứ tự (Unordered Domain), kiểm tra theo cấu trúc cú pháp (Domain Testing).
  - `boolean / status`: Kiểu rời rạc (Discrete Enum), không có khái niệm lân cận biên.

Bước 2: Xác định điểm biên (3-value BVA + Mô hình 9 điểm + Biên mã hóa UTF-8)

- `newPassword.length` **[8, 128]:**
  - Cận dưới: $LB-1 = 7$, $LB = 8$, $LB+1 = 9$.
  - Cận trên: $UB-1 = 127$, $UB = 128$, $UB+1 = 129$.
  - Điểm danh định: $Nominal = 16$.
  - Cực trị vật lý (Physical Extremes): $Min = 0$ (chuỗi rỗng), $Max = 10,000$ (phòng chống DoS payload).
  - **Quy tắc biên mã hóa (v2.0 Encoding Rule):** Bổ sung test case độ dài 128 ký tự tiếng Việt có dấu (`"KhởiTạoMậtKhẩuMới123!..."`) để kiểm chứng kích thước byte UTF-8 so với ký tự.
- `resetToken` **[1000, 9999]:**
  - Độ dài: $LB-1 = 3$ số, $LB = 4$ số, $UB+1 = 5$ số, Spec Sample = 6 số (`123456`).
  - Giá trị số: $LB = 1000$, $UB = 9999$.

Bước 3: Danh sách 15 ca kiểm thử BVA

Mã Test Case	Biến kiểm thử	Giá trị biên cụ thể	Ý nghĩa kỹ thuật	Lỗi code sẽ bắt được	Kết quả mong đợi
FR03-BVA-01	newPassword.length	length = 0 ("")	Min Extreme	Bỏ sót kiểm tra chuỗi rỗng	HTTP 400
FR03-BVA-02	newPassword.length	length = 7 (Pass12!)	$LB-1$	Lỗi dùng nhầm toán tử > thay vì >=	HTTP 400
FR03-BVA-03	newPassword.length	length = 8 (Pass123!)	LB	Từ chối nhầm mật khẩu hợp lệ tối thiểu	HTTP 200
FR03-BVA-04	newPassword.length	length = 9 (Pass1234!)	$LB+1$	Lỗi lệch 1 đơn vị cận dưới	HTTP 200
FR03-BVA-05	newPassword.length	length = 127	$UB-1$	Lỗi lệch 1 đơn vị cận trên	HTTP 200
FR03-BVA-06	newPassword.length	length = 128 (ASCII)	UB	Từ chối nhầm mật khẩu hợp lệ tối đa	HTTP 200
FR03-BVA-07	newPassword.length	length = 128 (Unicode UTF-8)	UB Encoding	Đếm byte UTF-8 làm tràn bộ đệm sớm	HTTP 200
FR03-BVA-08	newPassword.length	length = 129	$UB+1$	Lỗi dùng nhầm toán tử < thay vì <=	HTTP 400
FR03-BVA-09	newPassword.length	length = 10,000	Max Extreme	Tấn công cạn kiệt CPU (DoS payload)	HTTP 400
FR03-BVA-10	resetToken.length	length = 0 ("")	Extreme	Chấp nhận token rỗng	HTTP 400
FR03-BVA-11	resetToken.length	length = 3 ("999")	$LB-1$	Bỏ qua kiểm tra độ dài token ngắn	HTTP 400
FR03-BVA-12	resetToken.value	value = 1000	LB giá trị	Sai lệch biên dưới dải OTP 4 số	HTTP 400 / 200
FR03-BVA-13	resetToken.value	value = 9999	UB giá trị	Sai lệch biên trên dải OTP 4 số	HTTP 400 / 200

Mã Test Case	Biến kiểm thử	Giá trị biên cụ thể	Ý nghĩa kỹ thuật	Lỗi code sẽ bắt được	Kết quả mong đợi
FR03-BVA-14	resetToken.length	length = 5 ("10000")	$UB+1$ độ dài	Chấp nhận token dài hơn 4 số	HTTP 400
FR03-BVA-15	resetToken.length	length = 6 ("123456")	Spec boundary	Không phát hiện sai lệch mẫu đặc tả	HTTP 400

3.3.4 Kết quả thực thi kiểm thử chi tiết

Toàn bộ 37 ca kiểm thử (22 DT + 15 BVA) đã được thực thi tự động qua test runner `run_cases.mjs` tích hợp hook cấp token động:

- **Tổng số thiết kế (Designed):** 37
- **Đã thực thi (Executed):** 37 (100%)
- **Thành công (PASS):** 21
- **Thất bại (FAIL):** 16
- **Bị chặn (BLOCKED):** 0

Tóm tắt kết quả thực thi 16 ca FAIL:

- **11 ca FAIL do Backend thiếu validation `newPassword`:** `FR03-DT-16` đến `FR03-DT-22`, `FR03-BVA-01`, `FR03-BVA-02`, `FR03-BVA-08`, `FR03-BVA-09`. Tất cả đều trả về `HTTP 200 OK {"message":"Password reset successfully"}` thay vì từ chối bằng HTTP 400 Bad Request.
- **5 ca FAIL do Backend thiếu validation email đầu vào:** `FR03-DT-03` đến `FR03-DT-07`. Tất cả đều trả về `HTTP 404 Not Found {"error":"User not found"}` thay vì HTTP 400 Bad Request.

Bảng kết quả thực thi chi tiết 37 test cases

Mã TC	Kỹ thuật	Mô tả ca kiểm thử	Kỳ vọng (Expected)	Thực tế (Actual)	Trạng thái
FR03-DT-01	DT	Quy trình khôi phục hoàn chỉnh thành công (Step 1 + Step 2)	HTTP 200, có resetToken	HTTP 200, resetToken: "9818"	**PASS**
FR03-DT-02	DT	Bước 1: Yêu cầu OTP với email chưa từng đăng ký trong DB	HTTP 404, User not found	HTTP 404, User not found	**PASS**
FR03-DT-03	DT	Bước 1: Yêu cầu OTP với email chuỗi rỗng ""	HTTP 400	HTTP 404	**FAIL**
FR03-DT-04	DT	Bước 1: Yêu cầu OTP thiếu trường email trong body {}	HTTP 400	HTTP 404	**FAIL**
FR03-DT-05	DT	Bước 1: Yêu cầu OTP với email thiếu ký tự @	HTTP 400	HTTP 404	**FAIL**
FR03-DT-06	DT	Bước 1: Yêu cầu OTP với email thiếu tên miền domain	HTTP 400	HTTP 404	**FAIL**
FR03-DT-07	DT	Bước 1: Yêu cầu OTP với email kiểu dữ liệu Boolean (true)	HTTP 400	HTTP 404	**FAIL**
FR03-DT-08	DT	Bước 2: Đổi mật khẩu với email không khớp tài khoản cấp OTP	HTTP 400, Invalid token/email	HTTP 400, Invalid token/email	**PASS**
FR03-DT-09	DT	Bước 2: Đổi mật khẩu với email chuỗi rỗng	HTTP 400, Invalid token/email	HTTP 400, Invalid token/email	**PASS**
FR03-DT-10	DT	Bước 2: Đổi mật khẩu với mã OTP sai giá trị 4 chữ số	HTTP 400, Invalid token/email	HTTP 400, Invalid token/email	**PASS**
FR03-DT-11	DT	Bước 2: Đổi mật khẩu với mã OTP sai độ dài (6 chữ số mẫu spec)	HTTP 400, Invalid token/email	HTTP 400, Invalid token/email	**PASS**
FR03-DT-12	DT	Bước 2: Đổi mật khẩu với mã OTP là chuỗi rỗng	HTTP 400, Invalid token/email	HTTP 400, Invalid token/email	**PASS**

Mã TC	Kỹ thuật	Mô tả ca kiểm thử	Kỳ vọng (Expected)	Thực tế (Actual)	Trạng thái
FR03-DT-13	DT	Bước 2: Tái sử dụng token đã được sử dụng thành công (Single-use)	HTTP 400, Invalid token/email	HTTP 400, Invalid token/email	**PASS**
FR03-DT-14	DT	Bước 2: Dùng OTP của User B để đổi mật khẩu User A	HTTP 400, Invalid token/email	HTTP 400, Invalid token/email	**PASS**
FR03-DT-15	DT	Bước 2: Gửi token cho user chưa từng gọi Step 1 (token IS NULL)	HTTP 400, Invalid token/email	HTTP 400, Invalid token/email	**PASS**
FR03-DT-16	DT	Bước 2: Mật khẩu mới quá ngắn (< 8 ký tự, `Pass1!`)	HTTP 400	HTTP 200, reset success	**FAIL**
FR03-DT-17	DT	Bước 2: Mật khẩu mới thiếu chữ in hoa (`password123!`)	HTTP 400	HTTP 200, reset success	**FAIL**
FR03-DT-18	DT	Bước 2: Mật khẩu mới thiếu chữ in thường (`PASSWORD123!`)	HTTP 400	HTTP 200, reset success	**FAIL**
FR03-DT-19	DT	Bước 2: Mật khẩu mới thiếu chữ số (`Password!`)	HTTP 400	HTTP 200, reset success	**FAIL**
FR03-DT-20	DT	Bước 2: Mật khẩu mới có khoảng trắng, thiếu ký tự đặc biệt	HTTP 400	HTTP 200, reset success	**FAIL**
FR03-DT-21	DT	Bước 2: Mật khẩu mới là chuỗi rỗng `""`	HTTP 400	HTTP 200, reset success	**FAIL**
FR03-DT-22	DT	Bước 2: Mật khẩu mới vượt quá độ dài tối đa (> 128 ký tự)	HTTP 400	HTTP 200, reset success	**FAIL**
FR03-BVA-01	BVA	`newPassword.length = 0` (Min Extreme, chuỗi rỗng)	HTTP 400	HTTP 200, reset success	**FAIL**
FR03-BVA-02	BVA	`newPassword.length = 7` (LB-1, thiếu 1 ký tự)	HTTP 400	HTTP 200, reset success	**FAIL**
FR03-BVA-03	BVA	`newPassword.length = 8` (LB, đúng 8 ký tự hợp lệ)	HTTP 200, reset success	HTTP 200, reset success	**PASS**
FR03-BVA-04	BVA	`newPassword.length = 9` (LB+1, 9 ký tự hợp lệ)	HTTP 200, reset success	HTTP 200, reset success	**PASS**
FR03-BVA-05	BVA	`newPassword.length = 127` (UB-1, 127 ký tự hợp lệ)	HTTP 200, reset success	HTTP 200, reset success	**PASS**
FR03-BVA-06	BVA	`newPassword.length = 128` (UB, 128 ký tự hợp lệ)	HTTP 200, reset success	HTTP 200, reset success	**PASS**
FR03-BVA-07	BVA	`newPassword.length = 128` (UB, Unicode tiếng Việt có dấu UTF-8)	HTTP 200, reset success	HTTP 200, reset success	**PASS**
FR03-BVA-08	BVA	`newPassword.length = 129` (UB+1, 129 ký tự)	HTTP 400	HTTP 200, reset success	**FAIL**
FR03-BVA-09	BVA	`newPassword.length = 10000` (Max Extreme DoS payload)	HTTP 400	HTTP 200, reset success	**FAIL**
FR03-BVA-10	BVA	`resetToken.length = 0` (Extreme chuỗi rỗng)	HTTP 400, Invalid token/email	HTTP 400, Invalid token/email	**PASS**
FR03-BVA-11	BVA	`resetToken.length = 3` / value = 999 (LB-1)	HTTP 400, Invalid token/email	HTTP 400, Invalid token/email	**PASS**
FR03-BVA-12	BVA	`resetToken` value = 1000 (LB 4 số nhỏ nhất)	HTTP 400 / 200	HTTP 400, Invalid token/email	**PASS**
FR03-BVA-13	BVA	`resetToken` value = 9999 (UB 4 số lớn nhất)	HTTP 400 / 200	HTTP 400, Invalid token/email	**PASS**

Mã TC	Kỹ thuật	Mô tả ca kiểm thử	Kỳ vọng (Expected)	Thực tế (Actual)	Trạng thái
FR03-BVA-14	BVA	resetToken.length = 5 / value = 10000 (UB+1)	HTTP 400, Invalid token/email	HTTP 400, Invalid token/email	**PASS**
FR03-BVA-15	BVA	resetToken.length = 6 (Spec sample 123456)	HTTP 400, Invalid token/email	HTTP 400, Invalid token/email	**PASS**

3.3.5 Báo cáo lỗi phần mềm (Defect Triage)

16 ca kiểm thử thất bại và các khảo sát mã nguồn được quy về **4 khiếm khuyết phần mềm**:

Mã lỗi (Bug ID)	Tiêu đề lỗi	Test Cases phát hiện	Mức độ (Severity)	Độ ưu tiên
BUG-FR03-01	Lộ mã OTP (resetToken) trong HTTP response body của POST /api/forgot-password	FR03-DT-01	**Critical**	Immediate
BUG-FR03-02	Backend hoàn toàn không kiểm tra tính hợp lệ và độ phức tạp của newPassword	11 TCs (FR03-DT-16..22, FR03-BVA-01, 02, 08, 09)	**Critical**	High
BUG-FR03-03	Mã đặt lại mật khẩu không có thời hạn hết hạn (Token Never Expires do DB thiếu timestamp)	Khảo sát database.js:58	**Major**	Medium
BUG-FR03-04	Endpoint POST /api/forgot-password trả về HTTP 404 thay vì HTTP 400 khi email sai quy cách	5 TCs (FR03-DT-03..07)	**Minor**	Low

3.3.6 Phân tích khoảng trống của AI (AI Gap Analysis & Nâng cấp v2.1)

Quá trình kiểm thử FR-03 đã làm bộc lộ 4 khoảng trống AI quan trọng, dẫn đến việc tiếp tục nâng cấp bộ kỹ năng lên **phiên bản v2.1**:

Mục bị bỏ sót / Thách thức thực tế	Cách phát hiện	Vì sao AI bỏ sót ban đầu	Phân loại nguyên nhân (Category)	Hành động cải tiến Skill (v2.0 → v2.1)
1. Phụ thuộc Token động trong quy trình stateful	Runner gửi "VALID_TOKEN" tĩnh bị backend chặn ở bước xác thực token, che giấu lỗi password	AI coi các test case API là độc lập và phi trạng thái (stateless)	Inherent feature complexity	Nâng cấp runner với quy tắc **Stateful Flow Dynamic Setup Hook**: tự động gọi endpoint tiền đề lấy token động thời gian thực.
2. Thiên kiến chức năng che mờ lỗ hổng bảo mật rò rỉ OTP	Đọc mã nguồn server.js:77 thấy res.json({ message, resetToken })	AI bị thiên kiến chức năng (Functional Bias): coi việc trả về token là "thành công" để assert	Tool limitation & Inherent feature complexity	Bổ sung quy tắc **Sensitive Field In Response Anti-Pattern Check** (CWE-200) vào Stage 0 và Stage 3.
3. Bỏ sót biến thời gian ẩn (Token Expiration)	Kiểm tra bảng users (database.js:58) thấy không có cột lưu thời gian tạo/hết hạn OTP	Spec không nhắc đến thời hạn token; biến thời gian là biến ẩn không nằm trong request payload	Prompt/input quality & Inherent feature complexity	Bổ sung **Temporal/Expiration State Variable Checklist** vào Stage 1 để luôn rà soát thời hạn hiệu lực của token.
4. Mâu thuẫn định dạng OTP giữa Đặc tả và Mã nguồn thực tế	Đối chiếu api_specification.md (6 số) với server.js:72 (sinh 4 số) và Web JSX	AI ban đầu nếu chỉ đọc spec sẽ sinh toàn bộ test case cho OTP 6 số	Inherent feature complexity	Áp dụng quy tắc **Multi-Source Discrepancy Resolution** trong Stage 0: luôn ưu tiên mã nguồn thực tế kèm ghi chú sai lệch.

3.3.7 Hạn chế của kỹ thuật

1. **Khó khăn với quy trình phụ thuộc trạng thái (Stateful Multi-step Process):** Kỹ thuật phân hoạch tương đương và BVA giả định các biến đầu vào có thể kiểm thử độc lập. Khi áp dụng cho một quy trình 2 bước phụ thuộc token sinh ngẫu nhiên ở runtime, các ca kiểm thử Bước 2 không thể thực thi nếu không có cơ chế thiết lập trạng thái tiền đề (Dynamic Setup Hook).
2. **Lỗ hổng bảo mật nằm ngoài tầm quét của BVA:** Hai khiếm khuyết nguy hiểm nhất (`BUG-FR03-01` trả OTP qua HTTP và `BUG-FR03-03` token vô thời hạn) hoàn toàn không xuất phát từ việc vi phạm giá trị biên của dữ liệu, mà xuất phát từ lỗi thiết kế kiến trúc bảo mật.

3.4 FR-08 — Checkout

3.4.1 Tìm hiểu tính năng và đặc tả thực tế (Fact Collection & Code Investigation)

- **Endpoint chính:** `POST /api/checkout` (Base URL: `http://localhost:3000`) `[SPEC: api_specification.md:130]` `[CODE: server.js:297]`
- **Yêu cầu xác thực:** Bắt buộc token JWT thông qua middleware `authenticateToken` trong header `Authorization: Bearer <token>` `[CODE: server.js:297, 100-110]`.
- **Nguồn đặc tả đã sử dụng:**
  - `docs/api_specification.md` (mục 4.3 và 5.1) `[SPEC]`
  - Mã nguồn Backend: `eshop-sut/backend/server.js:297-309` và `eshop-sut/backend/database.js:74-81` `[CODE]`
  - Mã nguồn Frontend Web: `eshop-sut/frontend-web/src/pages/Checkout.jsx` `[CODE]`
  - Mã nguồn Frontend Mobile: `eshop-sut/frontend-mobile/App.js:384-395` `[CODE]`
  - Minh chứng log thực thi: `HW2/evidence/FR-08/test-execution.log` `[OBSERVED]`
- **Phát hiện quan trọng từ mã nguồn SUT (Code Investigation):**
  - **Backend hoàn toàn không có validation (`server.js:297-309`):** Handler nhận trực tiếp `total_amount` và `shipping_address` từ `req.body` rồi thực thi câu lệnh SQL INSERT mà không có bất kỳ lệnh kiểm tra nào:

    ```
    const { total_amount, shipping_address } = req.body;
    db.run(
      "INSERT INTO orders (user_id, total_amount, status, shipping_address) VALUES (?, ?, ?, ?)",
      [userId, total_amount, "pending", shipping_address],
      function (err) {
        if (err) return res.status(500).json({ error: err.message });
        res.json({ message: "Checkout successful", orderId: this.lastID });
      }
    );
    ```

    Hệ thống chấp nhận số tiền âm (`-50,000`), số 0đ, chuỗi ký tự (`"free"`), kiểu boolean, số thực, giá trị `null`, hoặc tràn số nguyên int32 (`2,147,483,648`).
  - **Lỗ hổng Parameter Tampering nghiêm trọng (`server.js:299` & `Checkout.jsx:93-102`):** Backend hoàn toàn không tính toán lại tổng tiền từ giỏ hàng `userCarts[userId]` hay bảng giá sản phẩm mà tin tưởng tuyệt đối vào giá trị client gửi lên. Đặc biệt, Frontend Web cung cấp hẳn một ô `<input type="number">` gán với state `editableTotal` cho phép người dùng tự do nhập số tiền cần thanh toán tùy thích!
  - **Bỏ quên địa chỉ giao hàng (`server.js:299`, Web & Mobile):** Backend không bắt buộc `shipping_address`, cho phép `null` hoặc chuỗi rỗng. Đáng chú ý, cả giao diện Web (`Checkout.jsx`) và Mobile (`App.js`) đều **hoàn toàn không có trường nhập địa chỉ giao hàng** và không gửi trường này trong payload JSON.
  - **Cho phép đặt đơn hàng khi giỏ hàng rỗng (`cart = []`):** Backend không kiểm tra xem giỏ hàng của người dùng có sản phẩm nào hay không, và sau khi checkout thành công cũng không xóa giỏ hàng.
  - **Dị biệt logic cố ý cắt bớt sản phẩm trên Mobile (`App.js:391`):** Frontend Mobile tự ý thực thi: `items: cart.length > 1 ? cart.slice(0, -1) : cart`. Nếu giỏ hàng có từ 2 sản phẩm trở lên, ứng dụng di động sẽ cố ý loại bỏ sản phẩm cuối cùng khỏi đơn hàng mà người dùng không hề hay biết!
  - **Không kiểm tra tồn kho (Overselling) & Cơ sở dữ liệu thiếu bảng `order_items` (`database.js:74-81`):** Backend không kiểm tra và giảm trừ cột `stock` trong bảng `products`. Hệ thống CSDL SQLite chỉ có bảng `orders` mà không hề có bảng `order_items`, đồng nghĩa với việc đơn hàng được tạo ra nhưng không lưu lại khách đã mua sản phẩm nào!
  - **Thiếu kiểm tra mã giảm giá (`coupon_id`):** Endpoint checkout không kiểm tra tính hợp lệ hay thời hạn của coupon.

3.4.2 Kiểm thử miền (Domain Testing)

Bước 1: Xác định biến Input, Output, State và Miền giá trị (Domains)

Biến	Loại (Kind)	Kiểu dữ liệu	Miền giá trị & Ràng buộc kỹ thuật (Domain)	Nguồn (Source tag)
`total_amount`	Input	Number	- Tổng tiền thanh toán của đơn hàng (VNĐ). - Ràng buộc lý thuyết: Số nguyên dương (`total_amount > 0`). - *Code thực tế:* `server.js:299` không validate, nhận mọi giá trị số âm, chuỗi, boolean, null.	`[SPEC]` api_spec:131 `[CODE]` server.js:299
`shipping_address`	Input	String	- Địa chỉ giao nhận hàng của người mua. - Ràng buộc: Bắt buộc, chuỗi không rỗng, độ dài hợp lý [5, 255] ký tự. - *Code thực tế:* `server.js:299` không validate; Web và Mobile không có UI nhập địa chỉ.	`[SPEC]` api_spec:131 `[CODE]` server.js:299
`cart_state`	State	Array	- Trạng thái danh sách sản phẩm trong giỏ hàng `userCarts[userId]`. - Miền giá trị: `{ EMPTY, NON_EMPTY_VALID, NON_EMPTY_OUT_OF_STOCK }`. - Ràng buộc: Giỏ hàng phải có ít nhất 1 sản phẩm còn tồn kho.	`[CODE]` server.js:284-295 `[ASSUMED]` Business rule
`coupon_id`	Input	String / Null	- Mã coupon giảm giá áp dụng cho đơn hàng. - Miền giá trị: `{ NULL, VALID_COUPON, INVALID_OR_EXPIRED_COUPON }`.	`[CODE]` server.js:363-442 `[CODE]` Checkout.jsx:48
`auth_token`	State/Header	String	- Token JWT xác thực phiên người dùng. - Miền giá trị: `{ VALID_USER_TOKEN, MISSING_TOKEN, INVALID_OR_EXPIRED_TOKEN }`.	`[CODE]` server.js:100-110
`system_output_response`	Output	Object	- Mã HTTP status (200, 400, 401, 403, 500) và JSON response (`orderId`, `message`, `error`).	`[SPEC]` api_spec:135 `[CODE]` server.js:305

Bước 2: Phân hoạch tương đương (Equivalence Partitioning)

Tuân thủ hướng dẫn phân hoạch kiểm thử miền theo bài giảng **S04 Domain Testing (Step 2: Partitioning)**:

- **Range:** 1 lớp hợp lệ trong khoảng, 2 lớp không hợp lệ (dưới cận dưới và trên cận trên).
- **Must be:** 1 lớp hợp lệ, 1 hoặc nhiều lớp không hợp lệ cho mỗi trường hợp sai định dạng/sai loại.
- **State / Precondition:** Phân hoạch các trạng thái xác thực, trạng thái giỏ hàng, tình trạng tồn kho và mã giảm giá.
- **Missing / Empty:** Phân hoạch cho chuỗi rỗng `""`, giá trị null và trường bị bỏ quên trong JSON payload.
- **Dị biệt đa nền tảng:** Phân hoạch lớp dị biệt Mobile tự ý cắt xén sản phẩm cuối cùng (`cart.slice(0, -1)`).
- **Lớp đầu ra (Outputs):** Bao quát đầy đủ các mã HTTP status (200, 400, 401, 403, 500).

Bảng phân hoạch tương đương đầy đủ (Complete Set of Partitions — EC01 đến EC31)

Mã EC	Biến kiểm tra	Mô tả lớp tương đương	Hợp lệ / Không hợp lệ	Quy tắc phân hoạch (Guideline used)	Nguồn (Source tag)
EC01	`auth_token`	Token JWT hợp lệ của Customer đã đăng nhập	**Valid**	Must be: valid JWT signature	`[SPEC]` FR-08 `[CODE]` server.js:100
EC02	`auth_token`	Thiếu hoàn toàn header `Authorization` (khách vãng lai)	Invalid	Missing: header omitted	`[CODE]` server.js:103
EC03	`auth_token`	Token sai chữ ký, bị chỉnh sửa hoặc sai định dạng chuỗi	Invalid	Must be: invalid signature	`[CODE]` server.js:106

Mã EC	Biến kiểm tra	Mô tả lớp tương đương	Hợp lệ / Không hợp lệ	Quy tắc phân hoạch (Guideline used)	Nguồn (Source tag)
EC04	`auth_token`	Header Authorization gửi token rỗng (`Bearer `)	Invalid	Missing/Empty: empty token	`[CODE]` server.js:103
EC05	`total_amount`	Số nguyên dương lớn hơn 0, khớp chính xác giỏ hàng	**Valid**	Range: `total_amount > 0` & Integrity	`[SPEC]` api_spec:131 `[CODE]` server.js:299
EC06	`total_amount`	Số tiền bằng 0 (`total_amount = 0`)	Invalid	Boundary: zero amount	`[SPEC]` api_spec:131
EC07	`total_amount`	Số tiền âm (`total_amount < 0`, ví dụ `-50000`)	Invalid	Range: negative amount (CWE-20)	`[SPEC]` api_spec:131
EC08	`total_amount`	Số tiền dương nhưng lệch giá giỏ hàng (Parameter Tampering)	Invalid	Integrity conflict: tampered price	`[CODE]` Checkout.jsx:55
EC09	`total_amount`	Sai kiểu dữ liệu: String (ví dụ `"free"`)	Invalid	Type: string instead of number	`[CODE]` server.js:299
EC10	`total_amount`	Sai kiểu dữ liệu: Boolean (`true`/`false`)	Invalid	Type: boolean instead of number	`[CODE]` server.js:299
EC11	`total_amount`	Sai kiểu dữ liệu: Số thực float (`199999.5`)	Invalid	Type: float instead of integer	`[SPEC]` api_spec:131
EC12	`total_amount`	Thiếu hoàn toàn trường `total_amount` trong request body `{}`	Invalid	Missing/Empty: field omitted	`[SPEC]` api_spec:131
EC13	`total_amount`	Giá trị `null`	Invalid	Type/Null: null value	`[CODE]` server.js:299
EC14	`shipping_address`	Địa chỉ hợp lệ, đầy đủ thông tin (độ dài 5..255 ký tự)	**Valid**	Range: valid string length	`[SPEC]` api_spec:131
EC15	`shipping_address`	Chuỗi rỗng `""`	Invalid	Missing/Empty: empty string	`[SPEC]` api_spec:131
EC16	`shipping_address`	Chuỗi chỉ chứa khoảng trắng `" "`	Invalid	Missing/Empty: whitespace only	`[CODE]` server.js:299
EC17	`shipping_address`	Thiếu hoàn toàn trường `shipping_address` (Web & Mobile skew)	Invalid	Missing/Empty: field omitted	`[CODE]` Checkout.jsx:55 `[CODE]` App.js:330
EC18	`shipping_address`	Sai kiểu dữ liệu (Number `12345` hoặc Boolean)	Invalid	Type: non-string data type	`[CODE]` server.js:299
EC19	`shipping_address`	Chuỗi địa chỉ cực lớn (> 1,000 ký tự payload)	Invalid	Range: above maximum limit	`[ASSUMED]` Security standard
EC20	`cart_state`	Giỏ hàng có ít nhất 1 sản phẩm hợp lệ (`quantity >= 1`)	**Valid**	State: non-empty cart	`[CODE]` server.js:284
EC21	`cart_state`	Giỏ hàng rỗng (`cart = []` hoặc chưa thêm sản phẩm)	Invalid	State: empty cart checkout	`[CODE]` server.js:284
EC22	`stock_state`	Số lượng sản phẩm trong giỏ $\le$ tồn kho trong DB	**Valid**	State: sufficient stock	`[ASSUMED]` Business rule
EC23	`stock_state`	Số lượng sản phẩm trong giỏ $>$ tồn kho (Overselling)	Invalid	State conflict: insufficient stock	`[ASSUMED]` Business rule
EC24	`coupon_state`	Không áp dụng mã giảm giá (`coupon_id` là null)	**Valid**	Optional field: no coupon	`[CODE]` server.js:306

Mã EC	Biến kiểm tra	Mô tả lớp tương đương	Hợp lệ / Không hợp lệ	Quy tắc phân hoạch (Guideline used)	Nguồn (Source tag)
EC25	`coupon_state`	Áp dụng coupon hợp lệ, còn hạn và tính đúng giảm giá	**Valid**	State: valid coupon applied	`[CODE]` server.js:363
EC26	`coupon_state`	Áp dụng coupon không tồn tại, hết hạn hoặc hết lượt	Invalid	State: invalid coupon applied	`[CODE]` server.js:363
EC27	`mobile_skew`	Giỏ có ≥ 2 sản phẩm, Mobile gửi `cart.slice(0, -1)`	Invalid	Client skew: mobile cuts last item	`[CODE]` App.js:330
EC28	`system_output`	`HTTP 200 OK` kèm `orderId`, đơn hàng tạo thành công	**Valid Output**	Output condition: checkout success	`[CODE]` server.js:306
EC29	`system_output`	`HTTP 401 Unauthorized`	Invalid Output	Output condition: unauthenticated	`[CODE]` server.js:103
EC30	`system_output`	`HTTP 403 Forbidden`	Invalid Output	Output condition: invalid token	`[CODE]` server.js:106
EC31	`system_output`	`HTTP 400 Bad Request`	Invalid Output	Output condition: validation failure	`[SPEC]` api_spec:38-40

Bước 3: Chọn giá trị đại diện cho từng lớp tương đương (Best Representatives)

Theo slide bài giảng S04 (Step 3), mỗi lớp tương đương được chọn đúng một giá trị đại diện điển hình:

- Đối với miền có thứ tự (Ordered domains như số tiền `total_amount`, độ dài địa chỉ): chọn giá trị tiêu biểu hoặc giá trị biên đại diện (`-> BVA`).
- Đối với miền không có thứ tự (Unordered domains như token, trạng thái giỏ, coupon): chọn giá trị thực tế, mô phỏng đúng ngữ cảnh hoạt động.

Mã EC	Biến / Trạng thái kiểm thử	Giá trị đại diện được chọn	Lý do lựa chọn đại diện	Ghi chú kỹ thuật
EC01	`auth_token`	JWT Token hợp lệ của user `test@eshop.com`	Mô phỏng phiên đăng nhập hợp lệ của người dùng thực	Lấy tự động qua API login
EC02	`auth_token`	Bỏ hoàn toàn header `Authorization`	Đại diện cho truy cập trái phép từ khách vãng lai	Kỳ vọng HTTP 401
EC03	`auth_token`	`Bearer invalid.jwt.token.signature`	Đại diện cho token bị chỉnh sửa chữ ký hoặc giả mạo	Kỳ vọng HTTP 403
EC04	`auth_token`	`Bearer ` (chuỗi rỗng sau tiền tố Bearer)	Đại diện cho lỗi logic client gửi header trống	Kỳ vọng HTTP 401
EC05	`total_amount`	`200000` (khớp giỏ hàng có 1 SP giá 200,000)	Số tiền dương nguyên tệ chuẩn, khớp chính xác giỏ hàng	Giá trị hợp lệ danh định
EC06	`total_amount`	`0`	Giá trị biên số 0 không được phép (đặt hàng 0 đồng)	-> Chi tiết tại BVA
EC07	`total_amount`	`-50000`	Số tiền âm vi phạm quy tắc số dương (CWE-20)	Nguy cơ đảo chiều tiền tệ
EC08	`total_amount`	`1000` (giỏ hàng thực tế trị giá `200000`)	Mô phỏng hành vi sửa tiền trong input `editableTotal` trên Web	Lỗ hổng Price Tampering
EC09	`total_amount`	`"free"`	Chuỗi ký tự (String) vi phạm kiểu dữ liệu Number	Lỗi Type validation
EC10	`total_amount`	`true`	Kiểu dữ liệu Boolean vi phạm schema	Lỗi Type validation

Mã EC	Biến / Trạng thái kiểm thử	Giá trị đại diện được chọn	Lý do lựa chọn đại diện	Ghi chú kỹ thuật
EC11	`total_amount`	`199999.5`	Số thực dấu phẩy động (Float) vi phạm tiền tệ nguyên	Kiểm tra tính toàn vẹn số tiền
EC12	`total_amount`	`{}` (thiếu hoàn toàn trường `total_amount`)	Đại diện cho request thiếu trường bắt buộc	Vi phạm schema request
EC13	`total_amount`	`null`	Giá trị null gửi lên qua JSON	Kiểm tra xử lý null của SQLite
EC14	`shipping_address`	`"123 Le Loi, Phuong Ben Nghe, Quan 1, TP.HCM"`	Địa chỉ giao hàng đầy đủ, độ dài hợp lý (45 ký tự)	Dữ liệu giao hàng chuẩn
EC15	`shipping_address`	`""` (chuỗi rỗng)	Địa chỉ rỗng không đủ thông tin giao hàng	Vi phạm điều kiện bắt buộc
EC16	`shipping_address`	`" "` (3 ký tự khoảng trắng)	Khoảng trắng không hợp lệ nếu thiếu hàm `.trim()`	Vi phạm chuỗi rỗng sau trim
EC17	`shipping_address`	Bỏ trống trường `shipping_address` trong JSON	Hành vi gửi request thực tế của cả Web và Mobile	Mâu thuẫn Client - Server
EC18	`shipping_address`	`12345` (kiểu số nguyên Number)	Sai kiểu dữ liệu chuỗi	Lỗi Type validation
EC19	`shipping_address`	Chuỗi 1,500 ký tự `"A".repeat(1500)`	Địa chỉ vượt quá độ dài tối đa hợp lý	Nguy cơ tràn bộ nhớ / DoS
EC20	`cart_state`	Giỏ hàng có 1 sản phẩm hợp lệ (`quantity = 1`)	Điều kiện tiên quyết giỏ hàng có hàng để thanh toán	Trạng thái hợp lệ
EC21	`cart_state`	Giỏ hàng rỗng (`cart = []`)	Điều kiện tiên quyết giỏ hàng rỗng không được checkout	Lỗi logic đặt đơn ma
EC22	`stock_state`	Sản phẩm trong giỏ còn đủ tồn kho (`quantity = 1 <= stock`)	Đáp ứng đủ số lượng hàng tồn trong kho	Trạng thái kho hợp lệ
EC23	`stock_state`	Sản phẩm đặt mua vượt quá tồn kho (`quantity = 999 > stock`)	Đặt mua vượt quá tồn kho thực tế	Nguy cơ Overselling
EC24	`coupon_state`	`coupon_id = null` (không áp dụng coupon)	Đơn hàng thông thường không sử dụng mã giảm giá	Luồng chuẩn
EC25	`coupon_state`	Áp dụng mã coupon hợp lệ (ví dụ: `DISCOUNT10`)	Đơn hàng áp dụng mã giảm giá hợp lệ còn lượt dùng	Luồng giảm giá thành công
EC26	`coupon_state`	Áp dụng mã coupon giả/hết hạn (`FAKECOUPON99`)	Mã coupon không tồn tại hoặc đã hết lượt/hết hạn	Kỳ vọng báo lỗi coupon
EC27	`mobile_skew`	Giỏ có 2 món, Mobile gửi `cart.slice(0, -1)`	Dị biệt mã nguồn Mobile cắt xén sản phẩm cuối cùng	Lỗi logic đa nền tảng
EC28	`system_output`	`HTTP 200 OK` kèm `orderId`	Đặt hàng thành công, tạo bản ghi `orders` (pending)	`[CODE: server.js:306]`
EC29	`system_output`	`HTTP 401 Unauthorized`	Từ chối truy cập khi thiếu token đăng nhập	`[CODE: server.js:103]`
EC30	`system_output`	`HTTP 403 Forbidden`	Từ chối truy cập khi token không hợp lệ	`[CODE: server.js:106]`
EC31	`system_output`	`HTTP 400 Bad Request`	Từ chối request sai dữ liệu (tiền âm, thiếu địa chỉ, giỏ rỗng)	`[SPEC: api_spec:38-40]`

Bước 4: Thiết kế bộ test case tối thiểu (Selecting Test Cases) & Ma trận bao phủ 100%

Tuân thủ nguyên tắc tối ưu hóa số ca kiểm thử của CSC13003 S04:

1. **Lớp hợp lệ (Valid Classes):** Kết hợp tối đa các lớp hợp lệ vào cùng một ca kiểm thử cho đến khi toàn bộ các lớp hợp lệ được bao phủ ít nhất một lần (`FR08-DT-01` và `FR08-DT-02`).
2. **Lớp không hợp lệ (Invalid Classes):** Áp dụng nguyên tắc **Cô lập lỗi đơn (Single-fault isolation)**: Mỗi ca kiểm thử chỉ chứa duy nhất 1 lớp không hợp lệ, tất cả các trường/trạng thái còn lại đều phải ở trạng thái hợp lệ (`FR08-DT-03` đến `FR08-DT-22`).

Bảng danh sách 22 ca kiểm thử Domain Testing chi tiết

Mã TC	Lớp tương đương bao phủ	Token (`auth`)	`total_amount`	`shipping_address`	Trạng thái giỏ / Tiền đề	Kết quả kỳ vọng (`expected`)	Nguồn gốc
FR08-DT-01	EC01, EC05, EC14, EC20, EC22, EC24, EC28	User hợp lệ	`200000`	`"123 Le Loi, TP.HCM"`	Giỏ có 1 SP (200k), đủ kho, không coupon	HTTP 200, `Checkout successful`, có `orderId`	`[CODE]` `[SPEC]`
FR08-DT-02	EC01, EC05, EC14, EC20, EC22, EC25, EC28	User hợp lệ	`180000`	`"123 Le Loi, TP.HCM"`	Giỏ có 1 SP (200k), áp dụng coupon hợp lệ -10%	HTTP 200, `Checkout successful`, có `orderId`	`[CODE]` `[SPEC]`
FR08-DT-03	EC02, EC29	Không gửi header	`200000`	`"123 Le Loi, TP.HCM"`	Giỏ hàng bình thường	HTTP 401 Unauthorized	`[CODE]` server.js:103
FR08-DT-04	EC03, EC30	Token sai chữ ký	`200000`	`"123 Le Loi, TP.HCM"`	Giỏ hàng bình thường	HTTP 403 Forbidden	`[CODE]` server.js:106
FR08-DT-05	EC04, EC29	Token rỗng (`Bearer `)	`200000`	`"123 Le Loi, TP.HCM"`	Giỏ hàng bình thường	HTTP 401 Unauthorized	`[CODE]` server.js:103
FR08-DT-06	EC06, EC31	User hợp lệ	`0`	`"123 Le Loi, TP.HCM"`	Giỏ hàng trị giá 200k	HTTP 400 Bad Request (tiền = 0)	`[ASSUMED]`
FR08-DT-07	EC07, EC31	User hợp lệ	`-50000`	`"123 Le Loi, TP.HCM"`	Giỏ hàng trị giá 200k	HTTP 400 Bad Request (tiền âm)	`[ASSUMED]`
FR08-DT-08	EC08, EC31	User hợp lệ	`1000`	`"123 Le Loi, TP.HCM"`	Giỏ hàng trị giá 200k (Web editableTotal)	HTTP 400 Bad Request (sai lệch giá giỏ)	`[ASSUMED]`
FR08-DT-09	EC09, EC31	User hợp lệ	`"free"`	`"123 Le Loi, TP.HCM"`	Giỏ hàng trị giá 200k	HTTP 400 Bad Request (sai type chuỗi)	`[ASSUMED]`
FR08-DT-10	EC10, EC31	User hợp lệ	`true`	`"123 Le Loi, TP.HCM"`	Giỏ hàng trị giá 200k	HTTP 400 Bad Request (sai type boolean)	`[ASSUMED]`
FR08-DT-11	EC11, EC31	User hợp lệ	`199999.5`	`"123 Le Loi, TP.HCM"`	Giỏ hàng trị giá 200k	HTTP 400 Bad Request (số thực float)	`[ASSUMED]`
FR08-DT-12	EC12, EC31	User hợp lệ	Bỏ trường	`"123 Le Loi, TP.HCM"`	Giỏ hàng trị giá 200k	HTTP 400 Bad Request (thiếu total_amount)	`[ASSUMED]`
FR08-DT-13	EC13, EC31	User hợp lệ	`null`	`"123 Le Loi, TP.HCM"`	Giỏ hàng trị giá 200k	HTTP 400 Bad Request (null total_amount)	`[ASSUMED]`

Mã TC	Lớp tương đương bao phủ	Token (auth)	total_amount	shipping_address	Trạng thái giỏ / Tiền đề	Kết quả kỳ vọng (expected)	Nguồn gốc
FR08-DT-14	EC15, EC31	User hợp lệ	200000	"" (chuỗi rỗng)	Giỏ hàng bình thường	HTTP 400 Bad Request (địa chỉ rỗng)	[ASSUMED]
FR08-DT-15	EC16, EC31	User hợp lệ	200000	" " (khoảng trắng)	Giỏ hàng bình thường	HTTP 400 Bad Request (khoảng trắng)	[ASSUMED]
FR08-DT-16	EC17, EC31	User hợp lệ	200000	Bỏ trường	Giỏ hàng bình thường (Client Web/Mobile)	HTTP 400 Bad Request (thiếu address)	[SPEC] api_spec:135
FR08-DT-17	EC18, EC31	User hợp lệ	200000	12345 (Number)	Giỏ hàng bình thường	HTTP 400 Bad Request (sai kiểu địa chỉ)	[ASSUMED]
FR08-DT-18	EC19, EC31	User hợp lệ	200000	Chuỗi 1,500 ký tự	Giỏ hàng bình thường	HTTP 400 Bad Request (địa chỉ quá dài)	[ASSUMED]
FR08-DT-19	EC21, EC31	User hợp lệ	200000	"123 Le Loi, TP.HCM"	Giỏ hàng rỗng (cart = [])	HTTP 400 Bad Request (giỏ hàng rỗng)	[ASSUMED]
FR08-DT-20	EC23, EC31	User hợp lệ	200000	"123 Le Loi, TP.HCM"	Giỏ hàng đặt vượt tồn kho (qty > stock)	HTTP 400 Bad Request (vượt tồn kho)	[ASSUMED]
FR08-DT-21	EC26, EC31	User hợp lệ	200000	"123 Le Loi, TP.HCM"	Áp dụng mã coupon giả FAKECOUPON99	HTTP 400 Bad Request (coupon sai)	[SPEC] api_spec:163
FR08-DT-22	EC27, EC31	User hợp lệ	200000	"123 Le Loi, TP.HCM"	Giỏ có 2 SP, Mobile gửi cart.slice(0, -1)	HTTP 400 hoặc Cảnh báo lệch giỏ hàng	[CODE] App.js:391

Ma trận bao phủ lớp tương đương (Coverage Matrix)

Mã lớp	Mô tả lớp tương đương	Phân loại	Ca kiểm thử bao phủ (TC IDs)	Trạng thái bao phủ
EC01	Token JWT hợp lệ của người dùng	**Valid**	FR08-DT-01, FR08-DT-02, FR08-DT-06 đến FR08-DT-22	☑ Đã bao phủ
EC02	Thiếu header Authorization	Invalid	FR08-DT-03	☑ Đã bao phủ
EC03	Token sai chữ ký / giả mạo	Invalid	FR08-DT-04	☑ Đã bao phủ
EC04	Token rỗng (Bearer)	Invalid	FR08-DT-05	☑ Đã bao phủ
EC05	total_amount nguyên dương hợp lệ khớp giỏ	**Valid**	FR08-DT-01, FR08-DT-02, FR08-DT-03..05, 14..22	☑ Đã bao phủ
EC06	total_amount = 0	Invalid	FR08-DT-06	☑ Đã bao phủ
EC07	total_amount < 0 (số âm)	Invalid	FR08-DT-07	☑ Đã bao phủ
EC08	total_amount sai lệch giỏ hàng (Web tampering)	Invalid	FR08-DT-08	☑ Đã bao phủ
EC09	total_amount là chuỗi (String)	Invalid	FR08-DT-09	☑ Đã bao phủ
EC10	total_amount là Boolean	Invalid	FR08-DT-10	☑ Đã bao phủ
EC11	total_amount là số thực (Float)	Invalid	FR08-DT-11	☑ Đã bao phủ

Mã lớp	Mô tả lớp tương đương	Phân loại	Ca kiểm thử bao phủ (TC IDs)	Trạng thái bao phủ
EC12	Thiếu hoàn toàn trường `total_amount`	Invalid	FR08-DT-12	☑ Đã bao phủ
EC13	`total_amount = null`	Invalid	FR08-DT-13	☑ Đã bao phủ
EC14	`shipping_address` hợp lệ	**Valid**	FR08-DT-01 đến FR08-DT-13, FR08-DT-19 đến FR08-DT-22	☑ Đã bao phủ
EC15	`shipping_address = ""` (chuỗi rỗng)	Invalid	FR08-DT-14	☑ Đã bao phủ
EC16	`shipping_address = " "` (khoảng trắng)	Invalid	FR08-DT-15	☑ Đã bao phủ
EC17	Thiếu trường `shipping_address` (Web & Mobile)	Invalid	FR08-DT-16	☑ Đã bao phủ
EC18	`shipping_address` sai kiểu (Number)	Invalid	FR08-DT-17	☑ Đã bao phủ
EC19	`shipping_address` vượt 1,000 ký tự	Invalid	FR08-DT-18	☑ Đã bao phủ
EC20	Giỏ hàng có sản phẩm hợp lệ	**Valid**	FR08-DT-01 đến FR08-DT-18, FR08-DT-21, FR08-DT-22	☑ Đã bao phủ
EC21	Giỏ hàng rỗng ($cart = []$)	Invalid	FR08-DT-19	☑ Đã bao phủ
EC22	Tồn kho đủ để bán	**Valid**	FR08-DT-01 đến FR08-DT-19, FR08-DT-21, FR08-DT-22	☑ Đã bao phủ
EC23	Đặt mua vượt quá tồn kho	Invalid	FR08-DT-20	☑ Đã bao phủ
EC24	Không dùng coupon (`coupon_id = null`)	**Valid**	FR08-DT-01, FR08-DT-03 đến FR08-DT-20, FR08-DT-22	☑ Đã bao phủ
EC25	Dùng coupon hợp lệ	**Valid**	FR08-DT-02	☑ Đã bao phủ
EC26	Dùng coupon không hợp lệ / hết hạn	Invalid	FR08-DT-21	☑ Đã bao phủ
EC27	Dị biệt Mobile tự cắt sản phẩm cuối giỏ	Invalid	FR08-DT-22	☑ Đã bao phủ
EC28	Output: `HTTP 200 OK` + `orderId`	**Valid Output**	FR08-DT-01, FR08-DT-02	☑ Đã bao phủ
EC29	Output: `HTTP 401 Unauthorized`	Invalid Output	FR08-DT-03, FR08-DT-05	☑ Đã bao phủ
EC30	Output: `HTTP 403 Forbidden`	Invalid Output	FR08-DT-04	☑ Đã bao phủ
EC31	Output: `HTTP 400 Bad Request`	Invalid Output	FR08-DT-06 đến FR08-DT-22	☑ Đã bao phủ

Độ bao phủ: 31/31 lớp EC (100%).

3.4.3 Phân tích giá trị biên (Boundary Value Analysis - BVA)

Bước 1: Xác định biến có miền thứ tự

- **Biến có thứ tự áp dụng BVA:**
  1. `total_amount`: Miền số nguyên giá trị tiền $[1, 2,147,483,647]$ VNĐ (cận trên theo giới hạn signed int32 thông dụng của CSDL).
  2. `shipping_address.length`: Miền số nguyên độ dài chuỗi $[5, 255]$ ký tự.
  3. `cart.item_count`: Miền số lượng mặt hàng trong giỏ $[1, 50]$ sản phẩm (đặc biệt nhạy cảm với cận dưới 1 và 2 do code Mobile).
- **Biến không áp dụng BVA:**
  - `auth_token`: Chuỗi token JWT mã hóa Base64 không có quan hệ thứ tự độ lớn tự nhiên.
  - `coupon_id`: Mã định danh chuỗi rời rạc.

Bước 2: Điểm biên (3-value BVA + Mô hình 9 điểm + Biên mã hóa UTF-8)

- `total_amount`: $LB-1 = 0$, $LB = 1$, $LB+1 = 2$, $UB-1 = 2,147,483,646$, $UB = 2,147,483,647$, $UB+1 = 2,147,483,648$, cực trị âm $-\alpha = -50,000$, cực trị dương $+\alpha = 9,007,199,254,740,991$ (`Number.MAX_SAFE_INTEGER`).
- `shipping_address.length`: $LB-1 = 4$, $LB = 5$, $LB+1 = 6$, $UB-1 = 254$, $UB = 255$, $UB+1 = 256$, cực trị 0 và $10,000$. Bổ sung test case **255 ký tự tiếng Việt có dấu (UTF-8 encoding)** theo quy tắc v2.0 Rule.
- `cart.item_count`: $LB-1 = 0$ (giỏ rỗng), $LB = 1$ (giỏ 1 món - Mobile không cắt), $LB+1 = 2$ (giỏ 2 món - Mobile kích hoạt cắt món).

Bước 3: Danh sách 20 ca kiểm thử BVA

Áp dụng nguyên tắc **Đơn biến (Single-fault assumption)**: Mỗi ca kiểm thử BVA chỉ thay đổi một biến duy nhất tại một điểm biên, toàn bộ các biến còn lại được giữ cố định ở giá trị danh định hợp lệ (Nominal valid values).

Bảng chi tiết 20 ca kiểm thử giá trị biên (BVA Test Suite)

Mã TC	Biến kiểm tra	Điểm biên	Giá trị kiểm thử cụ thể	Các biến khác (Nominal)	Kỳ vọng (Expected)	Lỗi code sẽ bắt được (Fault it would catch)	Nguồn
FR08-BVA-01	`total_amount`	$-\alpha$ (Extreme số âm)	`-50000` VNĐ	`token`: Valid `address`: Valid `cart`: 1 món	HTTP 400	Lỗ hổng cho phép thanh toán số tiền âm, đảo chiều dòng tiền giao dịch (CWE-20)	`[SPEC]` api_spec:134
FR08-BVA-02	`total_amount`	$LB - 1$ (Cận dưới - 1)	`0` VNĐ	`token`: Valid `address`: Valid `cart`: 1 món	HTTP 400	Lập trình viên viết nhầm điều kiện `if (amount < 0)` thay vì `<= 0`, cho phép đặt hàng miễn phí 0 đồng	`[SPEC]` api_spec:134
FR08-BVA-03	`total_amount`	LB (Cận dưới tối thiểu)	`1` VNĐ	`token`: Valid `address`: Valid `cart`: 1 món (giá 1đ)	HTTP 200	Lập trình viên viết nhầm điều kiện `if (amount > 1)` thay vì `>= 1`, từ chối đơn hàng hợp lệ nhỏ nhất	`[SPEC]` api_spec:134
FR08-BVA-04	`total_amount`	$LB + 1$ (Nội biên kề cận)	`2` VNĐ	`token`: Valid `address`: Valid `cart`: 1 món (giá 2đ)	HTTP 200	Lỗi lệch biên cận dưới nội vùng	`[SPEC]` api_spec:134
FR08-BVA-05	`total_amount`	$UB - 1$ (Cận trên - 1)	`2,147,483,646` VNĐ	`token`: Valid `address`: Valid	HTTP 200	Lỗi lệch biên cận trên giá trị giao dịch lớn	`[ASSUMED]` int32 max
FR08-BVA-06	`total_amount`	UB (Cận trên int32 max)	`2,147,483,647` VNĐ	`token`: Valid `address`: Valid	HTTP 200	Lập trình viên viết nhầm `< 2147483647` thay vì `<= 2147483647`	`[ASSUMED]` int32 max
FR08-BVA-07	`total_amount`	$UB + 1$ (Ngoài biên int32)	`2,147,483,648` VNĐ	`token`: Valid `address`: Valid	HTTP 400	Lỗi không giới hạn số tiền tối đa, gây tràn số nguyên (Integer Overflow) trong SQLite	`[ASSUMED]` int32 overflow

Mã TC	Biến kiểm tra	Điểm biên	Giá trị kiểm thử cụ thể	Các biến khác (Nominal)	Kỳ vọng (Expected)	Lỗi code sẽ bắt được (Fault it would catch)	Nguồn
FR08-BVA-08	`total_amount`	$+\alpha$ (Max Extreme)	`9,007,199,254,740,991` (`MAX_SAFE_INT`)	`token`: Valid `address`: Valid	HTTP 400	Lỗi mất độ chính xác số học IEEE 754 float / BigInt trong JavaScript	`[ASSUMED]` JS float limit
FR08-BVA-09	`shipping_address`	$-\alpha$ (Min Extreme)	Độ dài = 0 (chuỗi rỗng `""`)	`token`: Valid `total`: 200,000	HTTP 400	Chấp nhận đơn hàng không có địa chỉ giao, gây lỗi vận hành logistics	`[SPEC]` api_spec:135
FR08-BVA-10	`shipping_address`	$LB - 1$ (Thiếu 1 ký tự)	Độ dài = 4 ký tự (`"Q.1 "`)	`token`: Valid `total`: 200,000	HTTP 400	Chấp nhận địa chỉ quá ngắn, không có giá trị giao nhận	`[ASSUMED]` Address form
FR08-BVA-11	`shipping_address`	LB (Cận dưới tối thiểu)	Độ dài = 5 ký tự (`"Quan1"`)	`token`: Valid `total`: 200,000	HTTP 200	Lập trình viên viết nhầm `length > 5` thay vì `>= 5`, từ chối địa chỉ tối thiểu hợp lệ	`[ASSUMED]` Address form
FR08-BVA-12	`shipping_address`	$LB + 1$ (Nội biên kề cận)	Độ dài = 6 ký tự (`"Quan 1"`)	`token`: Valid `total`: 200,000	HTTP 200	Lỗi lệch biên cận dưới nội vùng	`[ASSUMED]` Address form
FR08-BVA-13	`shipping_address`	$UB - 1$ (Nội biên cận trên)	Độ dài = 254 ký tự (ASCII hợp lệ)	`token`: Valid `total`: 200,000	HTTP 200	Lỗi lệch biên cận trên nội vùng	`[ASSUMED]` 255 chars
FR08-BVA-14	`shipping_address`	UB (Cận trên tối đa ASCII)	Độ dài = 255 ký tự (ASCII hợp lệ)	`token`: Valid `total`: 200,000	HTTP 200	Lập trình viên viết nhầm `< 255` thay vì `<= 255`	`[ASSUMED]` 255 chars
FR08-BVA-15	`shipping_address`	UB (UTF-8 Diacritics v2.0)	Độ dài = 255 ký tự (Tiếng Việt có dấu, 330 bytes)	`token`: Valid `total`: 200,000	HTTP 200	Server kiểm tra sai theo số byte (`Buffer.byteLength`) thay vì số ký tự (`string.length`), làm lỗi địa chỉ tiếng Việt hợp lệ	`[ASSUMED]` UTF-8 Rule v2.0
FR08-BVA-16	`shipping_address`	$UB + 1$ (Vượt quá cận trên)	Độ dài = 256 ký tự (ASCII)	`token`: Valid `total`: 200,000	HTTP 400	Không kiểm soát độ dài tối đa cột `VARCHAR(255)`, gây lỗi DB hoặc cắt cụt dữ liệu ngầm (Silent Truncation)	`[ASSUMED]` 255 chars
FR08-BVA-17	`shipping_address`	$+\alpha$ (Extreme Payload DoS)	Độ dài = 10,000 ký tự	`token`: Valid `total`: 200,000	HTTP 400 / 413	Lỗ hổng tràn bộ nhớ / DoS khi client gửi payload địa chỉ siêu lớn	`[ASSUMED]` DoS Payload
FR08-BVA-18	`cart_items_count`	$LB - 1$ (Giỏ hàng rỗng)	0 sản phẩm trong giỏ hàng	`token`: Valid `total`: 200,000 `address`: Valid	HTTP 400	Backend không kiểm tra giỏ hàng, cho phép đặt "đơn hàng ma" khi không có sản phẩm nào	`[ASSUMED]` Business rule

Mã TC	Biến kiểm tra	Điểm biên	Giá trị kiểm thử cụ thể	Các biến khác (Nominal)	Kỳ vọng (Expected)	Lỗi code sẽ bắt được (Fault it would catch)	Nguồn
FR08-BVA-19	`cart_items_count`	LB (Mobile Skew Boundary)	1 sản phẩm trong giỏ hàng	`token`: Valid `total`: 200,000 `address`: Valid	HTTP 200	Kiểm tra biên 1 sản phẩm: trên Mobile giỏ hàng không bị cắt bỏ (`cart.length > 1 ? cart.slice(0, -1) : cart`)	`[CODE]` App.js:391
FR08-BVA-20	`cart_items_count`	$LB + 1$ (Mobile Skew Boundary)	2 sản phẩm trong giỏ hàng	`token`: Valid `total`: 400,000 `address`: Valid	HTTP 200	Phát hiện lỗi dị biệt Mobile cố tình cắt bỏ sản phẩm cuối cùng của đơn hàng khi `cart.length = 2`	`[CODE]` App.js:391

3.4.4 Kết quả thực thi kiểm thử chi tiết

Toàn bộ **42 ca kiểm thử** của FR-08 đã được thực thi tự động qua test runner đối chiếu trực tiếp với SUT Backend đang chạy tại `http://localhost:3000`:

- **Tổng số thiết kế (Designed):** 42
- **Đã thực thi (Executed):** 42 (100%)
- **Thành công (PASS):** 16 (38.1%)
- **Thất bại (FAIL):** 26 (61.9%)
- **Bị chặn (BLOCKED):** 0 (0%)

Bảng kết quả thực thi chi tiết 42 test cases

Mã TC	Kỹ thuật	Mô tả ca kiểm thử	Kỳ vọng (Expected)	Thực tế (Actual)	Trạng thái
`FR08-DT-01`	DT	Checkout hợp lệ chuẩn không dùng coupon	HTTP 200, orderId	HTTP 200, orderId: 1	**PASS**
`FR08-DT-02`	DT	Checkout hợp lệ có mã giảm giá (-10%)	HTTP 200, orderId	HTTP 200, orderId: 2	**PASS**
`FR08-DT-03`	DT	Từ chối khi thiếu header Authorization	HTTP 401 Unauthorized	HTTP 401 Unauthorized	**PASS**
`FR08-DT-04`	DT	Từ chối khi token sai chữ ký / tampered	HTTP 403 Forbidden	HTTP 403 Forbidden	**PASS**
`FR08-DT-05`	DT	Từ chối khi token rỗng (`Bearer `)	HTTP 401 Unauthorized	HTTP 401 Unauthorized	**PASS**
`FR08-DT-06`	DT	Từ chối khi `total_amount = 0` (mua 0 đồng)	HTTP 400 Bad Request	HTTP 200, orderId: 3	**FAIL**
`FR08-DT-07`	DT	Từ chối khi `total_amount < 0` (số âm `-50,000`)	HTTP 400 Bad Request	HTTP 200, orderId: 4	**FAIL**
`FR08-DT-08`	DT	Từ chối khi `total_amount` bị sửa sai giá giỏ hàng (Web skew)	HTTP 400 Bad Request	HTTP 200, orderId: 5	**FAIL**
`FR08-DT-09`	DT	Từ chối khi `total_amount` là chuỗi (`"free"`)	HTTP 400 Bad Request	HTTP 200, orderId: 6	**FAIL**
`FR08-DT-10`	DT	Từ chối khi `total_amount` là boolean (`true`)	HTTP 400 Bad Request	HTTP 200, orderId: 7	**FAIL**
`FR08-DT-11`	DT	Từ chối khi `total_amount` là số thực (`199999.5`)	HTTP 400 Bad Request	HTTP 200, orderId: 8	**FAIL**

Mã TC	Kỹ thuật	Mô tả ca kiểm thử	Kỳ vọng (Expected)	Thực tế (Actual)	Trạng thái
FR08-DT-12	DT	Từ chối khi thiếu trường total_amount trong body	HTTP 400 Bad Request	HTTP 200, orderId: 9	**FAIL**
FR08-DT-13	DT	Từ chối khi total_amount là null	HTTP 400 Bad Request	HTTP 200, orderId: 10	**FAIL**
FR08-DT-14	DT	Từ chối khi shipping_address là chuỗi rỗng ""	HTTP 400 Bad Request	HTTP 200, orderId: 11	**FAIL**
FR08-DT-15	DT	Từ chối khi shipping_address chỉ chứa khoảng trắng " "	HTTP 400 Bad Request	HTTP 200, orderId: 12	**FAIL**
FR08-DT-16	DT	Từ chối khi thiếu shipping_address (Web & Mobile skew)	HTTP 400 Bad Request	HTTP 200, orderId: 13	**FAIL**
FR08-DT-17	DT	Từ chối khi shipping_address sai kiểu (Number 12345)	HTTP 400 Bad Request	HTTP 200, orderId: 14	**FAIL**
FR08-DT-18	DT	Từ chối khi shipping_address payload lớn > 1,000 ký tự	HTTP 400 Bad Request	HTTP 200, orderId: 15	**FAIL**
FR08-DT-19	DT	Từ chối khi giỏ hàng rỗng (cart = [])	HTTP 400 Bad Request	HTTP 200, orderId: 16	**FAIL**
FR08-DT-20	DT	Từ chối khi đặt hàng vượt tồn kho (Overselling)	HTTP 400 Bad Request	HTTP 200, orderId: 17	**FAIL**
FR08-DT-21	DT	Từ chối khi áp dụng mã coupon không tồn tại/hết hạn	HTTP 400 Bad Request	HTTP 200, orderId: 18	**FAIL**
FR08-DT-22	DT	Kiểm tra dị biệt Mobile tự cắt sản phẩm (cart.slice(0, -1))	HTTP 400 Bad Request	HTTP 200, orderId: 19	**FAIL**
FR08-BVA-01	BVA	total_amount = -50000 ($-\alpha$ Extreme số âm)	HTTP 400 Bad Request	HTTP 200, orderId: 20	**FAIL**
FR08-BVA-02	BVA	total_amount = 0 ($LB - 1$ cận dưới)	HTTP 400 Bad Request	HTTP 200, orderId: 21	**FAIL**
FR08-BVA-03	BVA	total_amount = 1 VNĐ (LB tối thiểu)	HTTP 200 OK	HTTP 200, orderId: 22	**PASS**
FR08-BVA-04	BVA	total_amount = 2 VNĐ ($LB + 1$ nội biên)	HTTP 200 OK	HTTP 200, orderId: 23	**PASS**
FR08-BVA-05	BVA	total_amount = 2,147,483,646 VNĐ ($UB - 1$)	HTTP 200 OK	HTTP 200, orderId: 24	**PASS**
FR08-BVA-06	BVA	total_amount = 2,147,483,647 VNĐ (UB int32 max)	HTTP 200 OK	HTTP 200, orderId: 25	**PASS**
FR08-BVA-07	BVA	total_amount = 2,147,483,648 VNĐ ($UB + 1$ tràn int32)	HTTP 400 Bad Request	HTTP 200, orderId: 26	**FAIL**
FR08-BVA-08	BVA	total_amount = 9,007,199,254,740,991 ($+\alpha$ MAX_SAFE_INT)	HTTP 400 Bad Request	HTTP 200, orderId: 27	**FAIL**
FR08-BVA-09	BVA	shipping_address length = 0 (chuỗi rỗng $-\alpha$)	HTTP 400 Bad Request	HTTP 200, orderId: 28	**FAIL**
FR08-BVA-10	BVA	shipping_address length = 4 ($LB - 1$ thiếu 1 ký tự)	HTTP 400 Bad Request	HTTP 200, orderId: 29	**FAIL**
FR08-BVA-11	BVA	shipping_address length = 5 (LB tối thiểu hợp lệ)	HTTP 200 OK	HTTP 200, orderId: 30	**PASS**
FR08-BVA-12	BVA	shipping_address length = 6 ($LB + 1$ nội biên)	HTTP 200 OK	HTTP 200, orderId: 31	**PASS**

Mã TC	Kỹ thuật	Mô tả ca kiểm thử	Kỳ vọng (Expected)	Thực tế (Actual)	Trạng thái
FR08-BVA-13	BVA	shipping_address length = 254 ($UB - 1$ nội biên)	HTTP 200 OK	HTTP 200, orderId: 32	**PASS**
FR08-BVA-14	BVA	shipping_address length = 255 ASCII (UB cận trên)	HTTP 200 OK	HTTP 200, orderId: 33	**PASS**
FR08-BVA-15	BVA	shipping_address length = 255 UTF-8 tiếng Việt có dấu (v2.0)	HTTP 200 OK	HTTP 200, orderId: 34	**PASS**
FR08-BVA-16	BVA	shipping_address length = 256 ($UB + 1$ vượt chuẩn 255)	HTTP 400 Bad Request	HTTP 200, orderId: 35	**FAIL**
FR08-BVA-17	BVA	shipping_address length = 10,000 ($+\alpha$ Payload DoS)	HTTP 400 / 413	HTTP 200, orderId: 36	**FAIL**
FR08-BVA-18	BVA	cart_items_count = 0 ($LB - 1$ giỏ rỗng)	HTTP 400 Bad Request	HTTP 200, orderId: 37	**FAIL**
FR08-BVA-19	BVA	cart_items_count = 1 (LB giỏ 1 món - Mobile không cắt)	HTTP 200 OK	HTTP 200, orderId: 38	**PASS**
FR08-BVA-20	BVA	cart_items_count = 2 ($LB + 1$ Mobile kích hoạt cắt món)	HTTP 200 OK	HTTP 200, orderId: 39	**PASS**

3.4.5 Lỗi phần mềm phát hiện (Defects Discovered)

26 ca kiểm thử thất bại được tổng hợp và quy nạp về **7 lỗi phần mềm độc lập (Bugs)**:

Mã lỗi (Bug ID)	Tóm tắt khiếm khuyết	Các TC phát hiện	Mức độ nghiêm trọng (Severity)	Độ ưu tiên (Priority)	Liên kết tài liệu
BUG-FR08-01	Backend hoàn toàn thiếu validation cho trường total_amount (chấp nhận tiền âm, 0đ, chuỗi, boolean, float, null, tràn số int32)	FR08-DT-06 – 13, FR08-BVA-01, 02, 07, 08 (11 TCs)	**Critical**	**Immediate**	[BUG-FR08-01.md] (file:///d:/HCMUS/semester7/software-testing/Software-Testing/HW2/report/issues/BUG-FR08-01.md)
BUG-FR08-02	Lỗ hổng Parameter Tampering: Backend không tính lại tổng tiền từ giỏ hàng; Frontend Web cung cấp ô input tự sửa tiền	FR08-DT-08 (1 TC)	**Critical**	**Immediate**	[BUG-FR08-02.md] (file:///d:/HCMUS/semester7/software-testing/Software-Testing/HW2/report/issues/BUG-FR08-02.md)
BUG-FR08-03	Backend hoàn toàn thiếu validation cho shipping_address; Frontend Web và Mobile không có ô nhập địa chỉ giao hàng	FR08-DT-14 – 18, FR08-BVA-09, 10, 16, 17 (9 TCs)	**Major**	**High**	[BUG-FR08-03.md] (file:///d:/HCMUS/semester7/software-testing/Software-Testing/HW2/report/issues/BUG-FR08-03.md)
BUG-FR08-04	Backend không kiểm tra trạng thái giỏ hàng khi checkout (Cho phép đặt đơn hàng ma khi giỏ hàng rỗng)	FR08-DT-19, FR08-BVA-18 (2 TCs)	**Major**	**High**	[BUG-FR08-04.md] (file:///d:/HCMUS/semester7/software-testing/Software-Testing/HW2/report/issues/BUG-FR08-04.md)
BUG-FR08-05	Dị biệt logic nghiêm trọng trên Mobile: Frontend Mobile tự ý cắt xén sản phẩm cuối cùng trong giỏ hàng khi checkout	FR08-DT-22, FR08-BVA-20 (2 TCs)	**Critical**	**Immediate**	[BUG-FR08-05.md] (file:///d:/HCMUS/semester7/software-testing/Software-Testing/HW2/report/issues/BUG-FR08-05.md)

Mã lỗi (Bug ID)	Tóm tắt khiếm khuyết	Các TC phát hiện	Mức độ nghiêm trọng (Severity)	Độ ưu tiên (Priority)	Liên kết tài liệu
BUG-FR08-06	Backend không kiểm tra tồn kho (Overselling Defect) và không lưu chi tiết các mặt hàng trong đơn hàng (`order_items`)	`FR08-DT-20` (1 TC)	**Major**	**High**	[BUG-FR08-06.md] (file:///d:/HCMUS/semester7/software-testing/Software-Testing/HW2/report/issues/BUG-FR08-06.md)
BUG-FR08-07	Endpoint checkout không kiểm tra tính hợp lệ của mã giảm giá `coupon_id`	`FR08-DT-21` (1 TC)	**Minor**	**Medium**	[BUG-FR08-07.md] (file:///d:/HCMUS/semester7/software-testing/Software-Testing/HW2/report/issues/BUG-FR08-07.md)

3.4.6 Phân tích khoảng trống của AI (AI Gap Analysis & Nâng cấp v2.2)

Quá trình kiểm thử FR-08 đã làm bộc lộ 4 khoảng trống AI quan trọng, dẫn đến việc tiếp tục nâng cấp bộ kỹ năng lên **phiên bản v2.2**:

Mục bị bỏ sót / Thách thức thực tế	Cách phát hiện	Vì sao AI bỏ sót ban đầu	Phân loại nguyên nhân (Category)	Hành động cải tiến Skill (v2.1 → v2.2)
1. Thiên kiến tin tưởng tham số tài chính client (Client-trusted Financial Parameter Bias)	Đọc `server.js:299` thấy backend nhận thẳng `total_amount` từ body mà không tính lại từ giỏ	AI theo quán tính coi tham số trong body là độc lập, không tự động kích hoạt nguyên tắc "Toàn vẹn giao dịch tài chính" (server phải tự tính lại tiền)	`Inherent feature complexity` & `Tool limitation`	Bổ sung **Financial Transaction Integrity Rule** vào `domain-testing` và `test-execution-and-bugs`: Mọi tham số tiền bạc gửi từ client đều phải coi là Untrusted Input.
2. Lỗ hổng giao diện cho phép sửa dữ liệu chỉ đọc (Frontend UI Input Defect)	Đọc `Checkout.jsx:93-102` thấy ô input `type="number"` gán với `setEditableTotal`	AI khi chỉ tập trung kiểm thử API sẽ có "điểm mù" về UI: không kiểm tra các thuộc tính `readOnly`, `disabled` trên client	`Tool limitation`	Mở rộng quy tắc **Cross-Platform Inspection**: Luôn kiểm tra các thuộc tính form (`readOnly`, `disabled`, `type`) trên cả Web và Mobile UI.
3. Bỏ sót thực thể cốt lõi trong DB (`order_items`) & Bán vượt tồn kho (Overselling)	Khảo sát `database.js:74-81` và `server.js:301` thấy bảng `orders` không nối với sản phẩm và không trừ `stock`	Spec không mô tả cấu trúc database; AI nếu chỉ bám theo spec API sẽ bỏ qua kiểm tra tính toàn vẹn của giỏ hàng và kho	`Prompt/input quality` & `Inherent feature complexity`	Bổ sung **Database Schema & Inventory Lifecycle Checklist**: Luôn đối chiếu cấu trúc bảng DB để xác nhận đơn hàng có lưu chi tiết món hàng (`order_items`) và có trừ tồn kho hay không.
4. Dị biệt mã nguồn cố ý làm sai lệch dữ liệu (`cart.slice(0, -1)`)	Khảo sát `App.js:391` trên Mobile phát hiện lệnh cắt bỏ sản phẩm cuối cùng của giỏ hàng	Lỗi cố tình can thiệp biến dạng dữ liệu trên client thay vì vô tình lập trình sai; AI thông thường chỉ tìm lỗi off-by-one hoặc thiếu check null	`Inherent feature complexity`	Tích hợp **Client-Side Data Mutation Inspection**: Rà soát các thao tác biến đổi mảng/object (`slice`, `pop`, `filter`) trên client trước khi gửi payload.

3.4.7 Hạn chế của kỹ thuật

1. **Ràng buộc nghiệp vụ và tính toàn vẹn giao dịch nằm ngoài tầm kiểm soát của BVA:** Khiếm khuyết nghiêm trọng nhất của FR-08 (`BUG-FR08-02` Parameter Tampering) hoàn toàn không nằm ở các giá trị biên của số tiền (1đ hay 2 tỷ), mà nằm ở quy tắc nghiệp vụ: *client không bao giờ được phép tự quyết định số tiền phải trả*. BVA chỉ kiểm tra xem số tiền có nằm trong khoảng cho phép hay không, chứ không thể phát hiện việc số tiền đó có khớp với giỏ hàng hay không.
2. **Khiếm khuyết kiến trúc dữ liệu và logic dị biệt client:** Việc cơ sở dữ liệu thiếu bảng `order_items` (`BUG-FR08-06`) và Frontend Mobile tự cắt sản phẩm (`BUG-FR08-05`) là các khiếm khuyết thuộc về kiến trúc hệ thống và mã client, không thể phát hiện được nếu chỉ áp dụng kỹ thuật kiểm thử biên trên endpoint API đơn lẻ.

3.5 FR-12 — Access Control

3.5.1 Tìm hiểu tính năng và đặc tả thực tế

- **Phạm vi tính năng:** Kiểm soát quyền truy cập và phân quyền người dùng (Access control) theo `docs/eshop-features.md` và `docs/api_specification.md` (mục 6. Phân quyền và Bảo mật).
- **Các nhóm endpoint khảo sát (14 endpoints):**
  - Quản lý người dùng admin: `GET /api/admin/users`, `DELETE /api/admin/users/:id [SPEC: api_spec:120-128]`.
  - Quản lý đơn hàng admin: `GET /api/admin/orders`, `POST /api/admin/orders/:id/status [SPEC: api_spec:130-138]`.
  - Quản lý danh mục sản phẩm: `POST /api/categories`, `DELETE /api/categories/:id [SPEC: api_spec:100-108]`.
  - Quản lý sản phẩm: `POST /api/products`, `PUT /api/products/:id`, `DELETE /api/products/:id [SPEC: api_spec:60-70]`.
  - Quản lý mã giảm giá coupon: `POST /api/coupons`, `DELETE /api/coupons/:id [SPEC: api_spec:140-148]`.
  - Hồ sơ người dùng cá nhân: `GET /api/users/me`, `PUT /api/users/me [SPEC: api_spec:25-32]`.
- **Nguồn đặc tả đã sử dụng:**
  - `docs/api_specification.md` (mục 6 Phân quyền và Bảo mật) `[SPEC]`
  - Mã nguồn Backend: `eshop-sut/backend/server.js:100-301` và `database.js:45-75 [CODE]`
  - Quan sát thực thi qua HTTP API Runner: `[OBSERVED]`
- **Phát hiện quan trọng từ mã nguồn SUT (Code Investigation):**
  - **Middleware `authenticateToken` thiếu kiểm tra phân quyền (`server.js:100-110`):** Hàm này chỉ gọi `jwt.verify(token, SECRET_KEY)` để giải mã JWT và gán `req.user = user`. Hoàn toàn không có câu lệnh kiểm tra vai trò `admin`.
  - **Tất cả các route quản trị đều thiếu kiểm tra vai trò (`server.js:231-301`):** Các route `GET /api/admin/users`, `GET /api/admin/orders`, `POST /api/categories`, `POST /api/coupons`, v.v. chỉ gắn `authenticateToken` mà không kiểm tra `if (req.user.role !== 'admin') return res.status(403)...`. Do đó, người dùng thông thường (`role: 'user'`) có thể truy cập và thao tác toàn quyền như quản trị viên tối cao.
  - **Nhóm route sản phẩm (`POST/PUT/DELETE /api/products`) bỏ quên xác thực (`server.js:167`):** Các endpoint thay đổi kho hàng hoàn toàn là public endpoints, bất kỳ ai ngoài Internet (khách vãng lai unauthenticated) cũng có thể tạo, chỉnh sửa hoặc xóa sản phẩm.
  - **Lỗ hổng gán tham số hàng loạt (Mass Assignment) tại `PUT /api/users/me` (`server.js:124-127`):** Endpoint cập nhật thông tin cá nhân nhận trực tiếp trường `role` từ request body và ghi đè vào DB, cho phép người dùng thường tự leo quyền thành Admin.

3.5.2 Kiểm thử miền (Domain Testing)

Bước 1: Xác định biến Input, Output, State và Miền giá trị (Domains)

Biến	Loại (Kind)	Kiểu dữ liệu	Miền giá trị & Ràng buộc kỹ thuật (Domain)	Nguồn (Source tag)
`Authorization`	Input	String	- Header chứa JWT Bearer token xác thực: `Bearer <token>`. - Ràng buộc: Đúng tiền tố `Bearer` , đúng chữ ký HMAC Secret, đúng cấu trúc 3 phần `header.payload.sig`.	`[SPEC]` api_spec:120 `[CODE]` server.js:101-106
`token_role`	Input	String (Claim)	- Quyền hạn lưu trong payload JWT: `{ role: 'admin' }` hoặc `{ role: 'user' }`. - Ràng buộc: Các tài nguyên quản trị chỉ cho phép `admin`. Nếu là `user`, phải trả về HTTP 403 Forbidden.	`[SPEC]` api_spec:120 `[CODE]` server.js:106
`target_endpoint`	Input	String	- Endpoint API đích được gọi trong request. - Phân nhóm: Admin APIs, Category APIs, Product write APIs, Coupon APIs, User profile APIs.	`[SPEC]` api_spec:60-148 `[CODE]` server.js:120-301
`request_body`	Input	Object	- Dữ liệu kèm theo request (danh mục, sản phẩm, coupon, profile). - Rủi ro over-posting: Trường `role` gửi kèm trong `PUT /api/users/me`.	`[SPEC]` api_spec:28 `[CODE]` server.js:124

Biến	Loại (Kind)	Kiểu dữ liệu	Miền giá trị & Ràng buộc kỹ thuật (Domain)	Nguồn (Source tag)
`token_exp_state`	State	Enum	- Trạng thái thời hạn của token JWT: `{ ACTIVE (exp > now), EXPIRED (exp <= now) }`.	`[CODE]` server.js:105 `[SPEC]` JWT RFC 7519
`user_account_status`	State	Enum	- Trạng thái tài khoản người dùng trong SQLite: `{ ACTIVE_USER, ACTIVE_ADMIN, NON_EXISTENT }`.	`[CODE]` database.js:53 `[ASSUMED]` State Model
`privilege_state`	State	Enum	- Trạng thái quyền hạn của tài khoản: `{ NORMAL_USER, ESCALATED_ADMIN }`.	`[CODE]` server.js:126
`Response.status`	Output	Number	- Mã HTTP status trả về: `{ 200, 201, 400, 401, 403, 404, 500 }`.	`[SPEC]` api_spec:124 `[CODE]` server.js:100-301

Bước 2: Phân hoạch tương đương (Equivalence Partitioning)

Phân hoạch thành **23 lớp tương đương** (`EC01` đến `EC23`) bao quát phân quyền và xác thực:

Mã EC	Biến / Ngữ cảnh	Mô tả lớp tương đương	Hợp lệ / Không hợp lệ	Quy tắc phân hoạch (Guideline used)	Nguồn (Source tag)
EC01	`Authorization`	Header đúng định dạng `"Bearer <token>"`, JWT hợp lệ ký bởi `SECRET_KEY`	**Valid**	Must be: valid format & signature	`[SPEC]` FR-12 `[CODE]` server.js:101-108
EC02	`Authorization`	Thiếu hoàn toàn header `Authorization` trong HTTP request	Invalid	Missing: header omitted	`[CODE]` server.js:103
EC03	`Authorization`	Header rỗng `""` hoặc chỉ có tiền tố `"Bearer "` không kèm token	Invalid	Missing/Empty: empty token	`[CODE]` server.js:103
EC04	`Authorization`	Sai tiền tố xác thực (ví dụ `"Basic <token>"`, thiếu khoảng trắng)	Invalid	Format: wrong scheme prefix	`[CODE]` server.js:102-106
EC05	`Authorization`	Token JWT có chữ ký bị thay đổi (tampered) hoặc sai bí mật HMAC	Invalid	Must be: valid signature	`[CODE]` server.js:106
EC06	`Authorization`	Chuỗi token hỏng cú pháp (malformed JWT không đủ 3 phần)	Invalid	Format: malformed JWT	`[CODE]` server.js:106
EC07	`token_role`	Payload Token JWT chứa claim `role === 'admin'`	**Valid**	Set of values: admin role	`[SPEC]` README:176 `[CODE]` server.js:51
EC08	`token_role`	Payload Token JWT chứa claim `role === 'user'` (người dùng thường)	Invalid	Set of values: unauthorized user	`[SPEC]` README:176 `[CODE]` server.js:100
EC09	`token_role`	Payload chứa role ngoài danh mục (`guest`, `moderator`, `null`)	Invalid	Outside set: unrecognized role	`[SPEC]` FR-12 `[ASSUMED]`

Mã EC	Biến / Ngữ cảnh	Mô tả lớp tương đương	Hợp lệ / Không hợp lệ	Quy tắc phân hoạch (Guideline used)	Nguồn (Source tag)
EC10	`target_endpoint`	Nhóm 7 API Quản trị hệ thống (`/api/admin/*`) — Có auth, thiếu role check	**Valid Target**	Split class: Admin APIs	`[SPEC]` §6.1-§6.4 `[CODE]` server.js:494, 510
EC11	`target_endpoint`	Nhóm 3 API chỉnh sửa danh mục (`POST/PUT/DELETE /api/categories`) — Có auth, thiếu role check	**Valid Target**	Split class: Category write APIs	`[SPEC]` §3.4, FR-12 `[CODE]` server.js:249-269
EC12	`target_endpoint`	Nhóm 3 API chỉnh sửa sản phẩm (`POST/PUT/DELETE /api/products`) — **Hoàn toàn thiếu auth**	**Valid Target** *(Spec)* ⚠ *Skewed (Code)*	Split class: Unprotected product write	`[SPEC]` §3.3, FR-12 `[CODE]` server.js:167-191
EC13	`target_endpoint`	API tra cứu mã giảm giá quản trị (`GET /api/coupons`) — Có auth, thiếu role check	**Valid Target**	Split class: Coupons read API	`[SPEC]` §5.2, FR-12 `[CODE]` server.js:356
EC14	`token_exp` (State)	Token JWT còn trong thời hạn sử dụng (`exp > now` hoặc không claim exp)	**Valid State**	Range of timestamp: active lifetime	`[SPEC]` / `[CODE]` server.js:51
EC15	`token_exp` (State)	Token JWT đã quá hạn sử dụng (`exp < now`)	Invalid State	Range of timestamp: expired lifetime	`[CODE]` server.js:106
EC16	`user_status` (State)	Tài khoản người dùng đang hoạt động và tồn tại trong DB	**Valid State**	State: active user in DB	`[CODE]` database.js:55
EC17	`user_status` (State)	Tài khoản đã bị xóa khỏi DB (`DELETE /admin/users/:id`), nhưng client còn giữ token	Invalid State	State conflict: deleted account	`[CODE]` server.js:504
EC18	`privilege_state`	Tài khoản người dùng giữ nguyên vai trò, không can thiệp nâng quyền	**Valid State**	State: normal privilege	`[SPEC]` FR-04, SEC-06
EC19	`privilege_state`	Người dùng thường tự sửa `role: "admin"` qua lỗ hổng Mass Assignment tại `PUT /api/users/me`	Invalid State	Security Flaw: privilege escalation	`[CODE]` server.js:124-127
EC20	`Response.status`	Truy cập thành công: HTTP 200 OK (hoặc 201 Created)	**Valid Output**	Output: authorized access granted	`[SPEC]` / `[CODE]`
EC21	`Response.status`	Từ chối xác thực danh tính: HTTP 401 Unauthorized	Invalid Output	Output: authentication required	`[CODE]` server.js:103
EC22	`Response.status`	Từ chối phân quyền / Token không hợp lệ: HTTP 403 Forbidden	Invalid Output	Output: forbidden / invalid token	`[CODE]` server.js:106 `[SPEC]` README:176
EC23	`Response.status`	Lỗ hổng kiểm soát truy cập (Defect Output): HTTP 200 OK trả về cho User thường	**Defect Output**	Output: security bypass defect	`[CODE]` server.js:100, 167

Bước 3: Chọn giá trị đại diện cho từng lớp tương đương (Best Representatives)

Áp dụng nguyên tắc lựa chọn giá trị đại diện theo bài giảng môn học **S04 - Domain Testing (Step 3)**:

- **Miền có thứ tự (Ordered/Ranged):** Giá trị đại diện là giá trị biên hoặc cận biên (`-> BVA` khai thác sâu).
- **Miền không có thứ tự (Unordered):** Chọn giá trị điển hình, thực tế trong môi trường chạy của SUT.

Mã EC	Biến kiểm thử	Mô tả lớp tương đương	Giá trị đại diện được chọn	Lý do lựa chọn đại diện
EC01	`Authorization`	Header hợp lệ, JWT đúng chữ ký `SECRET_KEY`	`"Bearer <admin_jwt>"` (từ `admin@eshop.com`)	Token JWT hợp pháp của tài khoản quản trị hệ thống
EC02	`Authorization`	Vắng mặt hoàn toàn header `Authorization`	*(Omitted - không gửi header)*	Kiểm tra nhánh `authHeader === undefined` tại `server.js:103`
EC03	`Authorization`	Header rỗng hoặc chỉ có `"Bearer "` không token	`"Bearer "` *(chuỗi có tiền tố nhưng token rỗng)*	Kiểm tra nhánh `token == null` tại `server.js:103`
EC04	`Authorization`	Header sai tiền tố xác thực (e.g. Basic)	`"Basic dXNlcjpwYXNz"`	Kiểm tra logic tách chuỗi `authHeader.split(" ")[1]` khi dùng scheme sai
EC05	`Authorization`	Token JWT có chữ ký bị thay đổi (Tampered)	`"Bearer <admin_jwt_tampered_sig>"`	Payload hợp lệ nhưng chữ ký không khớp -> `jwt.verify` ném lỗi
EC06	`Authorization`	Token JWT bị sai cấu trúc (Malformed)	`"Bearer not_a_valid_jwt_string"`	Chuỗi không theo định dạng 3 phần -> JsonWebTokenError
EC07	`token_role`	Payload Token chứa `role === 'admin'`	Token chứa `{ id: 1, role: "admin" }`	Vai trò quản trị hợp chuẩn theo đặc tả FR-12
EC08	`token_role`	Payload Token chứa `role === 'user'`	Token sinh từ `user@domain.com` (`{ id: 2, role: "user" }`)	Người dùng thường; SPEC đòi hỏi 403, CODE hiện trả 200
EC09	`token_role`	Payload Token chứa role ngoài tập / không xác định	Token craft `{ id: 99, role: "guest" }` ký bởi `SECRET_KEY`	Kiểm tra hành vi hệ thống khi nhận role không hợp lệ
EC10	`target_endpoint`	Nhóm 7 API Admin (`/api/admin/*`)	`GET /api/admin/users`	Endpoint quản trị nhạy cảm tra cứu danh sách toàn bộ người dùng
EC11	`target_endpoint`	Nhóm 3 API Category write	`POST /api/categories` body `{"name": "Category_{{uniq}}"}`	API ghi danh mục có middleware auth nhưng thiếu role check
EC12	`target_endpoint`	Nhóm 3 API Product write	`POST /api/products` body `{"name": "Product_{{uniq}}", "price": 99000, ...}`	API ghi sản phẩm hoàn toàn thiếu `authenticateToken` (Lỗ hổng)
EC13	`target_endpoint`	API tra cứu mã giảm giá quản trị	`GET /api/coupons`	API tra cứu coupon quản trị có auth nhưng thiếu role check
EC14	`token_exp` (State)	Token còn trong thời hạn sử dụng	Token hiện tại của SUT hoặc token có `exp > now`	`-> BVA` cho các biên thời gian; đại diện cho token còn hiệu lực
EC15	`token_exp` (State)	Token đã quá hạn sử dụng	Token craft với `exp = now - 3600` ký bằng `SECRET_KEY`	`-> BVA` biên hết hạn; kích hoạt `TokenExpiredError` -> 403
EC16	`user_status` (State)	Tài khoản người dùng đang hoạt động trong DB	User ID 1 (`admin@eshop.com`) hoặc User ID 2	Tài khoản tồn tại bình thường trong cơ sở dữ liệu SQLite
EC17	`user_status` (State)	Tài khoản đã bị xóa khỏi DB nhưng token còn giữ	Tạo user tạm thời, cấp token, xóa user qua `DELETE /admin/users/:id`	Kiểm tra lỗ hổng thiếu kiểm tra trạng thái DB của Stateless JWT

Mã EC	Biến kiểm thử	Mô tả lớp tương đương	Giá trị đại diện được chọn	Lý do lựa chọn đại diện
EC18	`privilege_escalation`	Tài khoản giữ nguyên quyền hạn ban đầu	Giữ nguyên role `'user'` khi gọi API	Trạng thái bình thường, không can thiệp nâng quyền
EC19	`privilege_escalation`	Người dùng tự đổi role qua `PUT /api/users/me`	Gửi `PUT /api/users/me` với payload `{"role": "admin"}`	Lỗ hổng Mass Assignment biến user thường thành admin
EC20	HTTP Response (Output)	Truy cập thành công: HTTP 200/201	HTTP 200 OK kèm JSON danh sách tài nguyên	Kỳ vọng cho trường hợp hợp lệ của quản trị viên
EC21	HTTP Response (Output)	Từ chối xác thực: HTTP 401 Unauthorized	HTTP 401 `{"error": "Unauthorized"}`	Kỳ vọng khi không gửi token hoặc token rỗng
EC22	HTTP Response (Output)	Từ chối phân quyền: HTTP 403 Forbidden	HTTP 403 `{"error": "Forbidden"}`	Kỳ vọng theo SPEC khi role không phải admin hoặc sai chữ ký
EC23	HTTP Response (Output)	Lỗ hổng kiểm soát truy cập (Defect Output)	HTTP 200 OK (thực tế code trả về thay vì 403)	Hành vi dị biệt của SUT khi thiếu role check hoặc thiếu middleware

Bước 4: Thiết kế bộ test case tối thiểu (Selecting Test Cases) & Ma trận bao phủ 100%

Áp dụng quy tắc kết hợp theo slide bài giảng S04:

1. **Lớp hợp lệ (Valid classes):** Kết hợp tối đa các lớp hợp lệ vào một test case duy nhất (`FR12-DT-01`).
2. **Lớp không hợp lệ (Invalid classes):** Mỗi test case chỉ kiểm thử duy nhất một lớp không hợp lệ để cô lập chính xác lỗi (`FR12-DT-02` đến `FR12-DT-11`).
3. **Kiểm tra chéo các nhóm Endpoint:** Các ca kiểm thử từ `FR12-DT-12` đến `FR12-DT-18` được thiết kế nhằm quét qua cả 4 nhóm endpoint đích (Admin APIs, Category write, Product write, Coupons read) đối với các vai trò Admin, User thường và Khách vãng lai (Guest).

Bảng danh sách 18 Test Case Domain Testing chi tiết

TC ID	Lớp bao phủ (Partitions)	Header Authorization	Token Role	Target Endpoint & Method	Tiền điều kiện	Kỳ vọng đầu ra (Expected)	Nguồn gốc
FR12-DT-01	EC01, EC07, EC10, EC14, EC16, EC18, EC20	`Bearer <admin_jwt>`	`admin`	`GET /api/admin/users`	Tài khoản admin đang hoạt động trong DB	HTTP 200 OK, body chứa danh sách users	`[SPEC]` FR-12 `[CODE]` server.js:494
FR12-DT-02	EC02, EC21	*(Omitted - không gửi)*	*(None)*	`GET /api/admin/users`	Không có	HTTP 401 Unauthorized `{"error": "Unauthorized"}`	`[CODE]` server.js:103
FR12-DT-03	EC03, EC21	`"Bearer "`	*(None)*	`GET /api/admin/users`	Không có	HTTP 401 Unauthorized `{"error": "Unauthorized"}`	`[CODE]` server.js:103
FR12-DT-04	EC04, EC21	`"Basic dXNlcjpwYXNz"`	*(None)*	`GET /api/admin/users`	Không có	HTTP 401/403 (Từ chối định dạng scheme sai)	`[CODE]` server.js:103
FR12-DT-05	EC05, EC22	`Bearer <tampered_token>`	`admin`	`GET /api/admin/users`	Không có	HTTP 403 Forbidden `{"error": "Forbidden"}`	`[CODE]` server.js:106

TC ID	Lớp bao phủ (Partitions)	Header Authorization	Token Role	Target Endpoint & Method	Tiền điều kiện	Kỳ vọng đầu ra (Expected)	Nguồn gốc
FR12-DT-06	EC06, EC22	`Bearer not_a_valid_jwt`	*(None)*	`GET /api/admin/users`	Không có	HTTP 403 Forbidden `{"error": "Forbidden"}`	`[CODE]` server.js:106
FR12-DT-07	EC08, EC22, EC23	`Bearer <user_jwt>`	`user`	`GET /api/admin/users`	Tài khoản user thường đăng nhập hợp lệ	`[SPEC]` HTTP 403 Forbidden *(Thực tế SUT: HTTP 200 -> Bug SEC-03)*	`[SPEC]` README:176 `[CODE]` server.js:100
FR12-DT-08	EC09, EC22	`Bearer <guest_jwt>`	`guest`	`GET /api/admin/users`	Token craft ký bằng SECRET_KEY	`[SPEC]` HTTP 403 Forbidden	`[SPEC]` FR-12
FR12-DT-09	EC15, EC22	`Bearer <expired_jwt>`	`admin`	`GET /api/admin/users`	Token craft có `exp < now`	HTTP 403 Forbidden `{"error": "Forbidden"}`	`[CODE]` server.js:106
FR12-DT-10	EC17, EC22, EC23	`Bearer <token_deleted_user>`	`user`	`GET /api/admin/users`	User được tạo, cấp token, sau đó bị xóa khỏi DB	`[SPEC]` HTTP 401/403 (Token bị thu hồi) *(Thực tế SUT: HTTP 200 do Stateless JWT)*	`[ASSUMED]` Security Standard
FR12-DT-11	EC19, EC22, EC23	`Bearer <user_jwt>`	`user`	`PUT /api/users/me` body: `{"role": "admin"}`	User thường đăng nhập hợp lệ	`[SPEC]` HTTP 400/403 (Cấm tự đổi role) *(Thực tế SUT: HTTP 200 đổi role thành công)*	`[SPEC]` FR-04, SEC-06 `[CODE]` server.js:124
FR12-DT-12	EC11, EC20	`Bearer <admin_jwt>`	`admin`	`POST /api/categories` body: `{"name": "Cat_{{uniq}}"}`	Tài khoản admin hợp lệ	HTTP 200 OK, message: "Category added successfully"	`[SPEC]` §3.4 `[CODE]` server.js:249
FR12-DT-13	EC11, EC08, EC22, EC23	`Bearer <user_jwt>`	`user`	`POST /api/categories` body: `{"name": "Hacked_{{uniq}}"}`	Tài khoản user thường hợp lệ	`[SPEC]` HTTP 403 Forbidden *(Thực tế SUT: HTTP 200 do thiếu role check)*	`[SPEC]` FR-12
FR12-DT-14	EC12, EC20	`Bearer <admin_jwt>`	`admin`	`POST /api/products` body: `{"name": "P_{{uniq}}", ...}`	Tài khoản admin hợp lệ	HTTP 200 OK, message: "Product added successfully"	`[SPEC]` §3.3 `[CODE]` server.js:167
FR12-DT-15	EC12, EC08, EC22, EC23	`Bearer <user_jwt>`	`user`	`POST /api/products` body: `{"name": "P_{{uniq}}", ...}`	Tài khoản user thường hợp lệ	`[SPEC]` HTTP 403 Forbidden *(Thực tế SUT: HTTP 200 do thiếu role check)*	`[SPEC]` FR-12
FR12-DT-16	EC12, EC02, EC21, EC23	*(Omitted - không gửi)*	*(None)*	`POST /api/products` body: `{"name": "P_{{uniq}}", ...}`	Khách vãng lai (Guest) không token	`[SPEC]` HTTP 401 Unauthorized *(Thực tế SUT: HTTP 200 - Lỗ hổng Critical)*	`[SPEC]` FR-12 `[CODE]` server.js:167

TC ID	Lớp bao phủ (Partitions)	Header Authorization	Token Role	Target Endpoint & Method	Tiền điều kiện	Kỳ vọng đầu ra (Expected)	Nguồn gốc
FR12-DT-17	EC13, EC20	`Bearer <admin_jwt>`	`admin`	`GET /api/coupons`	Tài khoản admin hợp lệ	HTTP 200 OK, body chứa coupons	`[SPEC]` §5.2 `[CODE]` server.js:356
FR12-DT-18	EC13, EC08, EC22, EC23	`Bearer <user_jwt>`	`user`	`GET /api/coupons`	Tài khoản user thường hợp lệ	`[SPEC]` HTTP 403 Forbidden *(Thực tế SUT: HTTP 200 do thiếu role check)*	`[SPEC]` FR-12

Ma trận bao phủ lớp tương đương (Coverage Matrix)

Lớp tương đương (EC)	Loại lớp	Ca kiểm thử bao phủ (Covering Test Cases)	Tình trạng bao phủ
EC01 (`Authorization` hợp lệ)	**Valid**	`FR12-DT-01`, `FR12-DT-07`, `FR12-DT-12`, `FR12-DT-14`, `FR12-DT-17`	☑ Bao phủ đầy đủ
EC02 (`Authorization` vắng mặt)	Invalid	`FR12-DT-02`, `FR12-DT-16`	☑ Bao phủ đơn lập
EC03 (`Authorization` rỗng / `"Bearer "`)	Invalid	`FR12-DT-03`	☑ Bao phủ đơn lập
EC04 (`Authorization` sai scheme prefix)	Invalid	`FR12-DT-04`	☑ Bao phủ đơn lập
EC05 (`Authorization` sai chữ ký / tampered)	Invalid	`FR12-DT-05`	☑ Bao phủ đơn lập
EC06 (`Authorization` sai cấu trúc / malformed)	Invalid	`FR12-DT-06`	☑ Bao phủ đơn lập
EC07 (`role === 'admin'`)	**Valid**	`FR12-DT-01`, `FR12-DT-12`, `FR12-DT-14`, `FR12-DT-17`	☑ Bao phủ đầy đủ
EC08 (`role === 'user'`)	Invalid	`FR12-DT-07`, `FR12-DT-13`, `FR12-DT-15`, `FR12-DT-18`	☑ Bao phủ đơn lập & đa endpoint
EC09 (`role` ngoài tập / guest)	Invalid	`FR12-DT-08`	☑ Bao phủ đơn lập
EC10 (Target: Admin APIs `/api/admin/*`)	**Valid Target**	`FR12-DT-01` đến `FR12-DT-11`	☑ Bao phủ đầy đủ
EC11 (Target: Category write APIs)	**Valid Target**	`FR12-DT-12`, `FR12-DT-13`	☑ Bao phủ đầy đủ
EC12 (Target: Product write APIs - No Auth)	**Valid Target** *(Spec)*	`FR12-DT-14`, `FR12-DT-15`, `FR12-DT-16`	☑ Bao phủ đầy đủ
EC13 (Target: Coupons read API)	**Valid Target**	`FR12-DT-17`, `FR12-DT-18`	☑ Bao phủ đầy đủ
EC14 (`token_exp` còn hạn)	**Valid State**	`FR12-DT-01`, `FR12-DT-07` đến `FR12-DT-18`	☑ Bao phủ đầy đủ
EC15 (`token_exp` hết hạn)	Invalid State	`FR12-DT-09`	☑ Bao phủ đơn lập
EC16 (`user_status` đang hoạt động)	**Valid State**	`FR12-DT-01` đến `FR12-DT-09`, `FR12-DT-11` đến `FR12-DT-18`	☑ Bao phủ đầy đủ
EC17 (`user_status` đã bị xóa)	Invalid State	`FR12-DT-10`	☑ Bao phủ đơn lập
EC18 (`privilege_escalation` giữ nguyên)	**Valid State**	`FR12-DT-01` đến `FR12-DT-10`, `FR12-DT-12` đến `FR12-DT-18`	☑ Bao phủ đầy đủ

Lớp tương đương (EC)	Loại lớp	Ca kiểm thử bao phủ (Covering Test Cases)	Tình trạng bao phủ
EC19 (`privilege_escalation` Mass Assignment)	Invalid State	`FR12-DT-11`	☑ Bao phủ đơn lập
EC20 (Output HTTP 200/201 Thành công)	**Valid Output**	`FR12-DT-01`, `FR12-DT-12`, `FR12-DT-14`, `FR12-DT-17`	☑ Bao phủ trực tiếp
EC21 (Output HTTP 401 Từ chối xác thực)	Invalid Output	`FR12-DT-02`, `FR12-DT-03`, `FR12-DT-04`, `FR12-DT-16`	☑ Bao phủ trực tiếp
EC22 (Output HTTP 403 Từ chối phân quyền)	Invalid Output	`FR12-DT-05` đến `FR12-DT-11`, `FR12-DT-13`, `FR12-DT-15`, `FR12-DT-18`	☑ Bao phủ trực tiếp
EC23 (Output HTTP 200 Bypass Phân quyền)	**Defect Output**	`FR12-DT-07`, `FR12-DT-10`, `FR12-DT-11`, `FR12-DT-13`, `FR12-DT-15`, `FR12-DT-16`, `FR12-DT-18`	☑ Ghi nhận lỗ hổng SUT

Độ bao phủ: 23/23 lớp EC (100%).

3.5.3 Phân tích giá trị biên (Boundary Value Analysis - BVA)

Bước 1: Xác định các miền có thứ tự

- **Biến có thứ tự áp dụng BVA:**
  1. `token.exp`: Thời hạn JWT token dạng Unix timestamp (giây). So sánh tương đối với `now = Math.floor(Date.now() / 1000)`.
  2. `param.id`: Tham số số nguyên định danh tài nguyên trên các endpoint xóa (`DELETE /api/admin/coupons/:id`).
  3. `product.price`: Thuộc tính số tiền trên endpoint tạo sản phẩm `POST /api/products`.
- **Biến không áp dụng BVA:**
  - `role`, `Authorization header prefix`, `JWT signature`: Miền rời rạc (Discrete Set/Enum) hoặc chuỗi mã hóa mật mã học, không có quan hệ thứ tự liên tục.

Bước 2: Xác định điểm biên (3-value BVA + Mô hình 9 điểm + Biên mã hóa UTF-8)

- **`token.exp`:** Biên so sánh now. Các điểm kiểm thử: $exp = now - 1$ (vừa hết hạn 1s), $exp = now$ (hết hạn tại điểm biên), $exp = now + 1$ (còn hạn 1s), $exp = now + 3600$ (danh định), $exp = now + 10\text{ năm}$ (cực trị tương lai).
- **`param.id`:** Biên số nguyên dương $[1, 2147483647]$. Các điểm kiểm thử: $LB-1 = 0$, $LB = 1$, $LB+1 = 2$, $UB = 2147483647$ (Max 32-bit INT), Extreme âm = -1. *(Áp dụng quy tắc v2.2 Test Fixture Immutability: sử dụng endpoint coupon để không phá hủy tài khoản Admin/User mồi cốt lõi)*.
- **`product.price`:** Miền giá sản phẩm hợp lệ $[1, +\infty)$. Các điểm kiểm thử: Extreme âm = -1, $LB-1 = 0$, $LB = 1$, Cực trị dương = $999,999,999$.

Bước 3: Danh sách 14 ca kiểm thử BVA

Mã Test Case	Biến kiểm thử	Giá trị biên cụ thể	Ý nghĩa kỹ thuật	Lỗi code sẽ bắt được	Kết quả mong đợi
`FR12-BVA-01`	`token.exp`	`exp = now - 1`	$LB-1$ thời gian	Lỗi dùng toán tử so sánh `>` thay vì `>=`	HTTP 403
`FR12-BVA-02`	`token.exp`	`exp = now`	LB thời gian	Chấp nhận token hết hạn tại đúng giây 0	HTTP 403
`FR12-BVA-03`	`token.exp`	`exp = now + 1`	$LB+1$ thời gian	Từ chối nhầm token còn hạn 1 giây	HTTP 200
`FR12-BVA-04`	`token.exp`	`exp = now + 3600`	Nominal	Token danh định 1 giờ	HTTP 200
`FR12-BVA-05`	`token.exp`	`exp = now + 10 năm`	Max Extreme	Lỗi tràn số nguyên timestamp 32-bit (Y2038)	HTTP 200
`FR12-BVA-06`	`param.id`	`id = 0`	$LB-1$ số nguyên	Chấp nhận ID phi lý bằng 0	HTTP 404

Mã Test Case	Biến kiểm thử	Giá trị biên cụ thể	Ý nghĩa kỹ thuật	Lỗi code sẽ bắt được	Kết quả mong đợi
FR12-BVA-07	param.id	id = 1	LB biên hợp lệ	Lỗi từ chối ID nhỏ nhất	HTTP 200 / 404
FR12-BVA-08	param.id	id = 2	$LB+1$	Lỗi lệch 1 đơn vị cận dưới	HTTP 200 / 404
FR12-BVA-09	param.id	id = 2147483647	UB Max INT	Tràn bộ đệm kiểu dữ liệu integer	HTTP 404
FR12-BVA-10	param.id	id = -1	Min Extreme	Không kiểm tra số nguyên âm	HTTP 404
FR12-BVA-11	product.price	price = -1 (Guest)	Min Extreme	Khách tạo sản phẩm với giá âm	HTTP 401
FR12-BVA-12	product.price	price = 0 (Guest)	$LB-1$	Khách tạo sản phẩm với giá 0 đồng	HTTP 401
FR12-BVA-13	product.price	price = 1 (Guest)	LB	Khách tạo sản phẩm giá 1 đồng	HTTP 401
FR12-BVA-14	product.price	price = 999999999 (Guest)	Max Extreme	Khách tạo sản phẩm giá cực hạn	HTTP 401

3.5.4 Kết quả thực thi kiểm thử chi tiết

Toàn bộ 32 test cases (18 DT + 14 BVA) đã được thực thi tự động qua test runner `run_cases.mjs` đối chiếu trực tiếp với SUT Backend:

- **Tổng số thiết kế (Designed):** 32
- **Đã thực thi (Executed):** 32 (100%)
- **Thành công (PASS):** 21
- **Thất bại (FAIL):** 11
- **Bị chặn (BLOCKED):** 0

Tóm tắt kết quả thực thi 11 ca FAIL:

- **5 ca FAIL do Middleware thiếu kiểm tra vai trò Admin (Broken Access Control):** `FR12-DT-07`, `FR12-DT-08`, `FR12-DT-13`, `FR12-DT-15`, `FR12-DT-18`. Người dùng thường gửi request đến các API quản trị nhận HTTP 200/201 thay vì 403 Forbidden.
- **5 ca FAIL do Route sản phẩm bỏ quên xác thực (Missing Authentication):** `FR12-DT-16`, `FR12-BVA-11`, `FR12-BVA-12`, `FR12-BVA-13`, `FR12-BVA-14`. Khách vãng lai gửi request tạo sản phẩm nhận HTTP 201 Created thay vì bị chặn bằng 401 Unauthorized.
- **1 ca FAIL do Lỗ hổng Mass Assignment leo quyền:** `FR12-DT-11`. Người dùng gửi `PUT /api/users/me` với payload `role: 'admin'`, server trả về HTTP 200 OK và cập nhật quyền admin vào DB.

Bảng kết quả thực thi chi tiết 32 test cases

Mã TC	Kỹ thuật	Mô tả ca kiểm thử	Kỳ vọng (Expected)	Thực tế (Actual)	Trạng thái
FR12-DT-01	DT	Admin token hợp lệ truy cập `GET /api/admin/users`	HTTP 200, mảng users	HTTP 200, mảng users	**PASS**
FR12-DT-02	DT	Thiếu Authorization header khi gọi `GET /api/admin/users`	HTTP 401, Access token required	HTTP 401, Access token required	**PASS**
FR12-DT-03	DT	Authorization header rỗng khi gọi `GET /api/admin/users`	HTTP 401, Access token required	HTTP 401, Access token required	**PASS**
FR12-DT-04	DT	Token sai tiền tố (thiếu 'Bearer ')	HTTP 403, Invalid or expired	HTTP 403, Invalid or expired	**PASS**
FR12-DT-05	DT	Token sai chữ ký HMAC Secret (Signature Mismatch)	HTTP 403, Invalid or expired	HTTP 403, Invalid or expired	**PASS**

Mã TC	Kỹ thuật	Mô tả ca kiểm thử	Kỳ vọng (Expected)	Thực tế (Actual)	Trạng thái
FR12-DT-06	DT	Chuỗi JWT hỏng cú pháp (Malformed Token)	HTTP 403, Invalid or expired	HTTP 403, Invalid or expired	**PASS**
FR12-DT-07	DT	User thường truy cập `GET /api/admin/users`	HTTP 403, Forbidden	HTTP 200, mảng users	**FAIL**
FR12-DT-08	DT	User thường gọi `DELETE /api/admin/users/999`	HTTP 403, Forbidden	HTTP 200, User deleted	**FAIL**
FR12-DT-09	DT	Token chứa role không xác định (`guest`)	HTTP 403, Forbidden	HTTP 200, mảng users	**FAIL**
FR12-DT-10	DT	User thường truy cập hồ sơ chính mình `GET /api/users/me`	HTTP 200, user profile	HTTP 200, user profile	**PASS**
FR12-DT-11	DT	User tự nâng quyền thành admin qua `PUT /api/users/me`	HTTP 400 hoặc role ignored	HTTP 200, role: 'admin'	**FAIL**
FR12-DT-12	DT	Admin tạo danh mục mới qua `POST /api/categories`	HTTP 200/201, Category created	HTTP 200, Category created	**PASS**
FR12-DT-13	DT	User thường tạo danh mục qua `POST /api/categories`	HTTP 403, Forbidden	HTTP 200, Category created	**FAIL**
FR12-DT-14	DT	Admin xem toàn bộ đơn hàng `GET /api/admin/orders`	HTTP 200, orders array	HTTP 200, orders array	**PASS**
FR12-DT-15	DT	User thường xem danh sách đơn hàng `GET /api/admin/orders`	HTTP 403, Forbidden	HTTP 200, orders array	**FAIL**
FR12-DT-16	DT	Khách vãng lai (no auth) tạo sản phẩm `POST /api/products`	HTTP 401, Access token required	HTTP 201, Product created	**FAIL**
FR12-DT-17	DT	Admin tạo mã giảm giá `POST /api/coupons`	HTTP 201, Coupon created	HTTP 201, Coupon created	**PASS**
FR12-DT-18	DT	User thường tạo mã giảm giá `POST /api/coupons`	HTTP 403, Forbidden	HTTP 201, Coupon created	**FAIL**
FR12-BVA-01	BVA	JWT token đã hết hạn 1 giây (`exp = now - 1`)	HTTP 403, Invalid or expired	HTTP 403, Invalid or expired	**PASS**
FR12-BVA-02	BVA	JWT token vừa hết hạn đúng thời điểm (`exp = now`)	HTTP 403, Invalid or expired	HTTP 403, Invalid or expired	**PASS**
FR12-BVA-03	BVA	JWT token còn hạn 1 giây (`exp = now + 1`)	HTTP 200, mảng users	HTTP 200, mảng users	**PASS**
FR12-BVA-04	BVA	JWT token còn hạn 3600 giây (`exp = now + 3600`)	HTTP 200, mảng users	HTTP 200, mảng users	**PASS**
FR12-BVA-05	BVA	JWT token cực hạn tương lai (`exp = now + 10 năm`)	HTTP 200, mảng users	HTTP 200, mảng users	**PASS**
FR12-BVA-06	BVA	Admin xóa coupon với ID = 0 (LB-1 phi lý)	HTTP 404, Coupon not found	HTTP 404, Coupon not found	**PASS**
FR12-BVA-07	BVA	Admin xóa coupon với ID = 1 (LB biên hợp lệ)	HTTP 200 / 404	HTTP 200 / 404	**PASS**
FR12-BVA-08	BVA	Admin xóa coupon với ID = 2 (LB+1)	HTTP 200 / 404	HTTP 200 / 404	**PASS**
FR12-BVA-09	BVA	Admin xóa coupon với ID = 2147483647 (Max 32-bit INT)	HTTP 404, Coupon not found	HTTP 404, Coupon not found	**PASS**
FR12-BVA-10	BVA	Admin xóa coupon với ID = -1 (Extreme âm)	HTTP 404, Coupon not found	HTTP 404, Coupon not found	**PASS**

Mã TC	Kỹ thuật	Mô tả ca kiểm thử	Kỳ vọng (Expected)	Thực tế (Actual)	Trạng thái
`FR12-BVA-11`	BVA	Khách tạo sản phẩm với `price = -1` (Extreme âm)	HTTP 401 hoặc 400	HTTP 201, Product created	**FAIL**
`FR12-BVA-12`	BVA	Khách tạo sản phẩm với `price = 0` (LB-1 không hợp lệ)	HTTP 401 hoặc 400	HTTP 201, Product created	**FAIL**
`FR12-BVA-13`	BVA	Khách tạo sản phẩm với `price = 1` (LB biên hợp lệ)	HTTP 401 Unauthorized	HTTP 201, Product created	**FAIL**
`FR12-BVA-14`	BVA	Khách tạo sản phẩm với `price = 999999999` (Extreme)	HTTP 401 Unauthorized	HTTP 201, Product created	**FAIL**

3.5.5 Báo cáo lỗi phần mềm (Defect Triage)

11 ca kiểm thử thất bại được phân loại nguyên nhân gốc rễ thành **3 khiếm khuyết phần mềm**:

Mã lỗi (Bug ID)	Tiêu đề lỗi	Test Cases phát hiện	Mức độ (Severity)	Độ ưu tiên
BUG-FR12-01	Backend middleware `authenticateToken` không kiểm tra vai trò admin và các route quản trị thiếu role check	5 TCs (`FR12-DT-07, 08, 13, 15, 18`)	**Critical**	Immediate
BUG-FR12-02	Endpoint quản lý sản phẩm `POST /api/products`, `PUT/DELETE` bỏ quên middleware `authenticateToken`	5 TCs (`FR12-DT-16, FR12-BVA-11..14`)	**Critical**	Immediate
BUG-FR12-03	Lỗ hổng gán tham số hàng loạt (Mass Assignment) tại `PUT /api/users/me` cho phép user tự nâng quyền thành Admin	`FR12-DT-11`	**High**	High

3.5.6 Phân tích khoảng trống của AI (AI Gap Analysis & Nâng cấp v2.2)

Quá trình kiểm thử FR-12 đã làm bộc lộ 4 khoảng trống AI quan trọng, dẫn đến việc tiếp tục nâng cấp bộ kỹ năng lên **phiên bản v2.2**:

Mục bị bỏ sót / Thách thức thực tế	Cách phát hiện	Vì sao AI bỏ sót ban đầu	Phân loại nguyên nhân (Category)	Hành động cải tiến Skill (v2.1 → v2.2)
1. Nhầm lẫn giữa Xác thực (Authentication) và Phân quyền (Authorization)	Thấy `authenticateToken` ở các route admin và nghĩ rằng đã được bảo vệ đầy đủ; khi chạy test thì user thường vẫn vào được 200 OK	AI bị thiên kiến ngữ nghĩa (Semantic Bias): nhìn thấy hàm mang tên "authenticate" kèm JWT thì giả định nó kiểm tra cả quyền hạn (admin vs user)	`Tool limitation` & `Inherent feature complexity`	Nâng cấp `domain-testing` và `test-execution-and-bugs` với quy tắc **Authorization vs Authentication Separation Rule**: luôn kiểm tra sự hiện diện của lệnh so khớp vai trò (`req.user.role === 'admin'`) độc lập với việc verify JWT signature.
2. Nguy cơ phá hủy dữ liệu mồi cốt lõi (Destructive Test Fixture Mutation)	Khi viết BVA cho `DELETE /api/admin/users/:id`, nếu xóa ID=1 hoặc ID=2 thì mất tài khoản Admin/User, làm gãy toàn bộ bộ test cho các lần chạy sau	AI có xu hướng áp dụng các giá trị biên danh định (1, 2) một cách cơ học mà không đánh giá tác động phụ (side effect) lên dữ liệu mồi cố định trong cơ sở dữ liệu	`Inherent feature complexity`	Bổ sung quy tắc **Test Fixture Immutability & Re-runnability Rule**: tuyệt đối không dùng ID của tài khoản test cố định cho các ca kiểm thử xóa phá hủy; chuyển mục tiêu sang các thực thể tạm thời độc lập (như coupons).

Mục bị bỏ sót / Thách thức thực tế	Cách phát hiện	Vì sao AI bỏ sót ban đầu	Phân loại nguyên nhân (Category)	Hành động cải tiến Skill (v2.1 → v2.2)
3. Bỏ sót lỗ hổng leo quyền Mass Assignment ở endpoint thông thường	Soát mã nguồn `PUT /api/users/me` thấy bóc tách biến `role` từ `req.body` đưa vào câu lệnh SQL UPDATE	Khi được giao kiểm thử FR-12 (Kiểm soát truy cập), AI có xu hướng chỉ nhìn vào các endpoint có tiền tố `/api/admin/*`, bỏ qua các endpoint thông thường của người dùng	`Prompt/input quality` & `Inherent feature complexity`	Bổ sung **Cross-Feature Privilege Escalation Audit Checklist** vào Stage 0: rà soát tất cả các endpoint cho phép cập nhật dữ liệu người dùng xem có nguy cơ over-posting/mass assignment ảnh hưởng tới trường `role` hay không.
4. Lệch pha chuỗi thông báo phản hồi của SUT (SUT Specific Response String Matching)	`POST /api/categories` trả về `{ "message": "Category created" }` nhưng spec ghi `"Category added successfully"`, khiến runner đánh giá BLOCKED giả tạo	AI ban đầu gán kỳ vọng theo ví dụ trong tài liệu đặc tả thay vì đối chiếu trực tiếp với chuỗi string thực tế trong mã nguồn backend	`Prompt/input quality`	Bổ sung quy tắc **SUT Exact String Alignment Rule**: đối chiếu chính xác chuỗi message trong route handler của mã nguồn trước khi sinh tệp JSON máy đọc, ưu tiên xác thực theo mã HTTP status và các trường cấu trúc cốt lõi thay vì chuỗi văn bản tự do.

3.5.7 Hạn chế của kỹ thuật

1. **Bản chất nhị phân / rời rạc của Kiểm soát truy cập:** Phân quyền hệ thống (Authorization) chủ yếu vận hành dựa trên các thuộc tính rời rạc (Role: `admin` vs `user` vs `unauthenticated`) và các quyết định nhị phân (Cho phép / Từ chối). Kỹ thuật BVA vốn thiết kế cho các miền biến thiên liên tục có thứ tự nên không phát huy được nhiều tác dụng trên các trường cốt lõi của Access Control, ngoại trừ các trường số phụ trợ (như timestamp `exp`, `price`, `id`).
2. **Lỗ hổng bắt nguồn từ tầng kiến trúc và Middleware:** Toàn bộ các lỗi nghiêm trọng nhất của FR-12 (`BUG-FR12-01`, `BUG-FR12-02`, `BUG-FR12-03`) xuất phát từ việc thiếu hụt middleware phân quyền hoặc thiết kế API lỏng lẻo (insecure design), không thể phát hiện nếu chỉ phân tích các điểm biên dữ liệu thông thường mà không có góc nhìn kiểm thử bảo mật chuyên sâu (Security Testing).
3. **Tính phụ thuộc ngữ cảnh người gọi (Caller Identity Context):** Cùng một request payload (ví dụ `GET /api/admin/users`) là hợp lệ nếu người gửi là Admin nhưng lại là hành vi xâm nhập bất hợp pháp nếu người gửi là User thường. Điều này đòi hỏi ma trận kiểm thử phải luôn gắn liền ngữ cảnh danh tính với từng ca kiểm thử.

3.6 FR-20 — Mobile Login & Logout (Pool D)

3.6.1 Tìm hiểu tính năng và đặc tả thực tế

- **Phạm vi tính năng:** Kiểm thử luồng Đăng nhập & Đăng xuất trên thiết bị di động (React Native) theo `docs/eshop-features.md` (FR-20), `eshop-sut/README.md` (§7 Phân hệ Mobile) và `docs/api_specification.md` (mục 1.2 Đăng nhập).
- **Các thành phần mục tiêu khảo sát:**
  - Giao diện và logic Mobile Client: `eshop-sut/frontend-mobile/App.js` (`handleLogin`, `logout`, `renderLogin`, `renderProfile`, quản lý state `token`, `user`, `cart`, `orders`).
  - Backend Authentication Endpoint: `eshop-sut/backend/server.js:32-66` (`POST /api/login`).
  - Trạng thái cơ sở dữ liệu: Bảng `users` trong `database.sqlite` (các cột `email`, `password`, `login_attempts`, `locked_until`).
- **Nguồn đặc tả đã sử dụng:**
  - `docs/eshop-features.md` (FR-20: Mobile Features) `[SPEC]`
  - Mã nguồn Mobile Client: `eshop-sut/frontend-mobile/App.js:145-207, 755-797` `[CODE]`
  - Mã nguồn Backend: `eshop-sut/backend/server.js:32-66` `[CODE]`
  - Kết quả thực thi trực tiếp qua Automated Test Runner: `[OBSERVED]`
- **Bảng đối chiếu mâu thuẫn đa nền tảng (Cross-platform Skew Checklist v2.0):**

Vấn đề / Tiêu chí	Đặc tả (`[SPEC]`)	Backend (`[CODE]`)	Frontend Mobile (`[CODE]`)	Tác động kiểm thử
Kiểm tra rỗng `email` / `password`	Bắt buộc nhập	Không kiểm tra (`server.js:33`)	Không kiểm tra trước khi gửi request (`App.js:189`)	Cho phép gửi request rỗng lên server
Nhãn trường Email	Email	`email`	Label hiển thị là `"Username"` (`App.js:763`)	Mâu thuẫn thuật ngữ nhãn UI

Vấn đề / Tiêu chí	Đặc tả ([SPEC])	Backend ([CODE])	Frontend Mobile ([CODE])	Tác động kiểm thử
Ngôn ngữ nút Submit	Tiếng Việt (FR-21)	N/A	Nút ghi tiếng Anh "Sign In" (App.js:783)	Vi phạm tiêu chuẩn giao diện FR-21
Vị trí thông báo lỗi	Nằm TRÊN nút Submit (FR-22)	N/A	Nằm DƯỚI nút Submit (App.js:790)	Vi phạm tiêu chuẩn Form FR-22
Xử lý lỗi cụ thể	Phân biệt rõ lỗi sai mật khẩu vs khóa tài khoản	Trả về 401 hoặc 403 có thông điệp riêng	catch nuốt hết lỗi, luôn hiện chuỗi chung "Đăng nhập thất bại..."	Người dùng không biết tài khoản bị khóa hay gõ sai
Dọn giỏ hàng khi Logout	Đăng xuất phải xóa session và dữ liệu nhạy cảm	N/A (Stateless JWT)	**Bỏ quên** setCart([]) (App.js:161)	Giỏ hàng của user cũ bị rò rỉ cho user/guest tiếp theo

- **Phát hiện quan trọng từ mã nguồn SUT (Code Investigation):**
  - **Lỗ hổng rò rỉ dữ liệu phiên giữa các người dùng khi Logout (App.js:161-169):** Hàm logout() trên Mobile chỉ reset token, user, orders, hoàn toàn **bỏ quên lệnh** setCart([]). Hệ quả: Giỏ hàng của người dùng vừa đăng xuất vẫn được lưu trữ nguyên vẹn trên Client React state, khiến người dùng tiếp theo hoặc khách vãng lai sau đó nhìn thấy và thừa hưởng toàn bộ sản phẩm trong giỏ của người tiền nhiệm (Session Bleed / Cross-user Cart Retention).
  - **Lỗ hổng lộ mật khẩu người dùng dạng plaintext (server.js:52):** Tại route POST /api/login, khi xác thực thành công, server trả về trực tiếp đối tượng user thu được từ câu lệnh SELECT * FROM users, chứa nguyên vẹn trường password dạng plain-text truyền qua mạng về client (CWE-200 / Sensitive Data Exposure).
  - **Cơ chế tính số lần đăng nhập sai cộng dồn bước nhảy 2 (server.js:54):** Mỗi lần nhập sai mật khẩu, biến login_attempts bị cộng dồn **+2** (const newAttempts = user.login_attempts + 2) thay vì +1. Do ngưỡng khóa tài khoản là >= 3, chỉ sau **2 lần nhập sai** (0 + 2 + 2 = 4 >= 3), tài khoản đã bị khóa ngay lập tức trong 180 giây (3 phút) thay vì cho phép thử 3 lần như thiết kế (Premature Account Lockout).
  - **Vi phạm tiêu chuẩn thiết kế UI/UX trên Mobile (App.js:755-797):** Form đăng nhập trên Mobile dùng nhãn tiếng Anh "Username" thay vì "Email", nút gửi dùng "Sign In" (vi phạm chuẩn tiếng Việt thống nhất FR-21), và hộp thông báo lỗi đặt bên dưới nút gửi (vi phạm chuẩn vị trí lỗi FR-22).

3.6.2 Kiểm thử miền (Domain Testing)

Bước 1: Xác định biến Input, Output, State và Miền giá trị (Domains)

Tên biến	Loại biến	Kiểu dữ liệu	Miền giá trị và Ràng buộc kỹ thuật (Domain Constraints)	Nguồn dữ kiện (Source Tag)
input_email	Input	String	Chuỗi định dạng email RFC 5322 hợp lệ; có thể là email đã đăng ký hoặc chưa đăng ký; chuỗi rỗng ""; khoảng trắng; sai định dạng; null; sai kiểu (number, boolean).	[SPEC] §1.2, [CODE] server.js:33
input_password	Input	String	Chuỗi mật khẩu người dùng; khớp với mật khẩu tài khoản trong CSDL; sai mật khẩu; chuỗi rỗng ""; khoảng trắng; null; sai kiểu dữ liệu.	[SPEC] §1.2, [CODE] server.js:33
state_account	State	Enum	Trạng thái của tài khoản trong CSDL: • NORMAL: Tồn tại, không bị khóa (login_attempts = 0, locked_until is NULL). • FAILED_ONCE: Đã gõ sai 1 lần (login_attempts = 2). • LOCKED: Đang bị khóa (locked_until > now). • NON_EXISTENT: Không tồn tại trong CSDL.	[CODE] server.js:37-63, database.js:48-61
state_session	State	Enum	Trạng thái phiên làm việc trên Mobile: • GUEST: Chưa đăng nhập (token = "", user = null). • AUTHENTICATED: Đã đăng nhập (token != "", user != null).	[CODE] App.js:33-36
state_cart	State	Array	Trạng thái giỏ hàng trước khi Đăng xuất: • EMPTY: Giỏ hàng rỗng (cart = []). • HAS_ITEMS: Giỏ hàng chứa từ 1 sản phẩm trở lên.	[CODE] App.js:34, 161

Tên biến	Loại biến	Kiểu dữ liệu	Miền giá trị và Ràng buộc kỹ thuật (Domain Constraints)	Nguồn dữ kiện (Source Tag)
output_status	Output	Integer	Mã trạng thái HTTP trả về: 200 (OK), 401 (Unauthorized), 403 (Forbidden), 500 (Internal Error).	[SPEC] §1.2, [CODE] server.js:36-63
output_token	Output	String/Null	Chuỗi JWT Token hợp lệ khi đăng nhập thành công; hoặc null khi đăng nhập thất bại.	[SPEC] §1.2, [CODE] server.js:51
output_user_payload	Output	Object/Null	Đối tượng user trả về; ghi nhận lỗ hổng lộ trường password.	[CODE] server.js:52
output_mobile_view	Output	String	Màn hình điều hướng trên Mobile: "home" (khi login/logout thành công), "login" (khi đăng nhập thất bại), "profile" (nơi thực hiện logout).	[CODE] App.js:203, 168
output_cart_state	Output	Array	Giỏ hàng sau khi Đăng xuất: Kỳ vọng sạch []; thực tế bị giữ nguyên các món cũ.	[CODE] App.js:161-169

Bước 2: Phân hoạch các lớp tương đương (Equivalence Partitioning)

Mã lớp (EC ID)	Biến liên quan	Mô tả lớp tương đương	Phân loại	Quy tắc áp dụng	Nguồn dữ kiện (Source Tag)
EC01	input_email	Email hợp lệ đã tồn tại trong CSDL (VD: test@eshop.com)	**Valid**	Giá trị đại diện hợp lệ	[SPEC], [CODE] server.js:35
EC02	input_email	Email hợp lệ nhưng chưa từng đăng ký trong CSDL (VD: unregistered_user_99@domain.com)	**Invalid**	Dữ liệu không tồn tại (State conflict)	[CODE] server.js:37
EC03	input_email	Email là chuỗi rỗng ""	**Invalid**	Rỗng / Empty string	[SPEC], [CODE]
EC04	input_email	Email chỉ chứa khoảng trắng " "	**Invalid**	Whitespace-only string	[SPEC], [CODE]
EC05	input_email	Email sai quy cách định dạng (thiếu @, thiếu tên miền)	**Invalid**	Format violation	[SPEC]
EC06	input_email	Trường email bị thiếu (undefined / không truyền)	**Invalid**	Missing field	[CODE]
EC07	input_email	Trường email sai kiểu dữ liệu (Number 12345, Boolean true)	**Invalid**	Wrong data type	[CODE]
EC08	input_password	Mật khẩu chính xác khớp với mật khẩu tài khoản trong CSDL (VD: Test1234!)	**Valid**	Mật khẩu đúng	[SPEC], [CODE] server.js:46
EC09	input_password	Mật khẩu không chính xác / sai mật khẩu (VD: WrongPassword123!)	**Invalid**	Mật khẩu sai	[CODE] server.js:53
EC10	input_password	Mật khẩu là chuỗi rỗng ""	**Invalid**	Rỗng / Empty string	[SPEC], [CODE]
EC11	input_password	Mật khẩu chỉ chứa khoảng trắng " "	**Invalid**	Whitespace-only string	[SPEC], [CODE]
EC12	input_password	Trường password bị thiếu (undefined / không truyền)	**Invalid**	Missing field	[CODE]
EC13	input_password	Trường password sai kiểu dữ liệu (Number 123456, Boolean false)	**Invalid**	Wrong data type	[CODE]
EC14	state_account	Tài khoản ở trạng thái bình thường (login_attempts = 0, không bị khóa)	**Valid**	Trạng thái tài khoản chuẩn	[CODE] server.js:40
EC15	state_account	Tài khoản đang bị khóa tạm thời (locked_until > now)	**Invalid**	Trạng thái tài khoản bị khóa	[CODE] server.js:40-44

Mã lớp (EC ID)	Biến liên quan	Mô tả lớp tương đương	Phân loại	Quy tắc áp dụng	Nguồn dữ kiện (Source Tag)
EC16	`state_account`	Tài khoản cận điểm khóa: Đã sai 1 lần (`login_attempts = 2`), sai thêm lần này sẽ kích hoạt khóa	**Invalid**	Trạng thái ngưỡng chuyển đổi	`[CODE] server.js:54-58`
EC17	`state_session`	Đăng xuất khi đang có phiên đăng nhập hợp lệ (`token != ""`)	**Valid**	Đăng xuất người dùng bình thường	`[CODE] App.js:161`
EC18	`state_session`	Thao tác Đăng xuất khi đang là khách vãng lai (Guest - chưa đăng nhập)	**Invalid**	Xung đột ngữ cảnh trạng thái	`[CODE] App.js:161`
EC19	`state_cart`	Đăng xuất khi giỏ hàng rỗng (`cart = []`)	**Valid**	Trạng thái giỏ hàng rỗng	`[CODE] App.js:161`
EC20	`state_cart`	Đăng xuất khi giỏ hàng đang có sản phẩm (`cart.length > 0`)	**Invalid**	Lỗ hổng rò rỉ dữ liệu phiên giỏ hàng	`[CODE] App.js:161-169`
EC21	Output	Đăng nhập thành công: HTTP 200, cấp Token JWT, chuyển về `home`	**Valid**	Kết quả mong đợi thành công	`[SPEC]`, `[CODE] server.js:52`
EC22	Output	Đăng nhập thất bại do sai thông tin: HTTP 401, không cấp token	**Invalid**	Kết quả mong đợi thất bại 401	`[SPEC]`, `[CODE] server.js:63`
EC23	Output	Đăng nhập bị chặn do tài khoản bị khóa: HTTP 403, báo lỗi tài khoản bị khóa	**Invalid**	Kết quả mong đợi từ chối 403	`[CODE] server.js:42`
EC24	Output	Đăng xuất thành công: Xóa sạch `token`, `user`, dọn dẹp giỏ hàng, chuyển về `home`	**Valid**	Kết quả mong đợi đăng xuất	`[SPEC]`, `[CODE] App.js:168`

Bước 3: Chọn giá trị đại diện và lý giải kỹ thuật

Mã lớp	Biến	Giá trị đại diện tiêu biểu	Lý do lựa chọn
EC01	`input_email`	`"test@eshop.com"`	Email đã được seed sẵn trong CSDL SUT với vai trò Customer
EC02	`input_email`	`"unregistered_user_99@domain.com"`	Email đúng cú pháp nhưng chưa từng được đăng ký trong hệ thống
EC03	`input_email`	`""`	Chuỗi rỗng độ dài 0 ký tự
EC04	`input_email`	`" "`	Chuỗi gồm 3 ký tự khoảng trắng
EC05	`input_email`	`"invalid-email-format"`	Chuỗi thiếu ký tự `@` và thiếu domain
EC06	`input_email`	`undefined` (omitted in payload)	Trường bị thiếu hoàn toàn trong request JSON
EC07	`input_email`	`123456`	Số nguyên thay vì chuỗi văn bản
EC08	`input_password`	`"Test1234!"`	Mật khẩu chính xác tương ứng với tài khoản `test@eshop.com`
EC09	`input_password`	`"WrongPass123!"`	Mật khẩu sai có độ dài chuẩn
EC10	`input_password`	`""`	Chuỗi rỗng
EC11	`input_password`	`" "`	Chuỗi khoảng trắng
EC12	`input_password`	`undefined` (omitted in payload)	Trường bị thiếu trong request
EC13	`input_password`	`false`	Kiểu dữ liệu Boolean thay vì String
EC14	`state_account`	`login_attempts = 0, locked_until = null`	Trạng thái hoạt động bình thường
EC15	`state_account`	`locked_until = now + 180000ms`	Tài khoản đang trong thời hạn bị khóa
EC16	`state_account`	`login_attempts = 2`	Trạng thái cận điểm khóa (đã sai 1 lần)

Mã lớp	Biến	Giá trị đại diện tiêu biểu	Lý do lựa chọn
EC17	`state_session`	`token = "Bearer <valid_jwt>"`	Đang có phiên đăng nhập hợp lệ
EC18	`state_session`	`token = ""`	Phiên khách vãng lai (Guest)
EC19	`state_cart`	`cart = []`	Giỏ hàng rỗng
EC20	`state_cart`	`[{"id": 1, "name": "iPhone 15", "quantity": 1}]`	Giỏ hàng có chứa sản phẩm
EC21	Output	HTTP 200 OK + JWT Token	Đăng nhập thành công
EC22	Output	HTTP 401 Unauthorized	Thông tin sai
EC23	Output	HTTP 403 Forbidden	Tài khoản bị khóa
EC24	Output	`token = "", user = null, view = "home"`	Đăng xuất hoàn tất

Bước 4: Thiết kế ca kiểm thử tối thiểu và Ma trận bao phủ (Coverage Matrix)

Mã TC (Test ID)	Tiêu đề ca kiểm thử	Loại kiểm thử	Dữ liệu đầu vào (Payload / Input)	Trạng thái tiền điều kiện	Kết quả mong đợi	Lớp tương đương bao phủ
FR20-DT-01	Đăng nhập thành công với tài khoản User hợp lệ	API / UI	`email: "test@eshop.com", password: "Test1234!"`	User tồn tại, chưa bị khóa	HTTP 200 OK, trả về Token JWT, điều hướng về Home	`EC01`, `EC08`, `EC14`, `EC21`
FR20-DT-02	Đăng nhập thành công với tài khoản Admin hợp lệ	API / UI	`email: "admin@eshop.com", password: "Admin123!"`	Admin tồn tại, chưa bị khóa	HTTP 200 OK, trả về Token JWT với role='admin'	`EC01`, `EC08`, `EC14`, `EC21`
FR20-DT-03	Đăng nhập thất bại với email chưa từng đăng ký	API / UI	`email: "unregistered_user_99@domain.com", password: "Test1234!"`	Email không có trong CSDL	HTTP 401 Unauthorized, thông báo lỗi xác thực	`EC02`, `EC08`, `EC22`
FR20-DT-04	Đăng nhập với email là chuỗi rỗng `""`	API / UI	`email: "", password: "Test1234!"`	CSDL có sẵn	HTTP 400 Bad Request (hoặc 401 từ chối)	`EC03`, `EC08`, `EC22`
FR20-DT-05	Đăng nhập với email chỉ chứa khoảng trắng `" "`	API / UI	`email: " ", password: "Test1234!"`	CSDL có sẵn	HTTP 400 Bad Request (hoặc 401 từ chối)	`EC04`, `EC08`, `EC22`
FR20-DT-06	Đăng nhập với email sai quy cách định dạng	API / UI	`email: "invalid-email-format", password: "Test1234!"`	CSDL có sẵn	HTTP 400 Bad Request (hoặc 401 từ chối)	`EC05`, `EC08`, `EC22`
FR20-DT-07	Đăng nhập khi thiếu trường `email` trong payload	API	`{ "password": "Test1234!" }`	CSDL có sẵn	HTTP 400 Bad Request	`EC06`, `EC08`, `EC22`
FR20-DT-08	Đăng nhập khi trường `email` sai kiểu dữ liệu số	API	`{ "email": 123456, "password": "Test1234!" }`	CSDL có sẵn	HTTP 400 Bad Request	`EC07`, `EC08`, `EC22`

Mã TC (Test ID)	Tiêu đề ca kiểm thử	Loại kiểm thử	Dữ liệu đầu vào (Payload / Input)	Trạng thái tiền điều kiện	Kết quả mong đợi	Lớp tương đương bao phủ
FR20-DT-09	Đăng nhập thất bại do mật khẩu không chính xác	API / UI	`email: "test@eshop.com", password: "WrongPass123!"`	User tồn tại (`login_attempts=0`)	HTTP 401 Unauthorized, tăng attempts lên 2	`EC01`, `EC09`, `EC14`, `EC22`
FR20-DT-10	Đăng nhập với mật khẩu là chuỗi rỗng `""`	API / UI	`email: "test@eshop.com", password: ""`	User tồn tại	HTTP 400 Bad Request	`EC01`, `EC10`, `EC22`
FR20-DT-11	Đăng nhập với mật khẩu chỉ chứa khoảng trắng `" "`	API / UI	`email: "test@eshop.com", password: " "`	User tồn tại	HTTP 400 Bad Request	`EC01`, `EC11`, `EC22`
FR20-DT-12	Đăng nhập khi thiếu trường `password` trong payload	API	`{ "email": "test@eshop.com" }`	User tồn tại	HTTP 400 Bad Request	`EC01`, `EC12`, `EC22`
FR20-DT-13	Đăng nhập khi trường `password` sai kiểu Boolean	API	`{ "email": "test@eshop.com", "password": false }`	User tồn tại	HTTP 400 Bad Request	`EC01`, `EC13`, `EC22`
FR20-DT-14	Kiểm tra cơ chế khóa tài khoản khi gõ sai lần 2 liên tiếp	API / Logic	`email: "test@eshop.com", password: "WrongSecondTime!"`	Tài khoản đã sai 1 lần (`attempts=2`)	HTTP 401, CSDL cập nhật `locked_until != null`	`EC01`, `EC09`, `EC16`, `EC22`
FR20-DT-15	Đăng nhập bị từ chối khi tài khoản đang bị khóa	API / UI	`email: "test@eshop.com", password: "Test1234!"`	Tài khoản có `locked_until > now`	HTTP 403 Forbidden ("Tài khoản đã bị khóa")	`EC01`, `EC08`, `EC15`, `EC23`
FR20-DT-16	Kiểm tra an toàn dữ liệu: Không rò rỉ mật khẩu trong response đăng nhập	API / Sec	`email: "test@eshop.com", password: "Test1234!"`	Đăng nhập thành công	Response body KHÔNG được chứa trường `password`	`EC01`, `EC08`, `EC21`
FR20-DT-17	Mobile Logout: Đăng xuất người dùng bình thường khi giỏ hàng rỗng	Mobile UI	Bấm nút Thoát (`logout()`)	User đã đăng nhập, `cart = []`	Xóa Token, User = null, chuyển về màn hình Home	`EC17`, `EC19`, `EC24`
FR20-DT-18	Mobile Logout: Kiểm tra dọn dẹp giỏ hàng khi người dùng có sản phẩm	Mobile UI	Bấm nút Thoát (`logout()`)	User đã đăng nhập, `cart.length = 2`	Giỏ hàng phải được làm sạch (`cart = []`)	`EC17`, `EC20`, `EC24`
FR20-DT-19	Mobile UI: Kiểm tra nhãn trường Email và ngôn ngữ nút bấm	Mobile UI	Xem màn hình Login	Giao diện hiển thị	Nhãn là "Email", nút bấm là "Đăng nhập" (tiếng Việt)	`EC01`, `EC08`

Mã TC (Test ID)	Tiêu đề ca kiểm thử	Loại kiểm thử	Dữ liệu đầu vào (Payload / Input)	Trạng thái tiền điều kiện	Kết quả mong đợi	Lớp tương đương bao phủ
FR20-DT-20	Mobile Logout: Thao tác đăng xuất khi chưa có phiên đăng nhập	Mobile UI	Gọi `logout()` ở trạng thái khách	`token = "", user = null`	Không gây crash ứng dụng, duy trì trạng thái an toàn	`EC18`, `EC24`

Ma trận bao phủ: Toàn bộ 24 lớp tương đương (`EC01` đến `EC24`) đều được kiểm thử ít nhất một lần (**100% Equivalence Partition Coverage**).

3.6.3 Phân tích giá trị biên (Boundary Value Analysis - BVA)

Bước 1: Xác định miền có thứ tự và biến áp dụng/không áp dụng

Miền kiểm thử (Variable / Attribute)	Bản chất miền	Khoảng hợp lệ đóng `[LB, UB]`	Dạng bất đẳng thức mở	Bước nhảy nhỏ nhất (Smallest Step)	Nguồn dữ kiện (Source Tag)
`password.length` (Độ dài mật khẩu)	Độ dài chuỗi (Characters)	`[8, 128]`	`8 <= length <= 128`	1 ký tự	`[SPEC]` §1.1, §1.2
`email.length` (Độ dài email)	Độ dài chuỗi (Characters)	`[6, 254]`	`6 <= length <= 254`	1 ký tự	`[SPEC]` RFC 5321/5322
`login_attempts` (Số lần đăng nhập sai)	Số nguyên đếm (Integer count)	`[0, 2]`	`0 <= attempts < 3`	1 lần	`[CODE] server.js:54-58`
`locked_until` (Thời hạn khóa tài khoản)	Mốc thời gian (Timestamp)	`[now - ∞, now]`	`locked_until <= now`	1 giây (1s)	`[CODE] server.js:40`

Các biến không áp dụng BVA: `user.role` (Enum string: admin, user); `token` (JWT cryptographic hash vô hướng); `email format` (Regex grammar); `session_state` (Binary state).

Bước 2: Lựa chọn biến thể BVA 3-value và Mô hình 9 điểm

- Áp dụng kỹ thuật **3-value BVA** ($LB-1, LB, LB+1, UB-1, UB, UB+1$) khảo sát 3 giá trị tại mỗi biên.
- Áp dụng **Mô hình 9 điểm** kiểm thử các điểm cực trị: Điểm cực tiểu vật lý ($-alpha$: chuỗi rỗng độ dài 0) và Điểm cực đại vật lý ($+alpha$: chuỗi DoS 10,000 ký tự).
- Áp dụng **Quy tắc mã hóa tiếng Việt UTF-8 (v2.0 Encoding Rule)**: Chuỗi 128 ký tự tiếng Việt có dấu (kích thước thực tế hơn 250 bytes) tại cận trên UB để kiểm tra sự chênh lệch đo lường giữa ký tự và byte trong mã nguồn backend.

Bước 3: Thiết kế các ca kiểm thử BVA (14 ca kiểm thử)

Mã TC	Biến kiểm thử	Điểm biên	Giá trị kiểm thử (Value)	Các tham số cố định	Kết quả mong đợi	Lỗi phần mềm code sẽ bắt được (Fault it catches)	Nguồn
FR20-BVA-01	`password.length`	$LB - 1$ (7 ký tự)	`"Pass12!"` (7 chars)	`email: "test@eshop.com"`	HTTP 400 Bad Request ("Mật khẩu tối thiểu 8 ký tự")	Backend không kiểm tra độ dài tối thiểu của mật khẩu khi đăng nhập	`[SPEC]`

Mã TC	Biến kiểm thử	Điểm biên	Giá trị kiểm thử (Value)	Các tham số cố định	Kết quả mong đợi	Lỗi phần mềm code sẽ bắt được (Fault it catches)	Nguồn
FR20-BVA-02	`password.length`	LB (8 ký tự)	`"Pass123!"` (8 chars)	`email: "test@eshop.com"`	HTTP 200 OK (Đăng nhập thành công)	Nhầm lẫn dấu so sánh `>` thay vì `>=` tại cận dưới	`[SPEC]`
FR20-BVA-03	`password.length`	$LB + 1$ (9 ký tự)	`"Pass1234!"` (9 chars)	`email: "test@eshop.com"`	HTTP 200/401 xử lý chuẩn	Off-by-one lỗi lệch 1 đơn vị	`[SPEC]`
FR20-BVA-04	`password.length`	$UB - 1$ (127 ký tự)	Chuỗi ASCII 127 ký tự	`email: "test@eshop.com"`	HTTP 200/401 xử lý chuẩn	Tràn cận trên chuỗi mật khẩu	`[SPEC]`
FR20-BVA-05	`password.length`	UB (128 ký tự)	Chuỗi ASCII 128 ký tự	`email: "test@eshop.com"`	HTTP 200/401 xử lý chuẩn	Nhầm lẫn dấu so sánh `<` thay vì `<=` tại cận trên	`[SPEC]`
FR20-BVA-06	`password.length`	$UB + 1$ (129 ký tự)	Chuỗi ASCII 129 ký tự	`email: "test@eshop.com"`	HTTP 400 Bad Request ("Mật khẩu vượt quá 128 ký tự")	Thiếu ràng buộc chặn mật khẩu quá dài gây tốn chi phí hash Bcrypt/Argon	`[SPEC]`
FR20-BVA-07	`password.length`	Encoding UTF-8 v2.0	Chuỗi 128 ký tự tiếng Việt có dấu (`"MậtKhẩuViệtNam..."` = 250+ bytes)	`email: "test@eshop.com"`	Xử lý an toàn UTF-8	Backend đo byte length thay vì char length gây cắt cụt mật khẩu	`[SPEC]` v2.0
FR20-BVA-08	`password.length`	$+ alpha$ Extreme	Chuỗi DoS 10,000 ký tự	`email: "test@eshop.com"`	HTTP 400 Bad Request (Payload Too Large)	Lỗ hổng DoS máy chủ qua payload xác thực cực lớn	`[CODE]`
FR20-BVA-09	`email.length`	$LB - 1$ (5 ký tự)	`"a@b.c"` (5 chars)	`password: "Test1234!"`	HTTP 400 Bad Request (Email quá ngắn)	Thiếu kiểm tra độ dài email hợp lệ	`[SPEC]`
FR20-BVA-10	`email.length`	LB (6 ký tự)	`"a@b.co"` (6 chars)	`password: "Test1234!"`	Xử lý định dạng tối thiểu	Lỗi cắt cụt email cận dưới	`[SPEC]`

Mã TC	Biến kiểm thử	Điểm biên	Giá trị kiểm thử (Value)	Các tham số cố định	Kết quả mong đợi	Lỗi phần mềm code sẽ bắt được (Fault it catches)	Nguồn
FR20-BVA-11	`email.length`	UB (254 ký tự)	Chuỗi Email RFC 254 ký tự	`password: "Test1234!"`	Xử lý hợp lệ	Lỗi kích thước trường email trong CSDL SQLite	`[SPEC]`
FR20-BVA-12	`email.length`	$UB + 1$ (255 ký tự)	Chuỗi Email 255 ký tự	`password: "Test1234!"`	HTTP 400 Bad Request (Email vượt quá giới hạn)	Thiếu validation độ dài tối đa email	`[SPEC]`
FR20-BVA-13	`login_attempts`	Ngưỡng khóa SUT	Gõ sai mật khẩu lần 2 (`newAttempts = 4 >= 3`)	`email: "test@eshop.com"`	Khóa tài khoản ngay tại lần gõ sai thứ 2	Lỗi logic `+2` mỗi lần sai khiến ngưỡng 3 lần bị kích hoạt sớm ở lần 2	`[CODE] server.js:54`
FR20-BVA-14	`locked_until`	Time Boundary (Hết khóa)	Đăng nhập sau khi hết hạn khóa 1 giây (`now - 1s`)	`email: "test@eshop.com"`	HTTP 200 OK (Mở khóa đăng nhập lại)	So sánh thời gian `<` vs `<=` khiến tài khoản bị khóa vĩnh viễn	`[CODE] server.js:40`

3.6.4 Kết quả thực thi kiểm thử chi tiết (Execution Results)

Toàn bộ **34 ca kiểm thử** của FR-20 đã được thực thi tự động qua test runner `scratch/execute_fr20.mjs` đối chiếu trực tiếp trên hệ thống SUT Backend và Client logic:

- **Đã thiết kế (Designed):** 34
- **Đã thực thi (Executed):** 34 (100%)
- **Vượt qua (PASS):** 31 (91.2%)
- **Thất bại (FAIL):** 3 (8.8%)
- **Bị chặn (BLOCKED):** 0 (0.0%)

Bảng kết quả thực thi chi tiết 34 test cases

Mã TC	Kỹ thuật	Mô tả ca kiểm thử	Kỳ vọng (Expected)	Thực tế (Actual)	Trạng thái
`FR20-DT-01`	DT	Đăng nhập thành công với tài khoản người dùng hợp lệ	HTTP 200, JWT token, user info	HTTP 200, JWT token, user info	**PASS**
`FR20-DT-02`	DT	Đăng nhập thành công với tài khoản quản trị viên (Admin)	HTTP 200, JWT token, role: 'admin'	HTTP 200, JWT token, role: 'admin'	**PASS**
`FR20-DT-03`	DT	Đăng nhập thất bại do email không tồn tại trong hệ thống	HTTP 401, Invalid email or password	HTTP 401, Invalid email or password	**PASS**
`FR20-DT-04`	DT	Đăng nhập với email là chuỗi rỗng	HTTP 400 hoặc 401	HTTP 401, Invalid email or password	**PASS**
`FR20-DT-05`	DT	Đăng nhập với email chỉ chứa khoảng trắng	HTTP 400 hoặc 401	HTTP 401, Invalid email or password	**PASS**

Mã TC	Kỹ thuật	Mô tả ca kiểm thử	Kỳ vọng (Expected)	Thực tế (Actual)	Trạng thái
FR20-DT-06	DT	Đăng nhập với email sai cú pháp định dạng (thiếu @ và domain)	HTTP 400 hoặc 401	HTTP 401, Invalid email or password	**PASS**
FR20-DT-07	DT	Đăng nhập khi thiếu trường email trong payload JSON	HTTP 400 hoặc 401	HTTP 401, Invalid email or password	**PASS**
FR20-DT-08	DT	Đăng nhập khi trường email sai kiểu dữ liệu số	HTTP 400 hoặc 401	HTTP 401, Invalid email or password	**PASS**
FR20-DT-09	DT	Đăng nhập thất bại do mật khẩu không chính xác	HTTP 401, Invalid email or password	HTTP 401, Invalid email or password	**PASS**
FR20-DT-10	DT	Đăng nhập với mật khẩu là chuỗi rỗng	HTTP 400 hoặc 401	HTTP 401, Invalid email or password	**PASS**
FR20-DT-11	DT	Đăng nhập với mật khẩu chỉ chứa khoảng trắng	HTTP 400 hoặc 401	HTTP 401, Invalid email or password	**PASS**
FR20-DT-12	DT	Đăng nhập khi thiếu trường password trong payload	HTTP 400 hoặc 401	HTTP 401, Invalid email or password	**PASS**
FR20-DT-13	DT	Đăng nhập khi trường password sai kiểu Boolean	HTTP 400 hoặc 401	HTTP 401, Invalid email or password	**PASS**
FR20-DT-14	DT	Kiểm tra cơ chế khóa tài khoản khi gõ sai lần 2 liên tiếp	HTTP 401, ghi nhận vi phạm	HTTP 401, Invalid email or password	**PASS**
FR20-DT-15	DT	Đăng nhập bị từ chối khi tài khoản đang trong trạng thái bị khóa	HTTP 403, thông báo tài khoản bị khóa	HTTP 403, Tài khoản đã bị khóa...	**PASS**
FR20-DT-16	DT	Kiểm tra an toàn bảo mật: Không rò rỉ mật khẩu trong phản hồi	Response body KHÔNG chứa `"password":`	Body CHỨA trường `"password":"Password123!"`	**FAIL**
FR20-DT-17	DT	Mobile Logout: Đăng xuất người dùng khi giỏ hàng rỗng	Reset token, user, orders về null/rỗng	Session và tokens được reset về null	**PASS**
FR20-DT-18	DT	Mobile Logout: Kiểm tra dọn dẹp giỏ hàng khi người dùng có hàng	Giỏ hàng phải được reset `cart = []`	Bỏ quên `setCart([])`, giỏ hàng tồn lưu 2 sản phẩm	**FAIL**
FR20-DT-19	DT	Mobile UI: Kiểm tra nhãn trường Email và ngôn ngữ nút bấm tiếng Việt	Nhãn: "Email", Nút: "Đăng nhập", Lỗi trên nút	Nhãn: "Username", Nút: "Sign In", Lỗi dưới nút	**FAIL**
FR20-DT-20	DT	Mobile Logout: Thao tác đăng xuất khi chưa đăng nhập (Guest)	Ứng dụng an toàn, không crash ngoại lệ	Thực thi an toàn, chuyển view 'login'	**PASS**
FR20-BVA-01	BVA	Cận dưới mật khẩu LB-1: Độ dài 7 ký tự (Invalid)	HTTP 401, từ chối đăng nhập	HTTP 401, Invalid email or password	**PASS**
FR20-BVA-02	BVA	Cận dưới mật khẩu LB: Độ dài chuẩn 8 ký tự (Valid)	HTTP 200, đăng nhập thành công	HTTP 200, Login successful	**PASS**
FR20-BVA-03	BVA	Cận dưới mật khẩu LB+1: Độ dài 9 ký tự (Valid)	HTTP 401 (mật khẩu không khớp)	HTTP 401, Invalid email or password	**PASS**
FR20-BVA-04	BVA	Cận trên mật khẩu UB-1: Độ dài 127 ký tự (Valid)	HTTP 401 (mật khẩu không khớp)	HTTP 401, Invalid email or password	**PASS**
FR20-BVA-05	BVA	Cận trên mật khẩu UB: Độ dài 128 ký tự (Valid)	HTTP 401 (mật khẩu không khớp)	HTTP 401, Invalid email or password	**PASS**

Mã TC	Kỹ thuật	Mô tả ca kiểm thử	Kỳ vọng (Expected)	Thực tế (Actual)	Trạng thái
FR20-BVA-06	BVA	Cận trên mật khẩu UB+1: Độ dài 129 ký tự (Invalid)	HTTP 401 (từ chối do quá dài)	HTTP 401, Invalid email or password	**PASS**
FR20-BVA-07	BVA	Encoding Rule v2.0: Mật khẩu chứa chuỗi tiếng Việt UTF-8	HTTP 401 (xử lý byte an toàn)	HTTP 401, Invalid email or password	**PASS**
FR20-BVA-08	BVA	Physical Extreme (+alpha): Mật khẩu DoS payload 10,000 ký tự	HTTP 401 (hoặc 413, không crash server)	HTTP 401, Invalid email or password	**PASS**
FR20-BVA-09	BVA	Cận dưới Email LB-1: Độ dài 5 ký tự (Invalid)	HTTP 401, từ chối đăng nhập	HTTP 401, Invalid email or password	**PASS**
FR20-BVA-10	BVA	Cận dưới Email LB: Độ dài tối thiểu 6 ký tự `a@b.co` (Valid)	HTTP 401 (email không tồn tại)	HTTP 401, Invalid email or password	**PASS**
FR20-BVA-11	BVA	Cận trên Email UB: Chuỗi Email chuẩn RFC 254 ký tự (Valid)	HTTP 401 (email không tồn tại)	HTTP 401, Invalid email or password	**PASS**
FR20-BVA-12	BVA	Cận trên Email UB+1: Chuỗi Email 255 ký tự vượt giới hạn RFC	HTTP 401 (hoặc 400, từ chối)	HTTP 401, Invalid email or password	**PASS**
FR20-BVA-13	BVA	Ngưỡng số lần gõ sai: Chuyển đổi trạng thái khi sai lần 1 và 2	HTTP 401 cho lần 1 và 2, chưa bị khóa	HTTP 401, ghi nhận số lần sai	**PASS**
FR20-BVA-14	BVA	Biên thời gian khóa locked_until: Đăng nhập sau hết hạn khóa 1s	HTTP 200, mở khóa tự động thành công	HTTP 200, Login successful	**PASS**

3.6.5 Báo cáo khiếm khuyết phần mềm (Defect Reports)

Mã lỗi (Bug ID)	Tiêu đề lỗi	Các TC phát hiện	Mức độ (Severity)	Độ ưu tiên
BUG-FR20-01	Hàm `logout()` trong Mobile App không dọn dẹp giỏ hàng gây rò rỉ dữ liệu phiên giữa các người dùng (Cross-user Cart Retention)	FR20-DT-18	**High**	High
BUG-FR20-02	API `POST /api/login` trả về mật khẩu người dùng dạng plaintext trong response JSON (Sensitive Data Exposure)	FR20-DT-16	**Critical**	Immediate
BUG-FR20-03	Cơ chế tính số lần đăng nhập sai cộng dồn bước nhảy 2 khiến tài khoản bị khóa bất thường chỉ sau 2 lần sai (Premature Lockout)	FR20-DT-14, FR20-BVA-13	**Medium**	Medium
BUG-FR20-04	Giao diện đăng nhập Mobile App sử dụng nhãn tiếng Anh không đồng nhất và đặt vị trí thông báo lỗi sai chuẩn UI/UX	FR20-DT-19	**Low**	Low

3.6.6 Phân tích khoảng trống của AI (AI Gap Analysis & Nâng cấp kỹ năng v2.3)

Quá trình kiểm thử FR-20 đã làm bộc lộ 4 khoảng trống AI quan trọng:

Mục bị bỏ sót / Thách thức thực tế	Cách phát hiện	Vì sao AI bỏ sót ban đầu	Phân loại nguyên nhân (Category)	Cải tiến kỹ năng (Skill Fix)

Mục bị bỏ sót / Thách thức thực tế	Cách phát hiện	Vì sao AI bỏ sót ban đầu	Phân loại nguyên nhân (Category)	Cải tiến kỹ năng (Skill Fix)
1. Bỏ sót hiệu ứng phụ lưu giữ giỏ hàng khi Logout (Client Session Bleed)	Kiểm tra mã nguồn `App.js:161-169` thấy gọi `setToken(null)`, `setUser(null)`, `setOrders([])` nhưng thiếu `setCart([])`. Chạy thử trên mobile thấy giỏ hàng vẫn còn nguyên.	AI thường tập trung kiểm tra xem token và danh tính người dùng có bị hủy hay không, mà bỏ quên các mảng dữ liệu tạm thời phụ trợ (Auxiliary States) của ứng dụng như giỏ hàng cục bộ.	`Inherent feature complexity`	Bổ sung quy tắc **Mobile Client Session Teardown Checklist**: Trong bất kỳ hàm logout nào, phải kiểm tra danh sách đầy đủ tất cả các React state hook của ứng dụng để đảm bảo mọi dữ liệu tạm thời (cart, draft, cached items) đều được làm sạch.
2. Hiệu ứng khóa dây chuyền trong kiểm thử API có trạng thái (Cascading Lockout)	Khi chạy đợt test đầu tiên, các ca kiểm thử mật khẩu sai làm tài khoản `test@eshop.com` bị khóa, khiến tất cả các test case phía sau đều thất bại với HTTP 403.	AI có xu hướng xem các HTTP request là phi trạng thái (Stateless), không lường trước cơ chế đếm vi phạm cộng dồn trong CSDL backend làm biến đổi trạng thái của fixture test.	`Tool limitation` & `Execution isolation`	Bổ sung quy tắc **Stateful Database Test Isolation Rule**: Trước mỗi ca kiểm thử API đăng nhập độc lập, test runner phải thực hiện lệnh reset trạng thái (`UPDATE users SET login_attempts = 0, locked_until = NULL`) trong CSDL.
3. Điểm mù đối với bước nhảy tăng dần trong cơ chế khóa tài khoản	Đọc mã nguồn `server.js:54`: `(user.login_attempts || 0) + 2` và phát hiện chỉ cần 2 lần sai là đã khóa (0 -> 2 -> 4 >= 3).	AI có thiên kiến kỳ vọng mã nguồn luôn tăng theo bước nhảy chuẩn `+ 1`. Nếu không kiểm tra biểu thức đại số trực tiếp, AI sẽ tin rằng hệ thống luôn cho phép thử 3 lần như đặc tả.	`Prompt/input quality` & `Code analysis gap`	Bổ sung quy tắc **Step Increment & Threshold Audit**: Kiểm tra tỉ mỉ mọi toán tử tăng (`+= 1` vs `+= 2`) liên quan đến các biến đếm kiểm soát hạn mức (Rate-limiting/Lockout counters) so với ngưỡng so sánh.
4. Bỏ sót kiểm tra an toàn dữ liệu nhạy cảm trong phản hồi thành công	Soát kết quả trả về của `POST /api/login` thấy trường `password: "Password123!"` hiển thị trong JSON body.	AI thường chỉ assert mã trạng thái HTTP 200 và sự hiện diện của trường `token` mà không thực hiện kiểm tra loại trừ (Negative Assertion / `body_excludes: ['"password":']`).	`Prompt/input quality`	Bổ sung quy tắc **PII & Credential Negative Assertion Rule**: Mọi bài test xác thực đăng nhập (Login/Register/Profile) bắt buộc phải kiểm tra điều kiện loại trừ không được chứa trường `password` hoặc `hash` trong response payload.

3.6.7 Hạn chế của kỹ thuật

1. **Bản chất trạng thái (Statefulness) và chuỗi sự kiện tích lũy:** Kỹ thuật kiểm thử miền và BVA truyền thống hoạt động dựa trên giả định mỗi lần gọi hàm/request là độc lập (stateless). Tuy nhiên, cơ chế bảo vệ đăng nhập (Account Lockout) lại là một máy trạng thái hữu hạn (Finite State Machine) phụ thuộc vào lịch sử các lần gọi trước đó và thời gian trôi qua (`locked_until`). Để bao phủ trọn vẹn, cần kết hợp thêm kỹ thuật Kiểm thử chuyển trạng thái (State Transition Testing).
2. **Vòng đời phiên làm việc trên Client (Client-Side Session Lifecycle):** BVA chỉ đánh giá dữ liệu đầu vào và đầu ra của từng hàm hoặc endpoint đơn lẻ, hoàn toàn bất lực trước các lỗi kiến trúc quản lý trạng thái trên Client (như việc React state của `cart` không được dọn dẹp khi chuyển đổi giữa các phiên đăng nhập khác nhau). Lỗi này đòi hỏi kiểm thử luồng người dùng đầu cuối (End-to-End User Flow Testing).
3. **Khoảng cách giữa kiểm thử chức năng và kiểm thử an toàn thông tin:** Việc mật khẩu bị trả về dạng plaintext (`BUG-FR20-02`) không hề làm sai lệch chức năng đăng nhập (người dùng vẫn nhận được token và đăng nhập thành công). Kỹ thuật kiểm thử miền chức năng thông thường sẽ chấm điểm PASS cho ca kiểm thử này nếu không có quy chuẩn kiểm thử bảo mật chuyên biệt (Security Vulnerability Testing) nhằm soi quét việc phơi nhiễm dữ liệu nhạy cảm.

3.7 Consolidated Bug Reports

Hiện tại đã ghi nhận và phân loại tổng cộng **23 lỗi phần mềm** (5 lỗi từ FR-01, 4 lỗi từ FR-03, 7 lỗi từ FR-08, 3 lỗi từ FR-12, và 4 lỗi từ FR-20). Toàn bộ lỗi đều được xác thực trực tiếp qua phân tích mã nguồn SUT và thực thi test runner tự động:

Mã lỗi (Bug ID)	Tiêu đề lỗi	Tính năng	Các TC phát hiện	Mức độ (Severity)	Độ ưu tiên	Trạng thái
BUG-FR01-01	Backend thiếu kiểm tra các trường bắt buộc (`name`, `email`, `password` rỗng vẫn tạo user)	FR-01	FR01-DT-02, 03, 06, 07, 14, 15	**Major**	**High**	Confirmed
BUG-FR01-02	Backend không kiểm tra định dạng email và kiểu dữ liệu (nhận boolean/số vẫn trả 200)	FR-01	FR01-DT-04, 05, 08	**Major**	**High**	Confirmed
BUG-FR01-03	Cho phép đăng ký tài khoản trùng lặp email do thiếu ràng buộc UNIQUE trong DB	FR-01	FR01-DT-16	**Critical**	**Immediate**	Confirmed
BUG-FR01-04	Backend không kiểm tra độ dài tối thiểu và tiêu chí độ phức tạp của mật khẩu	FR-01	FR01-DT-09..13, FR01-BVA-07	**Major**	**High**	Confirmed
BUG-FR01-05	Không có giới hạn cận trên cho độ dài họ tên và mật khẩu (nguy cơ DoS payload lớn)	FR-01	FR01-BVA-05, 06, 12, 13	**Minor**	**Medium**	Confirmed
BUG-FR03-01	Lộ mã OTP (`resetToken`) trong HTTP response body của `POST /api/forgot-password`	FR-03	FR03-DT-01	**Critical**	**Immediate**	Confirmed
BUG-FR03-02	Backend hoàn toàn không kiểm tra tính hợp lệ và độ phức tạp của `newPassword`	FR-03	FR03-DT-16..22, FR03-BVA-01, 02, 08, 09	**Critical**	**High**	Confirmed
BUG-FR03-03	Mã đặt lại mật khẩu không có thời hạn hết hạn (Token Never Expires do DB thiếu timestamp)	FR-03	Khảo sát `database.js:58`	**Major**	**Medium**	Confirmed
BUG-FR03-04	Endpoint `POST /api/forgot-password` trả về HTTP 404 thay vì HTTP 400 khi email sai quy cách	FR-03	FR03-DT-03..07	**Minor**	**Low**	Confirmed
BUG-FR08-01	Backend hoàn toàn thiếu validation cho trường `total_amount` (chấp nhận tiền âm, 0đ, chuỗi, boolean, float, null, tràn số int32)	FR-08	FR08-DT-06..13, FR08-BVA-01, 02, 07, 08	**Critical**	**Immediate**	Confirmed
BUG-FR08-02	Lỗ hổng Parameter Tampering: Backend không tính lại tổng tiền từ giỏ hàng; Frontend Web cung cấp ô input tự sửa tiền	FR-08	FR08-DT-08	**Critical**	**Immediate**	Confirmed
BUG-FR08-03	Backend thiếu validation `shipping_address`; Frontend Web và Mobile không có ô nhập địa chỉ giao hàng	FR-08	FR08-DT-14..18, FR08-BVA-09, 10, 16, 17	**Major**	**High**	Confirmed
BUG-FR08-04	Backend không kiểm tra trạng thái giỏ hàng khi checkout (Cho phép đặt đơn hàng ma khi giỏ rỗng `cart = []`)	FR-08	FR08-DT-19, FR08-BVA-18	**Major**	**High**	Confirmed
BUG-FR08-05	Dị biệt logic nghiêm trọng trên Mobile: Frontend Mobile tự ý cắt xén sản phẩm cuối cùng trong giỏ hàng khi checkout (`cart.slice(0, -1)`)	FR-08	FR08-DT-22, FR08-BVA-20	**Critical**	**Immediate**	Confirmed
BUG-FR08-06	Backend không kiểm tra tồn kho (Overselling Defect) và không lưu chi tiết các mặt hàng trong đơn hàng (`order_items`)	FR-08	FR08-DT-20	**Major**	**High**	Confirmed

Mã lỗi (Bug ID)	Tiêu đề lỗi	Tính năng	Các TC phát hiện	Mức độ (Severity)	Độ ưu tiên	Trạng thái
BUG-FR08-07	Endpoint checkout không kiểm tra tính hợp lệ của mã giảm giá `coupon_id`	FR-08	FR08-DT-21	**Minor**	**Medium**	Confirmed
BUG-FR12-01	Backend middleware `authenticateToken` không kiểm tra vai trò admin và các route quản trị thiếu role check	FR-12	FR12-DT-07, 08, 13, 15, 18	**Critical**	**Immediate**	Confirmed
BUG-FR12-02	Endpoint quản lý sản phẩm `POST /api/products`, `PUT/DELETE` bỏ quên middleware `authenticateToken`	FR-12	FR12-DT-16, FR12-BVA-11..14	**Critical**	**Immediate**	Confirmed
BUG-FR12-03	Lỗ hổng gán tham số hàng loạt (Mass Assignment) tại `PUT /api/users/me` cho phép user tự nâng quyền thành Admin	FR-12	FR12-DT-11	**High**	**High**	Confirmed
BUG-FR20-01	Hàm logout() trong Mobile App không dọn dẹp giỏ hàng gây rò rỉ dữ liệu phiên giữa các người dùng (Cross-user Cart Retention)	FR-20	FR20-DT-18	**High**	**High**	Confirmed
BUG-FR20-02	API POST /api/login trả về mật khẩu người dùng dạng plaintext trong response JSON (Sensitive Data Exposure)	FR-20	FR20-DT-16	**Critical**	**Immediate**	Confirmed
BUG-FR20-03	Cơ chế tính số lần đăng nhập sai cộng dồn bước nhảy 2 khiến tài khoản bị khóa bất thường chỉ sau 2 lần sai (Premature Lockout)	FR-20	FR20-DT-14, FR20-BVA-13	**Medium**	**Medium**	Confirmed
BUG-FR20-04	Giao diện đăng nhập Mobile App sử dụng nhãn tiếng Anh không đồng nhất và đặt vị trí thông báo lỗi sai chuẩn UI/UX	FR-20	FR20-DT-19	**Low**	**Low**	Confirmed

4. **Test Summary Report**

Bảng tổng hợp số liệu kiểm thử được kết xuất tự động từ công cụ `node .claude/skills/test-execution-and-bugs/scripts/summary.mjs HW2/testcases`:

Feature	Designed	Executed	Passed	Failed	Blocked	Not executed
FR-01 (Account registration)	30	30	10	20	0	0
FR-03 (Forgot & Reset password)	37	37	21	16	0	0
FR-08 (Checkout)	42	42	16	26	0	0
FR-12 (Access control)	32	32	21	11	0	0
FR-20 (Mobile Login & Logout)	34	34	31	3	0	0
Tổng cộng (Total)	**175**	**175**	**99**	**76**	**0**	**0**